A U T O N O M O U S  C Y B E R  A G E N T

# 攻防一体自主智能体

基于 LangGraph 的设计与工程实现 · 原理推导 · 全套开发流程

推理架构：ReAct · Reflexion · Self-Critique 多层自省

能力矩阵：渗透评估 · 流量与样本分析 · 溯源推演 · 蓝队仿真

安全底座：ROE 授权门 · 隔离沙箱 · 熔断审计 · 幻觉抑制

# 目　录

第 1 章　总体设计：目标、边界与架构总览.....1
1.1 设计目标与能力地图.....1
1.2 架构总览.....2
1.3 合规与法律边界.....3
1.4 全书结构导读.....4
1.5 本章速记.....4
第 2 章　LLM 底层机制：从 Token 到函数调用.....6
2.1 Token 与词表：一切成本的计量单位.....6
2.2 采样机制：从概率分布到确定性工程.....8
2.3 上下文管理的底层约束.....9
2.4 函数调用全链路.....11
2.5 结构化输出与受限解码.....14
2.6 流式与中断.....16
2.7 模型能力分层与路由.....17
2.8 提示词工程的工程化.....18
2.9 Agent 视角的 LLM 缺陷清单.....19
2.10 本章速记.....20
第 3 章　推理范式推导：ReAct、Reflexion 与多层自省循环.....23
3.1 ReAct：推理与行动的交织.....23
3.2 Reflexion：语言化反馈作为无需梯度的策略改进.....25

3.3 Self-Critique 与自一致性.....27
3.4 多层自省循环的架构设计.....28
3.5 自省循环的状态转移形式化.....32
3.6 安全 Agent 中的特化.....34
3.7 本章速记.....35
第 4 章 LangGraph 底层原理：Pregel 模型与状态通道.....37
4.1 为什么需要图编排.....37
4.2 Pregel/BSP 执行模型.....38
4.3 状态通道与 reducer.....39
4.4 节点与边.....40
4.5 Send API 动态扇出.....41
4.6 中断与人工审批.....42
4.7 检查点机制.....43
4.8 子图与模块化.....44
4.9 综合示例：StateGraph 定义.....45
4.10 典型错误与排查.....47
4.11 本章速记.....48
第 5 章 多分支状态机设计与状态转移方程.....50
5.1 AgentState 的状态设计.....50
5.2 状态转移方程的形式化.....53
5.3 节点职责表.....54
5.4 状态流转全景.....55
5.5 条件路由函数的形式化.....56

5.6 中断与恢复的状态语义.....57
5.7 状态膨胀控制.....58
5.8 图结构代码示例.....58
5.9 本章速记.....61
第 6 章 数学建模：攻击路径概率与风险量化.....63
6.1 攻击图形式化.....63
6.2 利用成功概率建模.....64
6.3 路径成功概率与 log 域最短路.....66
6.4 贝叶斯证据更新.....69
6.5 蒙特卡洛替代方案.....70
6.6 风险量化与修复优先级.....70
6.7 决策建模与博弈视角.....71
6.8 与项目代码的对应.....72
6.9 失效条件与边界.....72
6.10 本章速记.....73
第 7 章 Token 经济：成本模型与开销控制.....75
7.1 成本模型推导.....75
7.2 上下文预算分配.....76
7.3 模型分级路由的经济学.....78
7.4 缓存与复用.....78
7.5 预算熔断机制.....79
7.6 迭代控制与自省的成本权衡.....79
7.7 并行分支的 token 复利.....80

7.8 度量体系.....80
7.9 与项目代码的对应.....81
7.10 本章速记.....83
第 8 章 MCP 插件集群：协议、热加载与安全治理.....85
8.1 协议分层：MCP 与 function calling 的关系.....85
8.2 工具描述的工程学.....87
8.3 安全 Agent 的 server 设计规范.....87
8.4 插件热加载与治理.....88
8.5 本章速记.....89
第 9 章 情报与知识对接：ATT&CK、CVE 与威胁情报库.....91
9.1 MITRE ATT&CK 的数据模型.....91
9.2 CVE 与 NVD 对接.....92
9.3 威胁情报库的工程消费.....93
9.4 混合检索架构.....94
9.5 本章速记.....94
第 10 章 能力子系统 I：侦察、漏洞挖掘与 POC 验证.....96
10.1 侦察子系统：从公开信息到攻击面清单.....96
10.2 指纹探测子系统.....98
10.3 漏洞挖掘子系统分层.....99
10.4 POC 自主编写与验证.....101
10.5 全链路的授权与审计.....104
10.6 失败降级与熔断.....104
10.7 本章速记.....105

第 11 章 能力子系统 II：流量、样本与逆向分析.....107
11.1 PCAP 深度分析管线.....107
11.2 PCAP 与 LLM 的协作分工.....111
11.3 恶意代码静态分析.....111
11.4 动态沙箱分析.....112
11.5 二进制逆向辅助.....113
11.6 取证衔接与证据固定.....114
11.7 本章速记.....114
第 12 章 能力子系统 III：检测、溯源、推演与对抗仿真.....116
12.1 入侵检测规则生成.....117
12.2 攻击链路溯源.....119
12.3 横向移动路径推演.....119
12.4 红蓝对抗仿真.....119
12.5 防御策略生成.....120
12.6 取证分析.....120
12.7 本章速记.....120
第 13 章 安全架构：权限隔离、沙箱、熔断与审计.....122
13.1 多级权限隔离模型.....122
13.2 隔离沙箱的七道防线.....123
13.3 熔断机制：三态与预算.....124
13.4 审计日志：append-only 与可回放.....125
13.5 纵深防御总图.....126
13.6 本章速记.....126

第 14 章 记忆系统与幻觉抑制.....128
14.1 记忆分层模型.....128
14.2 记忆的污染与遗忘.....129
14.3 幻觉的来源分析.....129
14.4 幻觉抑制七方案.....129
14.5 本章速记.....131
第 15 章 工程体系与全套开发流程.....133
15.1 项目结构总览.....133
15.2 并发调度：Send 扇出与资源控制.....135
15.3 异常回滚与断点续跑.....135
15.4 失败回溯重规划.....135
15.5 插件热加载与版本管理.....136
15.6 从 0 到 1 的九步开发流程.....136
15.7 测试策略与 CI 门禁.....136
15.8 已知边界缺陷清单.....137
15.9 本章速记.....137
第 16 章 横向对比与选型：五类 Agent 体系.....138
16.1 五类体系的架构画像.....139
16.2 LangGraph：控制流的极限表达.....139
16.3 低代码平台（Dify 类）与 Coze：速度换控制.....139
16.4 OpenClaw 类本地 Agent：个人主权与信任边界.....140
16.5 终端 CLI Harness（ZCode 类）：结对范式的极致.....140
16.6 选型决策与演进.....140

16.7 本章速记.....141

*注：本目录由域代码生成。若编辑文档后页码变化，请在目录上点击右键并选择“更新域”，即可刷新页码。*

# 第 1 章 总体设计：目标、边界与架构总览

本书给出一个攻防一体网络安全自主智能体的完整设计与实现，代号 **Aegis**：基于 LangGraph 状态机编排，ReAct + Reflexion + Self-Critique 多层自省循环驱动，MCP 风格插件集群对接 MITRE ATT&CK、CVE 与威胁情报，能力覆盖情报搜集、指纹探测、漏洞挖掘、POC 验证、PCAP 流量分析、恶意代码分析、入侵检测、攻击链溯源、横向路径推演、红蓝对抗仿真、防御策略生成与取证分析。本章先立边界：这是一台**在书面授权范围内工作的评估机器**，授权合规不是附加功能，而是第一优先级的设计约束——它决定了架构中每个安全组件的存在理由。

## 1.1 设计目标与能力地图

### 1.1.1 三条第一性原则

**授权即宪法**：智能体的一切目标类动作（扫描、验证、访问）必须通过 ROE（Rules of Engagement，交战规则）授权门，白名单外的目标与动作直接拒绝，硬约束优先于任务目标——任何"任务需要"都不能成为绕过理由。**证据即事实**：LLM 的任何断言（漏洞存在、路径可达、样本恶意）在独立验证前只是假设；报告中的每条高危发现必须携带可溯源的证据 ID，无证据不报告。**有损优于失控**：预算耗尽、工具熔断、迭代超限时，系统选择降级出部分报告而不是无界重试——可控性优先于完成率。

### 1.1.2 能力地图与阶段映射

| 能力域 | 具体能力 | 状态机阶段 |
| --- | --- | --- |
| 侦察面 | 情报搜集、指纹探测、服务发现 | recon / fingerprint |
| 漏洞面 | 漏洞挖掘、POC 自主编写与验证 | vuln_scan / poc_verify |
| 分析面 | PCAP 深度分析、恶意代码静态与沙箱分析、逆向辅助 | analysis（并行分支） |

| 能力域 | 具体能力 | 状态机阶段 |
| --- | --- | --- |
| 防御面 | 入侵检测规则生成、攻击链溯源、横向路径推演 | analysis（并行分支） |
| 对抗面 | 红蓝对抗仿真、防御策略生成 | blue_sim / report |
| 底座 | 长期记忆、审计回放、多模型评审 | 贯穿全程 |

能力不是并列的函数列表，而是状态机上的节点：侦察产出进入漏洞面作为输入，漏洞候选进入验证面需要人工批准，全部分析在 critic 节点汇聚做证据检查。**能力之间的数据流即攻击链的建模**——这是用图编排而非函数堆叠的根本原因。

### 1.1.3 与人工渗透测试的分工

智能体承担的是：信息收集的穷举、已知模式的匹配与关联、路径的概率计算、报告的结构化生成——高体量、低判断密度的部分。人类专家承担的是：ROE 的制定与审批、高危动作的授权、业务影响的判断、报告的最终背书——低体量、高判断密度的部分。所有高危动作（POC 执行、样本分析）设计为人工审批的中断点，这是人机协同的接口，不是自动化的缺陷。

## 1.2 架构总览

### 1.2.1 分层架构

自底向上五层：**模型层**（OpenAI 兼容接口、分层路由：规划用强模型低温度、抽取用快模型零温度、评审用独立模型集合）；**记忆层**（长期向量记忆：任务摘要、反思教训、发现模式的存取，跨任务复利）；**安全层**（授权门、熔断器、审计日志、沙箱——四件套构成所有动作的强制通道）；**工具层**（MCP 风格插件总线：侦察、指纹、CVE 情报、PCAP、静态分析、路径推演、报告生成，插件可热插拔）；**编排层**（LangGraph StateGraph 多分支状态机：节点为纯函数、Send API 动态扇出、interrupt 人工审批、checkpointer 断点恢复）。

层的依赖是单向的：编排层调用工具层必须穿过安全层，任何绕过总线直连工具的调用在代码评审中视为安全缺陷。这个强约束换来的是**安全性质的全局可证明**：审计日志里必然存在每一次工具调用的记录，因为不存在第二条调用路径。

### 1.2.2 一次任务的生命周期

主链路：任务加载（ROE + 目标）→ 记忆召回（历史经验注入规划）→ 规划器分解步骤 → 侦察（端口服务）→ 指纹（Web 技术栈与安全头）→ 漏洞关联（候选 CVE）→ **人工审批中断**（发现候选 CVE 时挂起，批准后 POC 在隔离沙箱内验证）→ 动态扇出并行分析（PCAP beacon 检测、攻击路径推演、蓝队规则草稿）→ 评审（证据完备性检查 + 多模型评审）→ 反思回环（缺陷转教训，受双重上限约束）→ 报告生成（含 ATT&CK 映射与加固建议）→ 写入长期记忆。三条退出路径：正常完成、审批拒绝（POC 标注未验证后继续）、预算熔断（降级出报告）。

### 1.2.3 多层自省循环在总架构中的位置

层 1 步内自检嵌在每个工具节点（结果与预期比对、异常即熔断计数）；层 2 轨迹级 Reflexion 在 critic→reflect→planner 回环（失败教训转语言化记忆，受最大迭代数与最大反思数约束防发散）；层 3 独立评审团在报告前（与执行模型解耦的多模型评审，规避自我确认偏差）。三层的目标不同：层 1 保单步正确、层 2 保轨迹方向、层 3 保结论可信。

## 1.3 合规与法律边界

### 1.3.1 红线清单

四条不可触碰的边界，以 ROE 的 denied_actions 强制表达：未授权访问（刑法二百八十五条）、破坏计算机信息系统（二百八十六条）、拒绝服务类动作（可用性攻击即使在授权测试中也默认禁止）、数据外带（评估数据的处理边界在授权书中约定）。智能体不得生成绕过这些边界的计划——规划器提示词中的约束是软性的，授权门是硬性的，**安全设计永远假设软约束会被提示注入或模型幻觉突破**。

### 1.3.2 数据治理

评估产生的数据（扫描结果、流量摘要、样本哈希）分级管理：审计日志只存摘要与哈希、样本只在沙箱临时目录存在且容器销毁即清除、报告数据脱敏后才可外发。长期记忆中不存原始凭据与客户敏感信息——记忆是跨任务的，泄露面最大，所以只允许存模式与教训。

### 1.3.3 与等保及行业规范的关系

本智能体的审计日志设计与等保 2.0 的安全审计要求对齐（留存六个月以上、可追溯）；授权门与审批链路可作为 SDL 与攻防演练管理的技术支撑；报告的 ATT&CK 映射与威胁情报对接遵循社区标准格式。合规不是上线后的补丁，而是第 13 章安全架构的组成部分。

## 1.4 全书结构导读

第 2~3 章打地基：LLM 底层机制与三大推理范式的原理推导。第 4~5 章讲编排：LangGraph 的 Pregel 执行模型与本项目多分支状态机的状态转移方程。第 6~7 章是数学：攻击路径概率建模与 Token 经济。第 8~9 章讲插件与情报：MCP 协议、ATT&CK/CVE 对接。第 10~12 章逐个展开能力子系统的设计。第 13~14 章回到底座：沙箱与权限隔离的纵深防御、记忆系统与幻觉抑制方案。第 15 章给全套工程开发流程与边界缺陷汇总。第 16 章横向对比主流框架并给出选型逻辑。全书与开源实现 aegis-agent 一一对应，代码与文档互为验证。

## 1.5 本章速记

- 三条第一性原则：授权即宪法、证据即事实、有损优于失控
- 能力不是函数列表而是状态机节点，能力间数据流即攻击链建模
- 机器管穷举与计算，人管授权与判断，高危动作是人机协同接口
- 五层架构：模型、记忆、安全、工具、编排；跨层调用必须穿安全层
- 安全层四件套：授权门、熔断器、审计日志、沙箱
- 审计的全局可证明性来自唯一调用通道：绕过总线即安全缺陷

- 三层自省：步内自检保单步、Reflexion 保轨迹、评审团保结论
- 反思回环有双重上限：最大迭代数与最大反思数，防自省发散
- 三条退出路径：正常完成、审批拒绝继续、预算熔断降级
- 红线四条以 denied_actions 硬编码：未授权访问、破坏、DoS、数据外带
- 安全设计假设软约束会被突破，硬约束必须在代码路径上不可绕过
- 评估数据分级治理：审计存摘要哈希、样本沙箱即焚、记忆不存敏感
- 审计留存与等保 2.0 对齐，ATT&CK 映射遵循社区标准

# 第 2 章 LLM 底层机制：从 Token 到函数调用

开发生产级安全 Agent，最大的认知风险是把 LLM 当作一个黑盒 API：只看输入输出格式，不理解内部机制。本章自底向上重建这个黑盒：先看 token 与上下文窗口的物理约束，再看采样机制如何决定输出分布，然后进入函数调用、结构化输出与流式协议的全链路，最后给出 Agent 视角的模型缺陷清单。后续章节的自省循环与安全架构，都建立在这些机制之上。

## 2.1 Token 与词表：一切成本的计量单位

理解 token 是理解 LLM 成本与能力边界的第一步。模型从不直接处理字符或单词，所有输入输出都被切分为 token，计费、限速、上下文长度限制全部以 token 为单位。对 Agent 而言，工具文档、系统提示、历史轨迹、工具返回值都要折算成 token 预算，忽略这一层的设计在长轨迹场景会迅速失控。

### 2.1.1 BPE 的直觉与代价模型

现代 LLM 主流使用 BPE（Byte Pair Encoding）或其字节级变体构建词表。其过程是：从最细粒度的字节序列出发，统计语料中相邻符号对的频率，反复把最高频的相邻对合并成新符号，直到达到目标词表规模（数万到数十万）。结果是高频词被整体压缩为单个 token，低频词与随机字符串则退化为多个字节片段。这解释了大量工程现象：模型对常见术语拼写流畅，而对罕见漏洞编号或哈希值的处理既浪费 token 又容易出错。

token 数量直接决定三件事：输入与输出的计费、首 token 延迟（服务端要对整个输入做一次 prefill 计算）、以及每个解码步的成本。安全场景的内容——原始日志、HTTP 报文、扫描器 JSON 输出——普遍字符多而信息密度低，token 放大严重。工程上必须在进入模型前做降噪与抽取，这是后续章节引入解析与预处理层的底层原因之一。

### 2.1.2 中文 token 效率

中文在 tokenizer 中的表现差异极大，取决于词表的构建语料。早期以英文为主的词表（如 GPT-2 的 50257）几乎没有中文词条，常用汉字按 UTF-8 字节展开，一

个汉字往往消耗 2 到 3 个 token；新一代面向多语言的词表（如 GPT-4o 的 o200k_base 以及主流国产模型词表）已把常用汉字压缩到约 1 字 1 token 的量级，部分高频双字词还会合并为单个 token。同一段中文报告，在新旧词表下的 token 数可以相差一倍以上。

在容量规划时，不同内容类型的 token 密度差异巨大，混用同一估算系数会造成成倍的预算偏差。下表给出安全 Agent 常见内容类型的估算基准与主要风险，供预算设计时参考。

| 内容类型 | 估算基准 | 主要风险 |
| --- | --- | --- |
| 中文安全报告 | 约 1 字 1 token 起估 | 词表差异导致翻倍 |
| 英文扫描日志 | 约 4 字符 1 token | JSON 冗余字段占额 |
| 中英混排漏洞库 | 按实测样本校准 | 编号与哈希多 token |
| Base64 与十六进制 | 接近字节级展开 | 极易超出预算 |

表中数字只是量级参考，正确做法是对自己的真实语料抽样实测：取代表性样本，调用计费接口或本地 tokenizer 统计，建立按内容类型的估算系数，并在 CI 中随词表版本更新。任何拍脑袋的 token 预算，都会在长轨迹 Agent 中被放大成稳定性问题。

### 2.1.3 上下文窗口的物理含义

上下文窗口常被描述成模型能记住多少内容，其物理含义更硬：自注意力要求每个位置与所有其他位置交互，单层注意力的计算量约为 $O(n^2 \cdot d)$，序列长度翻倍，注意力计算近似翻 4 倍。FlashAttention 等实现通过分块计算把激活显存从 $O(n^2)$ 降到 $O(n)$，但计算量仍随序列长度平方增长，这决定了长上下文请求的 prefill 时间下限。

另一个硬约束是 KV cache 显存。解码阶段为避免重算历史，每层都要缓存全部历史位置的 Key 与 Value，显存占用约为 2 × 层数 × KV 维度 × 序列长度 × 每元素字节数。以一个 32 层、KV 维度 4096、FP16 存储的 7B 级模型为例，每个 token 约占 2 × 32 × 4096 × 2 = 512 KB，32K 上下文仅 KV cache 就需要约 16 GB；采用 GQA 的模

型按 KV 头数等比缩减。服务侧的并发上限、长上下文加价与 prompt caching 折扣，全部由这两个约束推演而来。

对 Agent 的直接推论是：轨迹每延长一步，下一轮请求都要为全部历史付费。前缀不变的请求可命中服务端 prompt cache，把重复前缀的 prefill 成本大幅降下来——这就是把稳定的 system prompt 与工具文档放在上下文最前部、把易变内容放后部的底层理由。

## 2.2 采样机制：从概率分布到确定性工程

模型每步输出的不是词，而是词表上的概率分布，采样机制决定从分布中如何取值。它常被当作可随意调节的风格旋钮，但对 Agent 而言，它决定行为一致性与可复现性，是工程参数而非审美参数。

### 2.2.1 温度与 top-p 的数学含义

温度 T 对 logits 做缩放后再归一化：$P(v) = \exp(z(v) / T) \div \Sigma_u \exp(z_u / T)$。T 趋近 0 时分布退化为 argmax，即 greedy；T 升高则分布趋于均匀，长尾低概率词被放大。top-p（nucleus sampling）不改变分布形状，而是截断支撑集：取累积概率达到 p 的最小词集合，在集合内重新归一化后采样。温度控制分布的尖锐程度，top-p 控制尾部的截断位置；实现上总是先做温度缩放，再做 top-p 过滤。

对工具调用与结构化输出，长尾 token 几乎总是错误来源：一个 0.1% 概率的错别字就可能让 JSON 解析失败。Agent 的默认选择是低温加适度 top-p，把分布质量用在格式稳定与决策一致上，而不是用在词汇多样性上。

### 2.2.2 greedy、beam 与 Agent 场景的选择

greedy 每步取概率最高的 token，确定性最强，代价是可能陷入重复循环，同一短语无限复读是 greedy 的经典失效。beam search 维护 B 条候选序列并保留总似然最高者，在机器翻译这类有标准短答案的任务上占优；但在开放式长文本生成上会触发似然陷阱：高似然序列倾向于保守与重复，Holtzman 等人 2020 年的工作表明，beam 生成的开放式文本显著退化，top-p 采样反而质量更高。

Agent 场景的推荐是把温度压到 0 到 0.2：工具选择要求同一输入得到同一决策参数生成要求格式稳定，评测回归要求结果可比较。只有在面向人的解释性文本生成上才值得放宽。需要警惕的是，即使温度为 0，服务端批处理调度与 MoE 路由也可能带来偶发的非确定输出，工程协议必须容忍少量偏差而非假设严格一致。

### 2.2.3 seed 与可复现性

seed 固定采样器的随机数流：相同 seed、相同模型版本、相同请求内容，理论上应得到相同输出。它是审计与回归测试的基础，安全 Agent 的每一次关键决策都要能重放。但可复现性是四件事的叠加：低温度或 greedy、固定 seed、锚定模型版本快照、保存原始响应留档。任何一环缺失都会破坏重放，模型热更新、推理内核升级、批处理组合变化，都可能让同 seed 产生不同结果。

实践中应把决策可重放写进系统验收标准：关键路径用低温度加 seed，非关键路径放宽采样以获得多样性，审计问题靠响应留档回答，而不是重新问一次模型。

## 2.3 上下文管理的底层约束

上下文窗口不是无限记忆，而是有物理上限、有注意力偏置的工作区。本节讨论溢出行为、system prompt 的地位与多轮衰减，它们共同决定了后续记忆管理章节的设计空间。

### 2.3.1 溢出行为：报错、截断与滑窗

请求超过上下文上限时，不同实现给出三类行为：直接报错拒绝、静默截断最旧内容、滑动窗口只保留最近部分。对交互聊天，静默截断尚可接受；对安全 Agent，被静默丢弃的可能是早期收集的关键证据或授权边界声明，Agent 却毫不知情地继续推理。框架的默认行为不等于安全行为，必须显式确认所用模型与网关的溢出策略。

正确的做法是自己管理预算：为每类内容分配 token 配额，超限时执行分层压缩——把旧轨迹摘要化为事实清单，保留关键观察的原文摘录，丢弃可由工具重新获得的原始数据。压缩是有损操作，压缩策略要经过评测验证，不能未经测试直接上线。

### 2.3.2 system prompt 的优先地位与局限

现代模型在训练中建立指令分层：system（或 developer）消息的服从优先级高于 user，user 高于 tool 返回内容。这是安全边界的第一道闸，把不可协商的约束放在 system，使其在权重上压过会话中出现的内容。但它是概率意义上的优先，不是硬边界：长上下文稀释、大量工具输出的干扰、精心构造的注入内容，都会实际削弱 system 约束的效力。

位置也有讲究。Liu 等人 2023 年的 **lost in the middle** 研究显示，模型对上下文开头与结尾的信息利用最好，中段最弱，性能随位置呈 U 形曲线。工程推论是：关键约束放开头或结尾并周期性重申，批量参考材料放中段，不要指望模型逐字读到中间的每一个限制条件。

### 2.3.3 多轮对话的信息衰减

长会话中信息会衰减，机制有三：注意力被越来越多的内容稀释；模型存在近端偏置，旧事实与新内容冲突时偏向新内容；中间摘要式的改写悄悄替换原始细节。表现为早期设定的目标、范围与术语在几十轮后名存实亡，模型开始自作主张。

Agent 的对策是把关键状态外置：维护一个结构化状态对象，包含目标、授权范围、已完成步骤与硬约束，每轮以高优先级位置重注入，而不是依赖对话历史的自然延续。信息要么被刻意重复，要么就会衰减，这应作为设计默认假设。

## 2.4 函数调用全链路

函数调用是 Agent 的手脚，但其底层并无魔法：模型不执行任何代码，它只是在合适的时机输出一段结构化文本，真正的执行永远发生在模型之外的运行时。理解全链路，才能定位每个失败点的来源。

### 2.4.1 tool schema 如何进入模型上下文

调用 API 时传入的 tools 数组（每项含 name、description 与 JSON Schema 形式的参数定义）会被服务端序列化为模型上下文中的一个特殊区块，或并入 system 区域，或使用专用通道与特殊 token。模型由此知道有哪些工具、何时该用、参数怎么填。没有任何隐藏的执行通道，**工具调用本质是模型学会的一种输出格式**。

description 的质量直接决定工具选择准确率：模型靠描述区分相似工具，含糊的描述是误选的首要原因。参数 schema 的严格程度决定参数合法率：enum 与格式约束越明确，幻觉参数越少。同时每个工具都占上下文 token 并增加选择难度，工具数量过大时应转向检索式工具注册（后续章节），而不是无限堆叠。

### 2.4.2 tool_call 的解析与结果回填

完整闭环是四步：模型输出一条 assistant 消息，携带 tool_calls 数组，每项含 id、name 与 arguments——注意 arguments 是 JSON 字符串而非对象；运行时解析并校验参数，执行真正的函数；把执行结果封装为 role 为 tool 的消息回填，其 tool_call_id 必须与请求的 id 严格对齐；带着完整历史再次请求模型。漏回填或 id 错位会直接触发协议错误，或让模型在缺失信息上自由发挥。

下面的示例展示 OpenAI 风格的最小消息闭环：assistant 消息携带 tool_calls，tool 消息以 tool_call_id 对齐回填执行结果，整个序列随后作为下一轮请求的历史输入。

```
messages = [
    {"role": "user", "content": "扫描 10.0.0.5 的 80 端口"},
    {"role": "assistant", "tool_calls": [
        {"id": "call_1", "type": "function",
         "function": {"name": "port_scan",
                      "arguments": "{\"target\": \"10.0.0.5\", \"ports\":
[80]}"}}
    ]},
    {"role": "tool", "tool_call_id": "call_1",
     "content": "{\"status\": \"open\", \"service\": \"nginx\"}"},
]
```

arguments 以字符串形态存在是个刻意的设计信号：解析与校验是运行时的责任。模型可能产出不合 schema 的 JSON、多余字段或类型错误，这些都要在执行前拦截，否则错误参数会直接进入安全工具造成误操作。

### 2.4.3 并行工具调用与强制工具选择

一次 assistant 消息可以携带多个 tool_calls，即并行工具调用：模型判断多个动作互不依赖时一次性发出，运行时并发执行后逐 id 回填对应结果。并行能显著降低

轮次延迟，但要区分读与写：只读操作（查询、扫描）可放心并发，写操作（变更配置、发送验证性请求等授权动作）必须串行化并加锁，避免竞态破坏一致性。

tool_choice 参数控制强制程度：auto 由模型自行决定，required 强制产出某个工具调用，也可指定具体函数名，其实现多依赖受限解码或提示层约束。它适合固定流水线的关键步骤，例如必须先调用日志查询工具。但强制会让模型在没有自然依据时硬编参数，质量低于自然触发，只应确定性环节使用，不要全局开启。

## 2.5 结构化输出与受限解码

Agent 的每个环节都需要机器可读的输出，而 LLM 天然只会输出 token 序列。结构化输出的全部工程问题都源于这一鸿沟，解法分两层：生成时约束与生成后校验，二者缺一不可。

### 2.5.1 JSON mode 与 schema 约束的实现原理

**受限解码**（constrained decoding）是两类 JSON mode 的共同底层机制：解码的每一步对词表做 logits 掩码，把会导致非法输出的 token 概率清零，只允许语法或模式合法的延续。JSON mode 用 JSON 文法（可用有限状态机表达）保证输出是合法 JSON；schema 约束更进一步，把 JSON Schema 编译为文法或自动机，保证结构与类型一致。Outlines、guidance、XGrammar 等开源实现都走这条路，主流托管 API 的 structured outputs 也是同一原理。

受限解码保证的是语法，不是语义：字段值可以完全合法却全是幻觉。它还有代价——复杂 schema 的文法编译有开销，掩码可能迫使模型进入低概率延续而损害内容质量，极端情况下模型宁可返回 refusal 也不愿产出被约束的低质量内容。schema 设计要给模型留呼吸空间：必填字段最小化，枚举优先于自由字符串，嵌套深度受限。

### 2.5.2 失败率、校验与重试

即便有约束，三类失败仍然存在：生成长度达到 max_tokens 导致 JSON 被截断；模型触发 refusal 返回空结果；schema 过宽导致语义错位，类型对但值不可用。因此生成后校验不可省略，标准闭环是：解析、Pydantic 校验、失败则把错误信息回填给

模型重试、重试次数用尽后降级到人工或备用路径。重试必须携带上下文，也就是原输出加错误说明，否则模型只会换个方式再错一次。

```
from pydantic import ValidationError


async def generate_validated(client, prompt, model_cls, retries=2):
    raw = await client.complete(prompt)
    for _ in range(retries):
        try:
            return model_cls.model_validate_json(raw)
        except ValidationError as e:
            raw = await client.complete(
                prompt + f"\n上次输出不符 schema：{e}\n请修正后重新输出 JSON。")
    return None  # 降级路径：人工处理或备用模板
```

这段闭环的关键在重试语义：错误信息必须原样回填给模型，让它看到自己错在哪里，而不是简单重复请求。重试用尽后的降级路径要预先定义，宁可返回明确的失败，也不要让未经校验的数据流入下游工具。

### 2.5.3 Pydantic 校验在 Agent 中的位置

Pydantic 模型的位置应当是模型与工具执行之间的网关：所有要下发给工具的参数先过校验器，类型、枚举、取值范围、必填字段全部核对后才允许执行。它同时承担两个角色：对上游是输出质量的守门员，对下游是权限模型的输入归一化层，被校验器拒绝的输入永远到不了危险操作。

在安全 Agent 中这道闸的意义加倍：一个幻觉出来的目标地址或一条越界的扫描参数，若在执行前没有被类型与范围校验拦住，就直接变成一次未授权动作。校验器因此要和权限模型（后续安全架构章节）联合设计，而不是各自为政。

## 2.6 流式与中断

流式不是体验层面的优化，对长轨迹 Agent 它直接影响首结果延迟与资源占用；中断与超时处理则决定系统在异常下的行为是否可预期。两者都属于容易被推迟、上线后必补课的工程环节。

### 2.6.1 SSE 增量与 tool_call 拼装

主流 API 的流式基于 SSE（text/event-stream）：服务端把响应拆成增量分片（delta）逐个推送，文本增量与工具调用增量交错到达。工具调用的增量以 index 为键：首个分片携带 id 与函数名，后续分片只携带 arguments 的字符串片段，运行时按 index 归并拼接，直到结束事件才算完整。文本可以先到先渲染，但工具调用必须在 arguments 拼装完整并通过校验之后才能执行，提前执行半截 JSON 是流式集成最常见的事故。

工程要点是为拼装器写独立单测：乱序、重复 id、中途断流、arguments 含转义引号，都是真实会出现的情况。拼装器与业务逻辑解耦后，更换模型供应商只需要改适配层。

### 2.6.2 超时与取消传播

超时要分层设置：连接建立、首 token 到达、流间空闲间隔、总时长，各层含义不同。用一个总超时覆盖全部，会让慢而正常的推理被误杀，也会让真正挂死的连接占用资源。取消传播更容易被忽略：用户关闭页面或上游超时后，客户端要主动断开，服务端与正在执行的工具链要收到取消信号并停止工作，否则会出现无人消费的后台扫描这类幽灵任务。

对已启动的写操作，取消必须配合幂等与补偿设计：能安全中断的标记中断，不能中断的执行完并登记结果，保证系统状态可收敛。部分流出的文本要标注未完成状态，不能当作完整结论进入下游。

## 2.7 模型能力分层与路由

不是每个子任务都需要最强模型。分层路由是 Agent 成本控制的第一杠杆，其逻辑来自任务结构本身：输出空间小、判断局部、格式固定的任务，小模型足够；需要跨步骤规划、证据综合与风险权衡的决策，才值得调用大模型。下表给出安全 Agent 的典型分层。

| 层级 | 任务示例 | 采样与模型建议 |
| --- | --- | --- |
| 抽取层 | 字段提取、格式归一 | 小模型，低温 |
| 粗筛层 | 日志分类、误报初筛 | 小模型或专用分类器 |

| 层级 | 任务示例 | 采样与模型建议 |
| --- | --- | --- |
| 决策层 | 规划、证据综合、报告 | 大模型，低温加约束 |
| 应急层 | 复杂重规划、疑难研判 | 最强模型，配预算熔断 |

路由的实现形态有三：规则路由按任务类型硬编码；分类器路由用轻量模型判断任务归属；级联升级先用便宜模型，置信不足或校验失败时升级到更强模型。级联把成本压到最低的同时保住质量下限，是工程上最常用的形态。成本与质量的最优分配是一个可量化的优化问题，涉及每层价格、错误代价与升级阈值的数学推导，第 7 章展开。

## 2.8 提示词工程的工程化

提示词是 Agent 系统中变更最频繁、测试最少、风险最高的资产。把它当代码管理，是 hobby 项目与生产系统的分水岭；本节给出模板、示例与解析器三件套的工程化做法。

### 2.8.1 模板版本管理

模板必须进版本库：每个模板有唯一标识、版本号、变更记录与责任人，变更走代码评审，合入前必须通过离线评测集回归——一组冻结的输入与期望行为，用自动化指标打分。线上问题回滚到上一版本应是单操作。dev、staging 与生产可以有不同的 few-shot 与详略程度，但模板结构与关键约束必须一致，避免测试通过、上线变形。

配套的是效果监控：线上对每个模板版本记录调用量、解析失败率、重试率与人工纠错率，版本间可对比。底层逻辑是：模型行为是分布，模板是控制分布的参数，参数变更不经评测等于盲改，这在生产系统不可接受。

### 2.8.2 few-shot 池与输出解析器容错

few-shot 示例对输出格式与边界的塑造作用远大于长篇指令。工程化做法是建立示例池：按任务类型与难度分层入库、相似度去重、覆盖边界与异常样例；运行时按当前输入检索最相关的少数几条注入，而不是固定罗列。示例质量高于数量，一条错误示例的污染力超过十条正确示例的贡献。

输出解析器要有容错与指标：剥离代码围栏、截取最外层 JSON 对象、容忍尾逗号与单引号、处理常见转义错误，这些脏但可救的输出不应浪费一次重试。解析成功率要作为核心指标持续观测，它恶化往往先于任何业务指标报警，是模型或模板变化的早期信号。

## 2.9 Agent 视角的 LLM 缺陷清单

本节汇总在安全 Agent 中反复出现的模型缺陷。每条给出底层机制、在安全场景的具体影响与缓解方向，系统级方案在第 14 章展开。这份清单应在每次设计评审时逐条对照检查。

### 2.9.1 幻觉：伪造证据与不存在的实体

机制：下一词预测对听起来合理与事实上为真不作区分，模型在训练分布内做插值，对低频实体尤其危险。安全场景的高危实体恰好全是低频的：CVE 编号、内网 IP、主机名、规则 ID、合规条款编号。典型事故是编造不存在的漏洞编号并配上貌似合理的描述，或对空扫描结果补全出应该有的发现。

影响直接命中安全工作的底线：虚构结论混入报告，摧毁报告公信力。缓解方向：所有结论必须锚定工具证据，引用可追溯到具体观察记录；发现进入报告前必须经过独立验证（第 3 章的验证型自检）；对关键实体做存在性校验，不采信生成内容本身。

### 2.9.2 多步漂移与指令遗忘

机制：漂移来自长轨迹中局部观察对目标的渐进带偏，每一步的小偏差看起来都合理，累积后目标面目全非；遗忘来自注意力稀释与近端偏置，早期指令在长上下文中的效力随距离衰减。两者在轨迹超过数十步的 Agent 中几乎必然出现。

安全场景的影响以越界为最：授权范围限定在若干主机，扫描推进中模型顺手探测了相邻网段；或重复执行已完成任务，浪费配额并放大对目标环境的扰动。缓解方向：计划对象常驻上下文并设里程碑校验；硬约束以结构化形式周期性重注入；动作前用代码级校验器核对授权范围——仅靠 prompt 提醒不够，第 14 章给出完整方案。

### 2.9.3 日期、算术与工具依赖

机制：模型没有内建时钟，训练数据截止后的一切现在都是猜的；数字以 token 形式表示，多位乘除与进位在表征层面就不可靠，位数越多越危险。这不是版本问题，而是架构性质，升级模型只会缓解、不会消除。

安全场景的典型受害点：CVSS 评分计算出错、证书有效期与时间线排序判断错误、配额与速率的算术错误导致超限操作。缓解方向：当前时间由系统注入而非询问模型；一切非平凡算术走工具，例如代码执行或计算器；数值比较与日期运算在代码层完成，模型只做语义判断。原则是让模型做它擅长的语义工作，把确定性计算从它手里拿走。

## 2.10 本章速记

- 模型以 token 为单位处理与计费，一切成本与上下文预算先折算 token 再谈设计。
- BPE 的高频合并解释了常见词便宜、低频实体贵且易错的现象。
- 中文 token 效率取决于词表，估算必须基于实测样本而非经验公式。
- 自注意力计算量约为 $O(n^2 \cdot d)$，FlashAttention 只降显存占用、不降计算量。
- KV cache 显存约为 2 × 层数 × KV 维度 × 序列长度 × 字节数，是长上下文定价的根源。
- 稳定前缀放上下文前部以命中 prompt cache，是延迟与成本优化的第一动作。
- 温度调节分布尖锐度，top-p 截断长尾，工具调用场景建议温度 0 到 0.2。
- beam search 适合短答案任务，开放式生成会触发高似然低质量的退化。
- 温度为 0 仍有服务端非确定，可复现性靠低温、seed、版本锚定与响应留档叠加。
- 上下文溢出行为有报错、截断、滑窗三类，Agent 必须自管预算并分层压缩。
- system prompt 的优先地位是概率性的，关键约束需要重复注入与代码级校验兜底。
- 上下文利用呈 U 形，重要信息放两端，中段只放参考资料。
- 多轮信息衰减是默认假设，关键状态要外置为结构化对象并周期重注入。

- 工具 schema 被序列化进上下文，工具调用本质是模型学会的一种输出格式。
- description 决定工具选择准确率，参数 schema 严格度决定参数合法率。
- tool 结果回填必须按 id 对齐，漏回填会导致协议错误或模型自由发挥。
- 并行工具调用要区分读写，只读可并发，写操作必须串行化。
- 受限解码用 logits 掩码保证语法合法，但不保证语义正确。
- 结构化输出要配 Pydantic 校验与带错误上下文的重试，校验器是工具执行前的安全闸。
- 流式下工具调用按 index 拼装，必须完整且校验通过后才执行。

# 第 3 章 推理范式推导：ReAct、Reflexion 与多层自省循环

如果说第 2 章回答了模型如何工作，本章回答如何组织模型完成多步任务。单次调用只能完成局部推断，多步任务需要把调用组织成带反馈的循环结构。本章从 ReAct 出发推导其成立条件与失效模式，再推导 Reflexion 如何把失败经验变成无需梯度的策略改进，最后给出安全 Agent 的多层自省架构与形式化描述。

## 3.1 ReAct：推理与行动的交织

### 3.1.1 Thought-Action-Observation 循环的直觉

用三个基线可以推出 ReAct 的必要性。纯 Chain-of-Thought 让模型在闭卷状态下展开推理：没有外部观察校准，错误在推理链内自我强化，对事实性任务表现为幻觉链——每一步都顺着上一步的错误走，叙事通顺而整体错误。纯 Act 只发动作不做推理：行为退化为条件反射，无法处理多跳依赖，动作失败后也没有调整策略的认知原料。

ReAct 让推理与行动交织：Thought 决定下一步该做什么，Observation 用真实环境反馈修正认知，推理被观察接地，行动被推理指导。Yao 等人 ICLR 2023 的结论是：在 HotpotQA 与 FEVER 等知识密集任务上，ReAct 的事实幻觉明显少于纯 CoT，直接准确率与最强 CoT 变体互有胜负，两者结合效果最好；在 ALFWorld 与 WebShop 等交互决策环境上则显著优于纯 Act。

### 3.1.2 轨迹形式化与失效模式

把一次任务执行形式化为轨迹 $\tau = (s_1, a_1, o_1, s_2, a_2, o_2, \ldots, s_t, a_t, o_t)$，其中 s 是模型维护的任务状态（含历史压缩），a 是动作（工具调用），o 是环境返回的观察。Agent 的全部质量都体现在这条轨迹上，每步决策的条件分布是 $\pi(a \mid s, \text{历史})$，观察是唯一的外部信息源。

失效模式有三。第一，观察噪声导致漂移：安全扫描的高误报率意味着观察经常是错的，错误观察进入思考后被当作事实，后续推理整体偏移，且漂移单调累积，除非有显式校验。第二，死循环：同一动作与观察对反复出现，典型诱因是工具持续报错而模型不断重试同一参数，检测手段是对动作签名做去重与计数。第三，上

下文膨胀：轨迹线性增长使每步成本与延迟上升（第 2 章的 KV cache 约束），后期必须摘要剪枝，而摘要本身又是信息损失源。

### 3.1.3 改进变体：计划与执行分离

Plan-and-Execute 把循环拆成两个角色：规划器先产出完整的多步计划，执行器逐步执行并在计划对象上打勾。它的优势来自结构：计划常驻上下文充当目标的物理载体，漂移空间被压缩；计划本身可审计，安全场景可以在执行前审查计划。代价是灵活性：环境偏离预期时需要重规划，重规划频率过高时它退化为昂贵的 ReAct。工程上常用混合形态：小偏差执行器自行处理，偏差超过阈值触发重规划，重规划本身受硬约束保护，见 3.6 节。

### 3.1.4 设计追问链：ReAct 的边界在哪里

1. 为什么原论文建议 Thought 要短？——长思考扩大幻觉空间，原论文的实践是仅在需要时思考，把推理预算用在关键决策点上。
2. 观察被误报污染时如何止损？——步内自检先行，高噪声工具的结果默认降级为待验证假设而非事实，见 3.4.1。
3. 什么任务不该用 ReAct？——确定性流水线与输出空间极小的任务，直接函数调用加校验更便宜也更稳定。
4. 轨迹压缩如何不丢证据？——分层策略：结论进事实清单，关键观察保留原文摘录，可重取的原始数据丢弃并记录重取路径。

## 3.2 Reflexion：语言化反馈作为无需梯度的策略改进

### 3.2.1 从轨迹失败到语言教训

Reflexion（Shinn 等人，2023）解决的问题是：模型参数冻结、任务失败后，系统如何变得更强。它的答案是把失败轨迹转化为语言化的教训，存入情景记忆，下一轮尝试时注入上下文，让模型显式避开上次踩过的坑。论文把这称为 verbal reinforcement learning：不改一个参数，仅凭上下文中的语言反馈就实现了类似策略改进的效果，在 HumanEval 上达到 91% 的 pass@1，超过当时 GPT-4 直接生成的约 80%；在 AlfWorld 上经多轮反思后，134 个任务解决约 130 个，成功率约 97%。

与强化学习的类比可以这样推导：RL 用梯度更新参数 θ，使 π_θ 偏向高回报动作；Reflexion 冻结 θ，改为更新上下文 m，让 π(a | s, m) 中的 m 承载经验。梯度更新慢而持久，语言反馈快而易失——它不改变权重，只改变条件分布。这个性质决定了适用边界：任务可多次尝试、Evaluator 信号存在、上下文能装下有用的教训。

### 3.2.2 三组件：Actor、Evaluator 与 Self-Reflection

Reflexion 的架构由三个角色构成，职责边界清晰是它可工程化的原因。Actor 负责生成动作与轨迹；Evaluator 负责对轨迹打分或判定成败，信号可以来自环境（测试通过与否、任务完成标志）也可以来自评审模型；Self-Reflection 模型读取轨迹与评价，生成语言化教训，写入情景记忆缓冲，下一轮注入 Actor 的上下文。三组件的输入输出与典型失效如下表。

| 组件 | 输入 | 输出 | 典型失效 |
| --- | --- | --- | --- |
| Actor | 目标、记忆、环境观察 | 动作与轨迹 | 重复犯错、漂移 |
| Evaluator | 轨迹与环境信号 | 成败判定与评分 | 信号噪声、判据漂移 |
| Self-Reflection | 轨迹与评价 | 语言化教训 | 错误归因、编造教训 |

表格最后一列预示了 Reflexion 的两大风险：Evaluator 不可靠会让系统优化错误目标，Self-Reflection 不可靠会让错误教训进入记忆。两者的可靠性是工程化的主战场，失效模式在 3.2.4 节展开。

### 3.2.3 成功率迭代逻辑与收敛条件

迭代提升的逻辑可以形式化为：设第 k 轮任务成功率为 $p_k$，失败后生成教训 $m_k$，则下一轮 $p_{k+1} = p_k + \Delta(m_k) - \varepsilon(m_k)$，其中 Δ 是有效教训带来的增益，ε 是反思噪声带来的退化。收敛需要两个条件：Δ 大于 ε，即教训平均而言有效；迭代有上限，防止预算失控。若 Evaluator 信号无偏且 Self-Reflection 引用真实轨迹证据，Δ 大于 ε 通常成立；反之系统会震荡甚至退化。

工程上要主动构造收敛条件：Evaluator 优先使用客观信号，如测试结果与独立工具验证；记忆设容量上限与相似度去重，防止互相矛盾的教训同时在场；迭代上

限与预算熔断作为硬性终止，见 3.4.4。Reflexion 不是无限免费提升，它是在有限尝试次数内抬高每次尝试质量的机制。

### 3.2.4 失效模式：反思幻觉与记忆污染

反思幻觉指 Self-Reflection 生成看似合理却错误的归因：失败的真实原因是误报污染了观察，反思却把教训总结为应该换工具，错误归因的教训在下一轮把行为带偏。它源于生成式模型对因果的补全倾向——叙事通顺不等于归因正确。记忆污染是其放大器：错误教训进入记忆后跨轮存续，被后续反思引用而反复强化，一次污染、长期退化，且会跨任务传染。

对策有三：教训必须引用轨迹证据，无证据的反思不入库；教训入库前经独立评审，评审者与生成者解耦，见 3.4.3；记忆条目带置信度与时效，低置信教训不参与强引导。记忆治理的完整方案在第 14 章。

### 3.2.5 设计追问链：Reflexion 为什么有效

5. 它与 fine-tuning 的本质区别？——不改参数、零训练成本、即时生效，但上限受上下文容量约束，且记忆不跨模型泛化。
6. Evaluator 什么信号最可信？——环境客观信号优于模型自评；自评仅在拿不到客观信号时使用，且要做校准。
7. 记忆会无限增长吗？——会，必须设容量、去重、置信与淘汰策略，否则矛盾教训互相抵消甚至污染。
8. 何时退回普通 ReAct？——任务短、单次成本低、失败代价小时，反思开销可能超过重试开销，先做成本核算。

## 3.3 Self-Critique 与自一致性

### 3.3.1 结构化自评

Self-Critique 让模型对自己的产出做结构化评审：按证据充分性、步骤正确性、目标对齐等维度逐项打分，每项给出依据与修正建议。实现要点有二：评审 prompt 与执行 prompt 分离，评审者以独立视角读取目标、轨迹与结论；评审输出用第 2

章的受限解码加 Pydantic 校验，保证可被程序消费。自评的价值在于低成本发现显性缺陷，如格式错误、证据缺失、步骤跳跃，但它不能作为正确性的最终裁决。

### 3.3.2 判定性与经验性：与 Reflexion 的区别

Self-Critique 是判定性机制：对当前输出做一次裁决，决定通过、修正还是拒绝，作用域是单次产出。Reflexion 是经验性机制：跨轮累积教训，改变的是未来尝试的行为分布，作用域是任务全程。两者在流水线上的位置不同：自评是门禁，Reflexion 是学习。工程上组合使用——自评拦截本轮的显性错误，Reflexion 把本轮逃逸的错误转化为下一轮的教训。只装门禁不学习，同类错误反复出现；只学习不设门禁，错误版本会先发布出去。

### 3.3.3 自我一致性与自评偏差

self-consistency（Wang 等人，2022）对同一问题采样多条推理路径并以多数投票取答案，在算术与常识推理基准上带来稳定提升；代价是推理成本按采样路数线性放大，适合离线分析与高价值判定，不适合每步都跑。它利用的原理是：正确路径更倾向在不同采样下收敛到同一答案，而错误路径彼此发散。

自评的偏差必须有清醒认识：模型对自己产出的评分普遍偏高，区分自己正确与错误答案的能力有限，自评通过不等于正确，这是被多项研究反复验证的现象。机理与确认偏置同源：评审者与执行者共享参数与知识分布，容易被同一套错误叙事说服。对策是解耦：评审用不同模型或至少不同的上下文与视角，见 3.4.3；关键判定叠加独立验证。

## 3.4 多层自省循环的架构设计

### 3.4.1 层 1：步内自检

层 1 作用于每次工具调用之后，检查结果与预期是否一致：类型与格式是否符合 schema、取值是否在合理范围、非空性与完整性、与上下文已知事实是否冲突。检查按成本排序：先跑廉价的程序化断言，零模型成本、零噪声，异常再升级给模型判断，避免用大炮打蚊子。层 1 的目标是在污染扩散前拦截：空结果、超时、格式错误、异常值在这一层处理，不进入上层推理。它同时是观察噪声的第一道过滤器，见 3.6.1。

下面的参考实现展示层 1 的排序原则：程序化断言先于模型判断，异常结果先降级为待验证标记，避免把噪声观察当成事实送入后续推理。

```
def step_self_check(call, result, ctx):
    issues = []
    if not result.matches_schema(call.tool):
        issues.append("结果与工具 schema 不符")
    if result.is_empty() and call.tool.expects_nonempty:
        issues.append("空结果，工具或参数可能有误")
    if conflicts_with(result, ctx.known_facts):
        issues.append("与已确认事实冲突")
    if issues:
        ctx.mark_candidate(call, issues)  # 降级为待验证观察
    return issues
```

### 3.4.2 层 2：轨迹级 Reflexion

层 2 在任务失败或里程碑未达时触发，对整条轨迹生成语言化教训并写入记忆，产物是记忆条目而非即时动作。它与层 1 的分工是抽象层级：层 1 管单步质量，层 2 管策略修正——为什么计划走不通、哪类路径应避开、什么观察不可信。层 2 的全部约束来自 3.2 节：Evaluator 信号可信、教训引用证据、记忆有治理。触发条件要克制，不是每次失败都值得反思，微小且可自愈的失败只记指标、不生成教训。

### 3.4.3 层 3：评审团与解耦

层 3 引入独立评审：由不同模型或上下文隔离的评审实例，对轨迹与结论做最终裁决。独立性是设计核心——评审者只拿证据与结论，不拿执行者的推理过程，避免被自我叙述说服。参数相同的模型在隔离上下文中评审仍优于完全自评，预算允许时换不同供应商的模型效果更好。多个评审者的分歧本身是信息：一致通过才放行，分歧大则降级为待人工处理。层 3 与层 2 的职责区别在于：层 2 帮助下一轮做得更好，层 3 决定本轮结论能否出门，它是防止自我确认偏差的最后屏障。

### 3.4.4 循环终止条件设计

自省循环必须有硬性终止，否则反思本身会成为新的失效源：无限反思消耗预算，也会把正确结论越改越错。三类终止条件与对应的输出处理如下表。

| 终止条件 | 触发信号 | 输出处理 |
| --- | --- | --- |
| 迭代上限 | 轮次达到预设最大值 | 部分结果加风险声明 |
| 改进停滞 | 相邻轮目标改进小于阈值 | 冻结计划并升级人工 |
| 预算熔断 | token 或时间超出预算 | 保存状态等待续跑 |

三类条件是或的关系，任一触发即终止，不允许任何再试一次的例外逻辑。终止不等于失败：生产系统的正确行为是输出带置信标注的部分结果、声明未完成的原因、给出人工升级路径。把优雅终止作为设计目标而非兜底手段，是循环类系统可靠性的关键；阈值的量化与第 7 章的成本数学衔接。

### 3.4.5 设计追问链：为什么三层而不是一层

9. 层 1 能否替代层 2？——不能。步内检查只见局部，策略级错误（方向错了但每步都对）只有轨迹级反思能发现。
10. 层 3 与层 2 用同一模型可以吗？——上下文隔离的同一模型优于完全自评，跨供应商模型更优，关键是打破确认偏置的相关性。
11. 层数越多越安全吗？——不是。每层都引入成本、延迟与噪声，层数应与错误代价匹配，低风险任务两层足够。
12. 评审团自己错了怎么办？——评审质量单独监控，与后续实际结果回对照，长期失准的评审通道要下线重校。

## 3.5 自省循环的状态转移形式化

### 3.5.1 状态、记忆与策略

把多层自省循环形式化。状态 $s = (\tau, m, b)$：$\tau$ 是当前轨迹，$m$ 是反思记忆，$b$ 是剩余预算，包括 token、时间与迭代次数。策略 $\pi(a \mid s, m)$ 显式受记忆影响——这与标准 MDP 的差别在于 $m$ 使过程不再满足马尔可夫性，等价于把 $(s, m)$ 联合视为增

强状态。自省是一次特殊的状态转移：m' = R(τ, 评审结果)，R 是反思算子，生成语言教训并入记忆，同时 b 递减。整个系统于是成为增强状态空间上的决策过程，普通 Agent 是 m 恒为空的特例。

这个形式化的价值在于给出设计检查表：R 的质量（教训是否有效）、m 的治理（容量、去重、时效）、b 的管理（预算熔断）三个旋钮共同决定系统行为，任何一处失控都表现为 3.5.2 节的发散。

### 3.5.2 收敛判据与发散处理

收敛的判据是目标函数 J 在迭代间单调非降且增量趋零：设 J(k) 为第 k 轮评审得分，要求 $J(k+1) \geq J(k) - \delta$，且滑动窗口内平均改进小于阈值 ε 时停止。发散的典型形态有两种：J 震荡不升，反思在两个矛盾教训间摇摆；目标漂移，重规划悄悄换了目标，J 对着错误目标上升。发散处理要预先写进系统：冻结当前计划、回滚到最近的有效里程碑、发散轨迹交给层 2 反思但屏蔽其教训的自动入库，重复发散即触发人工介入。发散不是异常路径，而是长循环系统的常态路径，架构必须为它留好座位。

下面的参考实现把终止判定写成可单测的纯函数：迭代上限与预算熔断是硬条件，改进停滞用滑动窗口检测，与 3.4.4 的三类条件一一对应。

```
def should_stop(loop):
    if loop.iteration >= loop.max_iterations:
        return "ITER_LIMIT"
    if loop.budget_exceeded():
        return "BUDGET_CUT"
    window = loop.score_window
    if len(window) == loop.window_size and (
        max(window) - min(window) < loop.epsilon
    ):
        return "STAGNATION"
    return None
```

## 3.6 安全 Agent 中的特化

### 3.6.1 噪声观察与验证型自检

安全工具的观察质量是各领域中最差的之一：扫描器有结构性误报，情报源有陈旧数据，日志有缺失与乱序。第 2 章的幻觉缺陷叠加观察噪声，意味着未经处理

的发现同时面临两个错误来源。特化设计是把层 1 升级为证据分级：每个观察打上证据等级，未经交叉验证的发现一律标记为 candidate，只有通过独立验证——第二工具交叉验证、执行确认或人工复核——才允许进入报告。发现与报告之间由此形成两阶段流水线，验证是必经的闸门而非可选项。这与 3.3.3 的结论一致：验证必须独立于发现，同一条推理链不能既产生发现又确认发现。

### 3.6.2 失败重规划的安全性约束

自省循环在安全场景的最大风险不是低效，而是目标达成逻辑压倒安全约束：扫描路径失败后，重规划可能生成越权探测；验证失败后，重规划可能放宽判据。因此**硬约束必须优先于目标达成**，且优先的实现方式是结构性的：授权范围、破坏性操作白名单、数据外发限制作为不变量校验器置于每个动作之前执行，任何重规划产出的动作先过校验器，越界动作直接拒绝而非提醒。写在 prompt 里的约束是概率性的（第 2 章的结论），代码级校验才是硬边界。重规划的正确设计是先过滤动作空间再让模型选择，而不是让模型自由生成后靠劝告兜底。

## 3.7 本章速记

- ReAct 的优势来自推理与行动互为校准，纯 CoT 产生幻觉链，纯 Act 退化为条件反射。
- 轨迹 τ 是 s、a、o 的序列，Agent 的全部质量就是轨迹质量。
- 观察噪声、死循环、上下文膨胀是 ReAct 三大失效模式，各有对应的检测手段。
- Plan-and-Execute 用常驻计划对象压制漂移，代价是重规划开销。
- Reflexion 把失败轨迹转为语言教训，是无需梯度更新的策略改进，即 verbal RL。
- 三组件 Actor、Evaluator、Self-Reflection 中，Evaluator 信号质量决定成败。
- 成功迭代要求教训增益大于反思噪声，且迭代必须有上限。
- 反思幻觉与记忆污染是 Reflexion 两大风险，对策是证据引用与独立评审。
- Self-Critique 是判定性门禁，Reflexion 是经验性学习，位置与作用不同。
- self-consistency 用多路投票提升准确率，成本线性放大，适合高价值判定。

- 模型自评普遍偏高，关键判定必须与执行解耦。
- 层 1 步内自检廉价检查先行，在污染扩散前拦截异常观察。
- 层 2 轨迹反思产出记忆条目，负责策略修正而非即时动作。
- 层 3 评审团只拿证据不拿推理过程，切断自我确认的相关性。
- 终止条件三类：迭代上限、改进停滞、预算熔断，任一触发即终止。
- 优雅终止输出部分结果加风险声明，是设计目标而非兜底手段。
- 增强状态 s 等于 τ、m、b 的三元组，自省算子 m' = R(τ, 评审结果) 构成形式骨架。
- 收敛要求 J 单调非降且增量趋零，发散要冻结、回滚、人工介入。
- 安全发现必须经独立验证才入报告，且验证必须独立于发现。
- 硬约束以不变量校验器置于动作前，优先于任何目标达成逻辑。

# 第 4 章 LangGraph 底层原理：Pregel 模型与状态通道

## 4.1 为什么需要图编排

LangChain 生态最早提供的编排原语是顺序链（Chain），它把多个 Runnable 首尾相接成一条直线，上游输出即下游输入，执行拓扑在构造期就固定下来。对一次性问答任务，线性已经足够；但 Agent 的执行轨迹由运行时观察决定，线性模型没有表达循环、并发与中途暂停的位置，控制流只能以 if、while 与线程池的形式散落在业务代码里，随功能增长不断腐化。

### 4.1.1 Chain 的线性局限

顺序链的本质缺陷在于它只是函数组合：没有回到上一个节点的原语，没有同时走两条路的原语，更没有停下来等人决策的原语。要实现重试只能包裹重试适配器，要实现路由只能嵌套 RunnableBranch 与 RunnableLambda，层层嵌套之后，状态共享靠闭包变量，错误传播靠异常穿透，单元测试需要 mock 整条链路。对需要多轮循环与中途恢复的 Agent 系统，线性模型在第一个真实需求上就会触顶。

### 4.1.2 Agent 循环的控制流需求

典型安全 Agent 的轨迹不是直线：规划节点产出计划后进入侦察，侦察结果决定是否做指纹识别，漏洞扫描之后可能并发验证多个目标，critic 不满意则回到规划重来，危险动作前还要挂起等待人工批准。这里同时出现了四类控制流需求：回环、条件分支、动态并行、以及崩溃后从中间状态恢复。它们的共同特征是拓扑在运行时才确定，静态代码只能用命令式逻辑硬编码，审查与复现都很困难。

### 4.1.3 图模型下的表达

在图模型里，四类需求各有一个一等公民：回环是显式回边，分支是条件边的路由函数，并行是多出边静态扇出或 Send 动态扇出，恢复是检查点加线程。更重要的是，控制流从散落的代码变成图数据结构本身，可以整体可视化、静态审查、按图做影响分析；每个节点的输入输出都有明确的通道语义，失败恢复的粒度天然是单个超步。LangGraph 之所以选择图而不是更深的 Chain 封装，根源正在于此。

## 4.2 Pregel/BSP 执行模型

### 4.2.1 超步与 barrier 同步

LangGraph 的运行时直接借鉴 Google Pregel 的整体同步并行（BSP）模型，把一次图执行切分为一串**超步**（super-step）。每个超步内，所有被触发的节点并行执行；节点读到的状态是超步开始时的快照，产出的写入先暂存，直到超步结束的 barrier 处统一提交。同一超步内的节点互不可见对方的写入，超步之间严格串行，整图执行因此成为一串确定的状态转移，可暂停、可重放、可审计。

### 4.2.2 通道作为状态载体

Pregel 原文中顶点之间靠消息传递，LangGraph 把消息机制替换为共享**通道**（channel）：每个状态键对应一个通道，通道持有当前值与版本号。节点之间不直接通信，而是向通道写入部分更新；下游节点因监听这些通道而被调度。调度的依据是版本比对——节点记录自己见过的通道版本（versions_seen），当入边通道的当前版本比自己见过的新，该节点就成为待执行任务。通道因此同时承担状态存储、消息路由与调度触发三重职责。

### 4.2.3 与 Actor 模型的对比

Actor 模型同样以消息驱动并发，但语义差异明显：Actor 拥有私有状态与邮箱，消息异步到达，处理顺序不确定，系统没有全局时间概念；Pregel 的节点是无状态函数，状态全部在共享通道上，超步边界提供全局同步点。对 LLM Agent 而言，后者有三个直接好处：状态快照天然对应检查点；barrier 让并行分支的汇合语义清晰；确定性转移让任意中间状态可以离线重放。代价是放弃异步流水线的吞吐，而 Agent 的瓶颈本就在 LLM 调用上，这笔交换是划算的。

## 4.3 状态通道与 reducer

### 4.3.1 Annotated reducer 的合并语义

StateGraph 的第一个参数是状态 schema，通常以 TypedDict 定义。默认情况下每个键对应 LastValue 通道，语义是后写覆盖：同一超步内多个写入只保留最后一个

要改变合并语义，用 typing.Annotated 在键上标注 **reducer**，例如 Annotated[list, add_messages]；此后该通道的写入不再替换旧值，而是与旧值折叠。形式化地，设通道现值为 c，本超步写入序列为 $w_1$ 到 $w_k$，屏障提交时执行 $c' = \text{fold}(\text{reducer}, [w_1, w_2, \ldots, w_k], c)$。合并发生在 barrier 处，这是理解一切并行语义的起点。

### 4.3.2 add_messages 与自定义合并函数

add_messages 是使用频率最高的内建 reducer，语义为按消息 ID 合并：新消息追加，携带已有 ID 的消息视为对旧消息的就地更新，常用于流式补全或人工改写；在 0.2 后期版本中还可配合 RemoveMessage 按 ID 删除。这一合并是幂等的，重放同一批消息不会产生重复。自定义 reducer 只需一个二元函数：发现按 id 去重、证据只增不删、计数取最大值，都是十几行以内的实现。约束只有一条：reducer 必须是纯函数，同样的旧值与写入序列必须得到同样的结果，否则检查点恢复的正确性无从谈起。

### 4.3.3 并发写入冲突的消解

当静态扇出或多路 Send 产生并行分支，多个分支可能在同一超步内向同一通道写入。没有 reducer 的 LastValue 通道会直接抛出 InvalidUpdateError，提示每个超步对无 reducer 通道只能接收一个值；这不是缺陷而是保护，因为后写覆盖在并行场景下没有良定义。声明了 reducer 之后，写入按序折叠进通道，交换律与结合律保证最终状态与分支完成顺序无关。因此 reducer 不是可选的锦上添花，而是并行分支安全的先决条件：任何会被并行写入的键，都必须显式声明合并语义。

## 4.4 节点与边

### 4.4.1 节点：状态到部分更新的纯函数

节点是签名统一的函数：输入当前状态，返回一个只含要写入通道的字典，即**部分更新**，合并交给通道与 reducer 完成。这样设计有三重好处：测试时直接以构造的状态字典调用节点，无需启动运行时；恢复时只需重放节点写入，无需重算；并行时各节点只声明自己关心的键，冲突面最小。工程上要求节点近似纯——同样的

输入状态产生同样的更新，LLM 调用与工具调用等副作用通过入参显式传入，并在检查点中留下痕迹。

### 4.4.2 普通边与条件边

add_edge 定义固定跳转，上游完成即调度下游。add_conditional_edges 接受一个路由函数：它读取状态，返回目标节点名或路径映射中的键，在运行时决定去向。一个节点可以有多条出边：多条普通出边构成静态并行，所有目标同时触发；条件边返回多个目标名同样是静态扇出；返回 Send 对象列表则升级为动态扇出，交给下一节。路由函数必须保持纯函数约束，只依赖状态、不依赖外部可变量，否则同一检查点重放会走出不同路径，恢复语义即被破坏。

### 4.4.3 入度汇聚的 barrier 语义

并行分支汇聚到多入度节点时，调度规则是：该节点等待所有并行分支完成后才执行一次，而不是每完成一个分支就触发一次。这是超步 barrier 的直接推论，也恰好是工程上想要的 join 语义：三个分析分支全部写完 findings 之后，critic 只看到一次完整的合并结果，无需自己处理部分数据或防止重复触发。反过来，如果业务确实需要每完成一个就处理一个，应把汇聚逻辑放进每个分支各自的尾部节点，而不是期待 barrier 之外的语义。

## 4.5 Send API 动态扇出

### 4.5.1 map-reduce 模式

条件边的路由函数除了返回节点名，还可以返回 Send(node, arg) 对象的列表。每个 Send 在运行时实例化一个独立任务：第一参数是目标节点，第二参数是该任务专属的输入。这实现了任务数在运行时才确定的 map-reduce：map 阶段按观察生成任意个 Send，reduce 阶段靠 barrier 汇聚加 reducer 合并。安全场景的典型用法是按待验证目标数量动态生成验证任务，可能是一个也可能是二十个，而图结构保持不变。

### 4.5.2 与静态条件边的差异

静态扇出在定义期固定目标集合，所有分支共享同一份状态输入，分支数与图结构绑定；Send 把两件事都动态化了：分支数量由路由函数在运行时计算，每个分支的输入可以是从主状态裁剪出的独立子状态。差异的另一面是成本：Send 要求作者明确设计子状态与合并策略，静态扇出则天然共享主状态。经验法则是：分支之间无需共享可写状态时用 Send 隔离，分支都要读写主状态时用静态扇出加 reducer。

### 4.5.3 子状态与主状态的合并

Send 目标节点的输入由 Send 的第二参数决定，可以与主状态 schema 完全不同例如只含一个目标的字典。关键在返回侧：该节点返回的部分更新仍然写回主图的通道，经主状态的 reducer 合并，这也是扇出分支通常返回以 id 为键的结果列表、靠去重 reducer 合并的原因。若需要更强的隔离，例如分支内部有自己的消息历史与中间状态，则把分支实现为子图，Send 直接指向子图，子图结束时只把声明的输出键写回主图。分支返回值如何进入主状态，是 map-reduce 设计的核心决策。

## 4.6 中断与人工审批

### 4.6.1 挂起点语义

compile 时传入 interrupt_before=[node] 或 interrupt_after=[node]，图会在进入或离开指定节点时暂停，前提是配置了 checkpointer。挂起不是跳过：暂停发生时，待执行节点与当前状态都持久化在检查点中，graph.get_state(config).next 即可读到挂起位置。0.2 后期版本还引入了 interrupt 函数，允许在节点内部任意位置挂起并携带恢复值；本章以 interrupt_before 为主线，它是语义最简单、兼容面最大的一种。

### 4.6.2 checkpointer 与 thread

检查点按线程（thread）组织，调用时通过 configurable 传入 thread_id，同一 thread_id 的多次调用共享一份状态历史。thread 是恢复、回放与多轮会话的锚点：审批人第二天上班，用同一 thread_id 调用 get_state，看到的就是昨晚挂起时的精

确状态。工程上建议 thread_id 绑定业务实体，例如一次授权任务一个 thread，审计系统据此把所有超步快照串成完整证据链，这比按日期或按进程命名可追查得多。

### 4.6.3 update_state 注入与恢复

人工决策的注入与恢复在 LangGraph 中是一套固定的三步序列，顺序不能乱，尤其最后一步的输入形式有精确语义，写错一步，审批闭环就被打破。

13. 调用 update_state(config, values)，把人工决策写入通道，values 经各通道 reducer 合并。
14. 用 graph.get_state(config).next 校验待执行节点符合预期。
15. 以 None 作为输入调用 invoke 或 stream，从检查点续跑，被挂起的节点随之执行。

None 是明确的语义信号，表示不注入新输入、仅从挂起点继续；若误传了非 None 的输入字典，图会把它当作新一轮执行的输入，审批流程就被绕过了。update_state 的 as_node 参数声明这次写入视作哪个节点的产出，并据此重算恢复后的路由起点；该参数省略时的默认归属规则随版本演进，工程上建议显式传值，或恢复前用 get_state 校验，避免歧义。

### 4.6.4 time-travel 回放

checkpointer 保存每个超步的快照，graph.get_state_history(config) 按时间逆序返回全部历史。选中某个历史检查点，以携带 checkpoint_id 的 config 再以 None 调用，图就从那一刻重放，这就是 **time-travel**。它有两个用途：事故复盘，把系统拉回错误决策之前，改变人工决策再走一遍；分支实验，从同一检查点出发跑两种计划比较效果。重放以该快照为父检查点继续追加，先前的分支仍留在历史中可查，原证据链不被覆盖。

## 4.7 检查点机制

### 4.7.1 每超步落库

配置 checkpointer 之后，每个超步结束都会持久化一次完整状态，粒度是超步而非整次运行。长任务的崩溃窗口因此被压缩到一个超步之内：恢复时从最后一个

完整检查点继续，已完成的节点不重跑。interrupt 的挂起语义也正是靠停在某超步边界实现的。工程含义是，checkpointer 的写入路径在每次状态转移上都被执行，它的延迟与可靠性直接成为图的延迟与可靠性，生产环境必须把 saver 当作核心依赖做容量规划与监控。

### 4.7.2 状态快照结构

一份快照的核心字段有三组：channel_values 记录每个通道的当前值；channel_versions 记录每个通道的版本号，版本随写入单调递增；versions_seen 按节点记录其已消费的通道版本。调度器用后两者做差集，判断哪些节点有新输入需要执行，这也是 barrier 汇聚语义的实现基础。快照还携带来源节点与步骤号等 metadata。理解这个结构的直接价值是排查：线上问题时常需要直接读 saver 中的 channel_values，确认某个键最终合并成了什么。

### 4.7.3 MemorySaver、SqliteSaver 与 PostgresSaver 选型

三种常用 saver 对应三个部署档位。MemorySaver 存于进程内存，零依赖、最快但进程退出即丢失且不能跨进程共享，适合单测与本地实验。SqliteSaver 落单个 SQLite 文件，部署极简，适合个人工具与小规模单机服务，需要单独安装 checkpoint 扩展包，并发受单文件写锁限制。PostgresSaver 面向生产：多副本共享状态、事务保证、提供异步变体，配合连接池承载高并发 thread，代价是多运维一个 Postgres 实例。选型逻辑与选数据库一致：先问故障恢复与并发要求，再谈性能。

## 4.8 子图与模块化

子图就是把一张编译好的图当作另一张图的节点：add_node 直接接受已 compile 的子图，运行时子图在父图调度下执行，并默认共享父图的 checkpointer 与 thread。状态映射遵循同名即共享：父子 schema 中同名的通道自动互通，子图写入 findings，父图的 findings 通道立即经 reducer 合并；键名不同则需要一个包装节点，显式把父状态字段映射为子图输入，把子图输出映射回父状态。

模块化的收益体现在团队协作上：每个团队只拥有自己子图的 schema 与节点，对外契约收敛为少数几个共享通道名；父图作者不需要理解子图内部结构，只需要

信任其输入输出契约。安全 Agent 自然的切分是侦察子图、分析子图与报告子图，分别由不同方向的同学维护，调试时子图可以独立 compile、独立运行。需要注意子图会加深检查点层级，快照体积与恢复时间相应增加，粒度不宜过细，一般以团队边界为界，而不是以节点为界。

## 4.9 综合示例：StateGraph 定义

下例把本章机制收进一个可运行的骨架：两个 reducer 通道、一个后写覆盖通道一个条件边路由与编译期挂起点配置。import 语句与两个节点函数体是标准样板，此处聚焦通道语义相关的部分。

```
def merge_findings(left: list, right: list) -> list:
    idx = {f["id"]: f for f in left}            # 按 id 去重
    idx.update({f["id"]: f for f in right})
    return list(idx.values())

class SecState(TypedDict):
    messages: Annotated[list, add_messages]     # 按消息 ID 合并
    findings: Annotated[list, merge_findings]   # 按业务 id 去重合并
    verdict: str                                # 无 reducer，后写覆盖

def route_after_scan(s: SecState) -> str:
    return "poc_verify" if s["findings"] else END

g = StateGraph(SecState)
g.add_node("scan", scan_node); g.add_node("poc_verify", poc_node)
g.add_edge(START, "scan"); g.add_conditional_edges("scan", route_after_scan)
app = g.compile(checkpointer=MemorySaver(), interrupt_before=["poc_verify"])
```

三个通道刻意覆盖了三种合并语义：messages 按消息 ID 幂等合并，findings 按业务 id 去重，verdict 后写覆盖。route_after_scan 是路由函数的最小形态：读状态、返回节点名或 END，把它换成返回 Send 列表即得动态扇出。这段骨架配合 MemorySaver 即可在单测中全流程验证，替换成 PostgresSaver 即可进入生产，通道语义与执行机制解耦带来的可移植性正在于此。

## 4.10 典型错误与排查

新手踩坑高度集中在通道语义与恢复语义两类，以下五类错误覆盖了绝大多数线上症状。它们都出现在运行时而非定义期，图在定义时完全合法，跑起来才暴露。掌握 4.3 节的通道语义之后，这些症状都可以按表直接定位。

| 错误类型 | 典型现象 | 根因 | 处置 |
| --- | --- | --- | --- |
| 状态键未声明 | 更新不生效或直接报错 | 节点返回的键不在 schema 中 | 补齐 TypedDict 字段定义 |
| 并发写入冲突 | InvalidUpdateError 报错 | 并行分支写无 reducer 通道 | 为该键声明 Annotated 合并 |
| Send 后未收敛 | 图提前结束，汇聚节点未跑 | 扇出分支没有边指向汇聚点 | 为每个分支补指向汇聚点的边 |
| 恢复后重复执行 | 审批后的动作跑了两遍 | 恢复时误传了非 None 输入 | 恢复一律以 None 作为输入 |
| 状态莫名回退 | 旧值覆盖了新值 | 后写覆盖通道被并行写入 | 换 reducer 或改为串行写入 |

表中第二类与第五类同根同源：LastValue 通道在串行时的语义是后写覆盖，在并行时则变成没有良定义的谁后谁赢，LangGraph 用报错把后者暴露出来是刻意设计排查检查点类问题有一套通用手法：把 saver 换成 MemorySaver，以 stream_mode 为 updates 在本地复现，逐超步观察每个节点实际写入了哪些通道、经什么 reducer 合并，绝大多数状态谜题会在这里现出原形。

## 4.11 本章速记

- 图编排把回环、分支、并行与恢复变成一等公民，控制流成为可审查的数据结构。
- LangGraph 的执行模型是 Pregel/BSP：超步内节点并行，超步间经 barrier 严格串行。
- 节点读到的状态是超步开始时的快照，同超步写入互不可见，在屏障处统一提交。

- 每个状态键对应一个通道，通道同时承担状态存储、消息路由与调度触发。
- 节点被调度的依据是版本比对，versions_seen 落后于通道当前版本即产生任务。
- 默认通道是 LastValue，语义为后写覆盖；Annotated reducer 可改写合并语义。
- 提交时执行 reducer 对旧值与全部写入的折叠，合并发生在 barrier 处。
- add_messages 按消息 ID 幂等合并，同 ID 视为更新，重放不产生重复。
- reducer 必须是纯函数，满足交换律与结合律时结果与写入到达顺序无关。
- 并行分支写无 reducer 通道会抛 InvalidUpdateError，这是保护而非缺陷。
- 节点是状态到部分更新的函数，天然可单测、可重放、可并行。
- 条件边路由函数必须保持纯函数，否则同一检查点重放会走出不同路径。
- 多入度节点等待所有并行分支完成再执行一次，即 join 语义。
- Send 实现运行时动态扇出，分支数与每分支输入都在运行时确定。
- Send 分支的返回值仍写回主图通道，经主状态 reducer 合并。
- interrupt_before 配合 checkpointer 实现挂起，get_state 的 next 给出挂起位置
- 恢复执行的输入必须是 None，非 None 输入会被当作新一轮执行的注入。
- 检查点每超步落库，核心字段是 channel_values、channel_versions 与 versions_seen。
- MemorySaver 用于测试，SqliteSaver 用于单机，生产环境选 PostgresSaver。
- 子图按同名通道与父图自动共享状态，团队协作以通道名为对外契约。

# 第 5 章 多分支状态机设计与状态转移方程

## 5.1 AgentState 的状态设计

### 5.1.1 字段分组

aegis-agent 的 AgentState 按职责分为五组。对话与任务组承载 messages 与 task，task 是一个字典，包含目标、范围与授权凭证编号；计划与迭代组承载 plan、iteration 与 reflections；结果数据组承载 findings 与 evidence；安全与预算组承载 human_approval 与 budget，后者登记 tokens、时长与请求数的上限和已用量；输出组承载 report_md。分组的直接目的是降低读代码时的认知负担，深层目的是为状态裁剪做铺垫：不同组的生命周期不同，裁剪策略也不同。

### 5.1.2 findings 的按 id 去重合并

findings 是整个状态里被写入最频繁的键：recon、vuln_scan、pcap_analysis、lateral_sim 都会产出发现，扇出分支还会并发写入。aegis-agent 为它定义了按 id 去重的合并 reducer：每条发现生成时必须携带全局唯一 id，由节点名、目标标识与内容哈希拼接而成；合并时以 id 为主键做 upsert，不同 id 全部保留，同 id 冲突按 confidence 字段取优，置信度相等视为内容等价、任取其一。这个语义天然幂等，同一批发现无论写入多少次，结果集合不变，这正是它敢于被并行写入的底气。

### 5.1.3 只增 reducer 与后写覆盖

reflections 使用只增 reducer，critic 与 reflect 的产物只追加、永不修改：反思记录本质是审计日志，历史不可篡改是需求而非实现细节，后续预算判定与终止判定还要引用它的条数。其余标量与字典字段用默认的 LastValue 后写覆盖，因为这些字段都有唯一属主在推进：iteration 只由 reflect 递增，human_approval 只由人工注入，budget 只由预算检查节点更新，后写覆盖恰好表达最新决策生效。设计口诀：会被并行写的键给合并 reducer，单属主推进的键后写覆盖，审计性质的键只增不减。

### 5.1.4 状态定义节选

下例给出状态骨架与两个核心 reducer，聚焦合并语义相关的部分。merge_findings 正是 5.2.3 节交换律论证的实现载体，operator.add 承担只增语义，evidence 与 reflections 共用它；标量组全部裸声明，交给默认的后写覆盖。

```
def merge_findings(left: list, right: list) -> list:
    out = {f["id"]: f for f in left}            # id 为主键
    for f in right:                             # 同 id 按 confidence 取优
        if f["confidence"] >= out.get(f["id"], {"confidence": -1})
["confidence"]:
            out[f["id"]] = f
    return list(out.values())


class AgentState(TypedDict):
    messages: Annotated[list, add_messages]; task: dict    # 对话与任务
    plan: list; iteration: int                  # 计划与迭代
    reflections: Annotated[list, operator.add]  # 只增
    findings: Annotated[list, merge_findings]   # 结果数据
    evidence: Annotated[list, operator.add]
    human_approval: bool; budget: dict          # 安全与预算
    report_md: str                              # 输出
```

## 5.2 状态转移方程的形式化

### 5.2.1 节点作为局部转移函数

把图执行抽象成状态机：记全局状态为 s，每个节点 n 是一个局部转移函数，转移方程为 $s' = \delta(s, node(s, obs))$，其中 obs 是节点执行期间从外部世界获得的观察，包括工具输出与 LLM 回复。δ 只改写该节点声明负责的通道，未触碰的通道保持不变。整图执行是 δ 沿调度序的复合：超步内并行分支各自给出局部更新，屏障处按通道 reducer 折叠为全局下一状态。这个视角把节点写状态从编程惯例上升为可推导的代数结构，5.2.2 节的并行安全论证就建立在其上。

### 5.2.2 并行合并的代数性质

并行分支安全的形式化依据是：被并行写入的通道，其合并必须构成满足交换律与结合律的 reduce。设并行分支 i 的写入为 $w_i$，通道现值为 c，则最终状态为 $c' = c \oplus w_1 \oplus w_2 \oplus \cdots \oplus w_k$，其中 $\oplus$ 表示该通道 reducer 的折叠。只要 $\oplus$ 交换且结合，$c'$ 就与 $w_i$ 的到达顺序无关，分支完成时序的不确定性被代数性质吸收；反之若 $\oplus$ 不交换，例如后写覆盖，$c'$ 就依赖顺序，并行写入在语义上未定义。LangGraph 在无 reducer 通道上直接抛出 InvalidUpdateError，等于拒绝执行未定义的并行语义。

### 5.2.3 findings 合并的交换律证明直觉

把一次写入看成对 id 到发现这一映射的更新，合并等价于映射逐键合并：键集做并集，同键按 confidence 取最大。并集运算满足交换律与结合律，取最大值同样满足交换律与结合律，且两者逐键独立，所以整次合并可交换、可结合。交换律的直觉表述：两个分支无论谁先落库，按 id 去重、按置信度取优的结果相同；结合律的直觉表述：先两两局部合并再全局合并，与一次全量合并等价。工程推论是，同 id 写入必须保证置信度可比，这是把数据库 upsert 语义安全搬进内存通道的前提。

## 5.3 节点职责表

下表给出 aegis-agent 十二个节点的输入、输出与失败降级策略。输入输出均以通道名表述；降级指节点内部异常或上游数据不足时的行为。所有降级遵循同一原则：降级不吞错，降级事件本身作为一条 finding 或 reflection 落入状态，保证 report 阶段可见、可审计。

| 节点 | 输入 | 输出 | 失败降级 |
| --- | --- | --- | --- |
| memory_recall | messages、task | 历史相关摘要写入 messages | 检索失败则空召回，主链路继续 |
| planner | task、findings、reflections | plan 更新，iteration 递增 | LLM 重试 2 次后降级为单步直扫计划 |
| recon | plan | findings 追加资产与端口 | 部分失败则保留部分结果并标注残缺 |

| 节点 | 输入 | 输出 | 失败降级 |
| --- | --- | --- | --- |
| fingerprint | recon 产出 | findings 追加技术栈指纹 | 指纹失败按保守技术栈处理并标注不确定 |
| vuln_scan | plan、findings | findings 追加候选 CVE | 扫描器异常逐项隔离，坏目标跳过 |
| poc_verify | findings、human_approval | findings 标记已验证与否 | 验证失败标记未证实，不删除候选 |
| pcap_analysis | task 携带的 pcap 路径 | findings 追加流量异常 | 解析失败产出脏数据 finding 并跳过 |
| lateral_sim | plan 与授权范围 | findings 追加横向路径 | 越界目标直接拒绝并记审计事件 |
| blue_sim | findings | findings 追加检测缺口 | 演练失败降级为静态规则比对 |
| critic | findings、plan | verdict 与 must_fix 列表 | critic 超时按放行处理并记 reflection |
| reflect | must_fix、reflections | 改进项追加，推进迭代 | 无可反思则输出空并触发终止判定 |
| report | findings、evidence | report_md | 渲染失败回退为纯文本清单 |

表中有两处值得展开。planner 的降级是换一份更平庸的计划而不是终止任务，对授权测试而言，以保守方式继续推进通常优于空手而归。critic 的降级是放行并留痕：critic 是守门员，守门员自身失能时把决定权交还主链路并记录 reflection，好于让整张图卡死在评审环节。所有降级动作都同时写入 reflections，这正是 5.1.3 节只增语义的直接受益者，事后复盘可以完整还原每一次降级发生的位置与原因。

## 5.4 状态流转全景

### 5.4.1 主链路

主链路为 START、memory_recall、planner、recon、fingerprint、vuln_scan 依次串行，随后进入 poc_gate 条件路由：发现中存在候选 CVE 且未获人工批准时进入 poc_verify，否则直接进入扇出。扇出分支全部完成后经 barrier 汇入 critic，critic 判定通过则 report，否则经 reflect 回到 planner 构成回环。整条链路中状态沿通道单向流动，回环不制造新的状态空间，每一轮迭代只是对同一组通道的再次折叠，iteration 与 reflections 负责留下轮次痕迹。

### 5.4.2 回环与双重上限

critic、reflect、planner 构成的回环受两个独立上限约束：max_iterations 限制 planner 发起的侦察与分析总轮数，max_reflections 限制 reflect 产出的反思总条数。只设单一上限有两个缺陷：只限迭代数，系统可能在低质量反思里空转耗尽预算；只限反思条数，则等价于变相的迭代上限，失去独立性。两个上限任一触顶，route_after_critic 都强制走向 report，并在报告中显式声明因迭代上限未收敛，把非正常终止变成可审计事件而非静默截断。

### 5.4.3 动态扇出

扇出由 Send 实现，分支数在运行时确定：vuln_scan 或 poc_verify 完成后，路由函数遍历任务字段，任务携带 pcap 文件路径就生成 pcap_analysis 分支，配置了横向模拟范围就生成 lateral_sim 分支，启用蓝队演练就生成 blue_sim 分支，任意组合都合法。每个 Send 携带从主状态裁剪出的子状态，分支之间互不可见，返回值经 findings 的去重 reducer 合并回主状态，最后由 barrier 汇聚到 critic，三个分支全部落定，评审才开始。

### 5.4.4 中断与降级路径

两类旁路决定图的实际走向。其一是中断：poc_verify 之前设置 interrupt_before，存在待验证候选时图挂起等待人工批准，批准则恢复执行验证，拒绝则注入否决并改道 report。其二是降级：预算检查内嵌在 vuln_scan 与扇出路由

中，任一维度耗尽，路由函数跳过一切分析分支直接指向 report，报告标注已完成部分与未覆盖面。两条旁路共同保证图在任何情况下都有确定终点，且终点状态足以解释为什么停在这里。

## 5.5 条件路由函数的形式化

### 5.5.1 poc_gate 与 _fanout 的判定条件

poc_gate 挂在 vuln_scan 之后，形式化为对状态谓词的求值：当 findings 中存在待验证候选且 human_approval 为假时返回 poc_verify，让图挂起在人工审批点；否则调用 _fanout。_fanout 是纯映射：任务字段的 pcap 路径非空则生成 Send(pcap_analysis, 子状态)，横向范围非空则生成 Send(lateral_sim, 子状态)，蓝队演练开启则生成 Send(blue_sim, 子状态)；三者全不满足时返回单个 Send(critic, 主状态)直达评审，保证扇出点之后必有汇聚，不会出现空扇出导致图静默结束。

### 5.5.2 route_after_critic 的终止判定

route_after_critic 读取 critic 写入的 verdict 与 must_fix，形式化为一行合取式：进入回环当且仅当 must_fix ≠ ∅ ∧ iteration < I_max ∧ 反思条数 < R_max，其中 I_max 与 R_max 分别是 5.4.2 节的双重上限。三个条件是合取关系，任何一个不满足都走向 report：缺陷清单为空说明质量达标，上限触顶说明资源用尽，两类终止必须在报告里区分呈现。路由函数同样保持纯函数约束，只读状态、不读外部可变量，这是回环可重放的前提。

## 5.6 中断与恢复的状态语义

挂起点配置为 interrupt_before 列表中的 poc_verify。当 poc_gate 路由到 poc_verify 时，图在验证节点执行之前停下，此刻 findings 已包含候选 CVE，但没有任何验证动作发生。挂起语义的精确性正是安全审计需要的：审批人看到的是将要执行什么，而不是已经做了什么再追认。interrupt 依赖 checkpointer，未配置 saver 时编译或运行期会直接报错；挂起状态因此天然落库，不怕审批人离开期间进程重启。

审批通过在 aegis-agent 中是一段固定的恢复序列，三步的顺序与输入形式都有精确语义，写错任何一步，轻则审批闭环失效，重则验证动作在未经批准的情况下被执行。

16. update_state(config, {"human_approval": True})，把人工决策写入通道。
17. 以 None 作为输入调用 invoke，语义是不注入新输入、仅从检查点续跑。
18. 挂起的 poc_verify 携带批准标志执行，验证动作全部落库后经 _fanout 进入后续分支。

关于 as_node：update_state 的这一参数声明写入归属的节点，并影响恢复后的路由起点，其省略时的默认归属规则随版本演进。aegis-agent 在挂起恢复场景选择省略该参数，并以恢复前 graph.get_state(config).next 仍为 poc_verify 作为校验条件，把版本差异收敛在一个显式检查点上，任何偏离都会在恢复前暴露而不是在恢复后造成破坏。

拒绝批准的序列相同，只是注入值保持 human_approval 为 False 并同时写入一条否决 reflection，恢复后 poc_verify 读到否决，将候选标记为未验证而不执行任何动作。poc_verify 结束时把 human_approval 复位为 False，使批准成为一次性令牌：回环产生的下一批候选必须重新走审批，避免一次批准被无限次复用。恢复后的状态一致性由两点保证：注入只改写人工决策通道，不触碰 findings 与 plan；reducer 幂等使得重复注入同一决策不产生副作用，审批人重复提交也不会造成重复验证。

## 5.7 状态膨胀控制

长任务下有两类通道线性增长：messages 随对话轮次与工具原始输出膨胀，findings 随扫描轮次累积。膨胀的代价有三：LLM 上下文被历史淹没，关键信息被稀释；每个超步的快照都携带全量通道值，检查点体积与序列化耗时随任务时长恶化；审计检索变慢。控制膨胀因此不是优化项，而是长任务能跑完的前提。

aegis-agent 的策略按字段生命周期分三类处理。messages 在进入 vuln_scan 与 critic 前由节点内裁剪副本再送 LLM，通道本身保留全史，这是不破坏可重放性的裁剪；确需压缩存储时，用携带同 ID 消息的替换写回完成收缩。findings 不删除但分级，critic 只消费置信度达阈值的子集，低置信候选留在通道中供复盘。evidence 的

原始产物在入通道前压缩为摘要与 SHA-256 哈希，原文落对象存储，report 阶段用哈希回链，通道里只留可读摘要。

裁剪必须遵守两条纪律。其一，裁剪动作要么发生在派生读取路径，即裁剪后的副本只作为下一节点或 LLM 的输入，要么通过显式写回完成，绝不静默篡改只增通道，否则 5.2.2 节的代数性质被破坏，重放不再一致。其二，裁剪与预算联动：messages 超限时先触发一次摘要压缩再进入下一超步，压缩本身作为一个节点写入 reflection，让状态瘦身也留下审计痕迹。

## 5.8 图结构代码示例

下例给出 aegis-agent 的图装配骨架，裁去 import 与各节点函数体：_fanout 展示动态扇出与空扇出兜底，注册循环收拢十二个节点，两条条件边与一条回环边表达 5.4 节的全景流转。

```
def _fanout(s: AgentState):
    t = s["task"]                                   # 按任务字段动态定分支数
    pairs = [("pcap_analysis", t.get("pcap_path")), ("lateral_sim",
t.get("lateral_scope")),
             ("blue_sim", t.get("blue_drill"))]
    return [Send(n, s) for n, on in pairs if on] or "critic"


g = StateGraph(AgentState)
for name, fn in NODES.items(): g.add_node(name, fn)    # 12 个节点统一注册
g.add_edge(START, "memory_recall"); g.add_edge("memory_recall", "planner")
g.add_edge("planner", "recon"); g.add_edge("recon", "fingerprint")
g.add_edge("fingerprint", "vuln_scan")
g.add_conditional_edges("vuln_scan", poc_gate)         # 有候选未批准则挂起等审批
g.add_conditional_edges("poc_verify", _fanout)         # 批准恢复后动态扇出
for n in ("pcap_analysis", "lateral_sim", "blue_sim"): g.add_edge(n,
"critic")
g.add_conditional_edges("critic", route_after_critic); g.add_edge("reflect",
"planner")
g.add_edge("report", END)
app = g.compile(checkpointer=MemorySaver(), interrupt_before=["poc_verify"])
```

三处细节值得注意：Send 的第二个参数传入 s，即每个分支获得主状态在发送时刻的独立副本，若要强隔离应改为构造裁剪后的子状态；_fanout 的兜底返回 critic 字符串，防止空扇出让图静默结束；interrupt_before 只在配置了 checkpointer

时合法。示例使用 MemorySaver 便于本地验证，生产环境替换为 PostgresSaver，装配代码其余部分不变。

## 5.9 本章速记

- 状态字段按生命周期分为对话任务、计划迭代、结果数据、安全预算与输出五组。
- findings 用按 id 去重的 upsert reducer，同 id 冲突按置信度取优，天然幂等。
- reflections 用只增 reducer，反思记录是审计日志，只追加不修改。
- 标量与字典字段用后写覆盖，前提是该键有唯一属主节点在推进。
- 节点即局部转移函数，转移方程为 $s' = \delta(s, \text{node}(s, obs))$。
- 并行安全的依据是合并满足交换律与结合律，结果与分支完成顺序无关。
- findings 合并可交换，因为按 id 求并集与按置信度取最大都可交换可结合。
- 无 reducer 通道上的并行写入会被运行时以 InvalidUpdateError 拒绝。
- 主链路为 memory_recall 到 vuln_scan 串行，后接 poc_gate 条件路由。
- critic 回环受 max_iterations 与 max_reflections 双重上限约束。
- 扇出用 Send 实现，分支数与分支输入在运行时确定，空扇出必须兜底。
- 预算耗尽时路由直接指向 report，降级路径必须留痕。
- poc_verify 前挂起等待人工批准，批准经 update_state 注入后以 None 恢复。
- poc_verify 结束将 human_approval 复位，批准是一次性令牌。
- 恢复后一致性靠注入面最小化与 reducer 幂等共同保证。
- messages 裁剪优先在派生读取路径做，不静默篡改只增通道。
- evidence 存摘要与哈希，原文落对象存储，report 用哈希回链。
- 路由函数全部为纯函数，只读状态，这是回环可重放的前提。
- 双上限触顶与质量达标是两类终止，必须在报告中区分呈现。

# 第 6 章 数学建模：攻击路径概率与风险量化

本章把攻击图从定性结构升级为可计算的概率与风险模型，回答三个工程问题：哪条路径最可能被走通、一个资产当前被拿下的概率有多大、下一步修什么最划算。所有概率都服务于授权测试中的行动排序与防御侧的资源投放，是决策辅助量而非物理测量值。全章遵循先建模动机、再推导、后失效条件的顺序展开。

## 6.1 攻击图形式化

攻击图建模为有向图 **G = (V, E)**。节点集合 V 表示主机与权限状态，典型形态是二元组（主机, 权限级），权限级取 none、user、root 或某服务的访问权；边集合 E 表示一次利用或凭据使用导致的状态转移。攻击者在图上的位置不是一个节点，而是其已控制状态的集合，攻击推进就是把新节点并入该集合。

边 e = (u → v) 的语义由两侧条件定义。前置条件：攻击者已处于状态 u，且具备该转移所需能力，例如已取得源主机 user 权限、目标存在对应漏洞且网络可达。后置条件：执行成功后进入状态 v，例如目标主机权限提升。图构造就是把扫描器输出的漏洞、凭据与可达性逐条翻译成这种转移规则。

这一形式化的思想源流一句话说清：MulVAL 用 Datalog 规则从主机配置与漏洞推导攻击路径，Nessus 把漏洞扫描结果映射到资产与拓扑，本章的攻击图是这两条路线在 Agent 时代的合流。经典文献中完整攻击计划可能是多条边的与关系，图退化为 AND-OR 图；本章采用工程简化，把与关系折进单条边的前置条件，或关系留给图结构，代价是损失部分表达力，6.5 节给出补救。

构造器按边类型实例化转移规则，最常见的四类边整理如下表：

| 边类型 | 前置条件 | 后置条件 |
| --- | --- | --- |
| 漏洞利用 | 源可达且目标存在对应漏洞 | 目标权限提升一档 |
| 凭据复用 | 已获得有效口令或密钥 | 目标服务登录成功 |
| 信任利用 | 源主机被目标显式信任 | 免认证获取访问 |
| 跳板转发 | 已控制中继主机 | 攻击面延伸到新网段 |

前三类边的参数来自漏洞与凭据数据，第四类来自路由与防火墙规则。每类边的成功概率建模方式不同：凭据类接近确定性事件，漏洞类需要下一节的映射函数。

## 6.2 利用成功概率建模

攻击图本身是定性的，边只有存在与不存在。要让 Agent 排序行动、让防御方排序修复，必须给每条边一个成功概率 p(e)。这个概率不是物理量，而是**校准出来的决策量**，建模动机是让漏洞利用边与凭据边落在同一尺度上比较，进而支撑路径计算与风险排序。

### 6.2.1 CVSS 到概率的分段映射

**建模动机**：CVSS 基础分数是为严重程度沟通设计的序数量表，不是为概率校准设计的，直接除以 10 当概率会在低分段严重失真。工程做法是分段线性映射 p = f(cvss)：人工设定锚点，锚点之间线性插值。选分段而非全局函数，是因为分数在各区间的语义密度不同，9 分以上几乎都有武器化利用，4 分附近则混杂大量带苛刻前置条件的漏洞。

一组常见的**起始锚点**形如 (0, 0.02)、(4, 0.10)、(7, 0.40)、(9, 0.70)、(10, 0.90)，两端刻意不取 0 与 1，为贝叶斯更新留出余地。必须强调，锚点数值是工程经验校准而非理论推导，照抄任何文献的数字都没有意义，必须用本组织数据回归得出，这是分段设计的核心纪律。

### 6.2.2 红队命中率回归修正

校准方法论：在授权红队演练中，对每次利用尝试记录 CVSS 向量、利用成熟度网络位置、认证状态与成败结果，按分数段统计命中率，或对命中率做逻辑回归后回填锚点。这条管线把 f(cvss) 从拍脑袋的初值变成随演练数据迭代的经验模型，也把映射的误差来源从黑箱变成可审计的样本偏差。

三个工程要点。样本必须区分网络位置与认证状态，否则混入环境效应，拟合出的锚点只对演练环境成立；公开 POC 可用的样本命中率显著更高，Exploit Maturity 应作为回归特征单独建模；样本量不足的分段保持保守并加宽区间。EPSS 这类基于在野利用观测的分数可作额外特征或先验来源，校准须随每轮演练重新拟合。

### 6.2.3 环境修正因子

基础映射只刻画漏洞本身的可行性，实际边概率还要乘环境修正：p(e) = f_base(cvss(e)) × c_exp(e) × r(e)。c_exp 是**可利用性修正**，覆盖认证要求、目标加固水平与利用武器化程度，取值通常落在 0.2 到 1 之间；r 是**可达性修正**，源到目标被防火墙隔离时置 0，处于信任边界内侧时接近 1，实现里常用按网段分段衰减近似。

乘法形式的隐含假设是各修正因子条件独立，该假设的失效场景集中在 6.9 节讨论。另一失效点是 CVSS 9 到 10 分段的饱和：基础分失去区分度时，只能依赖 c_exp 拉开差距，这要求可利用性修正的粒度足够细，至少区分有公开 POC、有武器化框架与仅在野观测三种状态。

## 6.3 路径成功概率与 log 域最短路

### 6.3.1 独立性假设下的连乘

**建模动机**：给定边概率后，自然的问题是整条路径走通的概率。假设每条边在其源状态达成后的成败与其他边独立，即把每次利用尝试视为独立伯努利试验。在此假设下，路径上各边依序尝试，成功概率为 **P(path) = ∏ p(e)**，e 取遍路径各边。推导只有两步：条件概率链式法则展开，再按独立性把各条件概率替换为边缘概率。该假设明显偏乐观，凭据复用与共享漏洞都会破坏它，但连乘换来了下一节的最短路求解框架，是可解性与真实性的关键折中。

### 6.3.2 连乘下溢与 log 域转换

**建模动机**是数值稳定性。双精度浮点在 1e-308 以下归零，40 条边、每边概率 0.1 的路径积是 1e-40，虽未归零，但数千条路径相互比较时精度与可读性都会崩坏，且连乘在增量计算里每加一条边都要重算全部乘积。解法是取负对数：定义边代价 **w(e) = ln(1/p(e))**，则路径代价 Σ w(e) = −ln P(path)。

于是最大化 P(path) 等价于最小化代价和，问题从概率连乘变成标准最短路。又因 p(e) ≤ 1 恒有 w(e) ≥ 0，恰好满足 Dijkstra 的非负权前提，堆实现复杂度 O((|V| + |E|) log |V|)。两个边界情形：p = 1 的边代价为 0，对应近乎免费的凭据边；p 趋于 0 时代价发散，实现须设下限 p_min 防止 ln(0)，量级取 1e-4 即可。log 域还有附带好处：边代价可预先缓存，证据更新只改单边代价，整图重算是毫秒级操作。

log 域 Dijkstra 的核心实现如下，概率还原只需对最短代价取指数：

```
import heapq, math

def max_prob_path(graph, src, dst, p):
    # graph: 邻接表；p[(u, v)] 为边成功概率，取值 (0, 1]
    dist, pq = {src: 0.0}, [(0.0, src)]   # dist[u] = -ln P(到 u 最优路径)
    while pq:
        d, u = heapq.heappop(pq)
        if u == dst:
            return math.exp(-d)           # exp(-Σw) 还原路径概率
        for v in graph[u]:
            nd = d + math.log(1.0 / p[(u, v)])   # 边代价 w = ln(1/p)
            if nd < dist.get(v, math.inf):
                dist[v] = nd
                heapq.heappush(pq, (nd, v))
    return 0.0
```

失效条件有两层。独立性崩塌时连乘模型整体失效，须转入 6.5 节的采样方法；数值上若锚点映射把某条边概率压到 p_min 以下，代价被钳制，最短路可能系统性偏爱这条本应极差的边，钳制参数要与校准下限联动调整。

### 6.3.3 为什么不能简单用最短跳数

直觉上跳数少的路径更危险，在概率语义下这是错的。示意反例：路径甲一跳、单边概率 0.2；路径乙两跳、每边 0.9，连乘得 0.81。按跳数选甲，按概率选乙，结论相反。跳数只有在所有边概率近似相等时才是概率的合理代理，而 CVSS 覆盖 0 到 10 的现实里边概率相差可达一个数量级以上。

更隐蔽的偏差是对凭据边不敏感。一条 $p \approx 0.95$ 的凭据复用边几乎免费，跳数模型却把它与一次高风险利用等价记为一跳，横向移动风险评估因此系统性失真。结论：所有排序一律在 log 域进行，跳数只作为展示层信息保留给人类读者。

## 6.4 贝叶斯证据更新

### 6.4.1 更新框架

**建模动机**：6.2 节的概率是先验，粒度到漏洞条目为止，不含具体目标的实测信息；对目标跑一次扫描、做一次 POC 验证后必须修正。设假设 H 为该边实际可利用

证据 E 为观察结果，更新公式为 **P(H|E) = P(E|H) × P(H) / P(E)**，分母按全概率展开为 P(E) = P(E|H) × P(H) + P(E|¬H) × (1 − P(H))。后验回写边概率后整图重算最短路，代价毫秒级，因此证据更新可以在 Agent 主循环里每轮触发。

### 6.4.2 先验与似然的来源

两个输入各有固定来源。先验 P(H) 来自威胁情报：EPSS 分数、在野利用通报、厂商公告，编码的是该漏洞被真实利用的可能性。似然来自扫描器置信度：P(E|H) 是真阳性率 TPR，即真实存在漏洞时扫描器报出的概率；P(E|¬H) 是假阳性率 FPR。

示意算例：先验 0.3、TPR 0.95、FPR 0.10，则 P(E) = 0.285 + 0.070 = 0.355，后验 P(H|E) = 0.285 / 0.355 ≈ 0.80，一次高置信扫描把边概率从 0.3 拉到 0.8。若再叠加一次 POC 实测验证（TPR 约 0.98、FPR 约 0.02），后验升至 0.98 附近，工程上按确认边处理。

多次独立扫描建议在几率域连乘：后验几率等于先验几率乘以各证据的似然比，比反复套贝叶斯公式更稳。失效条件必须写明：两台扫描器共用同一签名引擎时证据强相关，连乘更新会双重计数并导致过度自信；TPR 与 FPR 不是环境不变量，镜像环境测出的指标不能直接用于生产网段；若先验已含 EPSS 的在野信息，又把 POC 公开当新证据计入，构成循环引用，应保持证据源互斥。

## 6.5 蒙特卡洛替代方案

连乘模型的独立性假设在两类场景崩塌：**共享漏洞**，同一镜像或同批补丁让大量主机的边同开同关；**同一凭据**，一组泄露密钥同时打开多台主机的门。这些相关性有共同结构：存在公共原因事件。闭式解随相关性结构组合爆炸，工程上放弃解析，改用采样估计分布。

建模方式是把公共原因显式建成潜变量：例如凭据泄露是一个伯努利事件，一旦发生，其覆盖的边集合整体转为高概率。采样流程分三层：先抽公共原因，再在各边条件概率下独立抽取成败，然后沿图前向模拟攻击者的可达状态集。重复 N 次后统计各资产被攻陷频率，得到分布而不只是均值，P95 分位数对尾部风险更有意义复杂度 O(N × |E|)，N 取万级时只适合对关键路径与核心资产精算，不适合全图。

失效条件是稀有事件：路径概率在 1e-6 量级时，朴素采样要百万级样本才能凑出几个命中，方差大到不可用，需要重要性采样或重要性拆分；此外潜变量结构依赖对相关性来源的正确认知，漏掉一个公共原因，估计就系统性偏低，相关性清单要与凭据与镜像管理流程联动维护。

## 6.6 风险量化与修复优先级

### 6.6.1 单资产风险与年化损失

**建模动机**是把概率翻译成钱，才能与业务方对话并参与预算竞争。单资产风险定义为 **R = P(compromise) × Impact**。P(compromise) 取攻击图计算结果：快速模式取到该资产的最大概率路径，精算模式取 top-k 条边不相交路径的 $1 - \prod(1 - P(path_i))$，或直接用 6.5 节的采样分布；Impact 为资产价值乘以暴露因子。

经典年化框架为 **ALE = ARO × SLE**，其中 SLE = AV × EF，即单一损失预期等于资产价值乘以暴露因子，ARO 为年度发生频率。安全 Agent 里的分工是：ALE 服务年度预算与保险决策，回答一年大概损失多少；每次扫描后的实时修复排序用当前概率的 R，因为 ARO 把近期剧变平滑掉了，对本月刚出现公开 POC 这类事件反应太钝。

### 6.6.2 修复优先级排序

修复决策的排序规则是**风险下降量除以修复成本**。对每个候选动作，把对应边的概率从 p 改为修复后的 p′，重算图上风险取差值 ΔR，再除以修复的人力、停机与许可成本。项目实现里资产价值权重把 core 资产定为 10、普通资产为 1 量级，核心资产上的中等漏洞因此天然排在边缘资产高危漏洞之前。

不追求组合最优的原因：预算约束下选修复集合使总风险最小是覆盖问题，NP-hard，而修复流程需要的是可解释的排序加人工复核，贪心是工程折中。下表给出一组示意性排序，说明绝对收益与性价比会给出不同次序：

| 候选修复 | 修复前风险 | 修复后风险 | ΔR 与成本比 |
|---|---|---|---|
| 核心库弱口令轮换 | 0.90 | 0.20 | 极高，立即执行 |
| 边缘机高危补丁 | 0.45 | 0.10 | 中等，纳入排期 |
| 内网低危配置加固 | 0.12 | 0.08 | 偏低，挂起观察 |

表中数值为示意，真实分数来自各自环境的校准概率与资产权重。排序的意义在于把最危险与最划算分开：告警按 ΔR 绝对值输出，修复清单按比值输出，两者使用不同的展示通道，避免把吓人的事当成划算的事。

## 6.7 决策建模与博弈视角

攻击侧决策建模为期望效用最大化：效用 ≈ P(成功) × 目标价值 − 行动成本，行动成本包括 token 开销、时间与**被检测风险**。检测风险项是授权红队 Agent 与理想化攻击者在模型上的分界：高噪声批量扫描要在效用里折价，这促使 Agent 偏好低噪声、高确定性的凭据边而非大范围漏洞扫描，该偏好在后续章节的探索策略里会反复出现。

防御侧排序已由 6.6 节给出，攻防互动的完整博弈均衡在工程上不可解：信息不完全、收益难以币值化、策略空间随攻击图组合爆炸。实践采用**假设已失陷**的下限原则：无论攻击图算出的概率多低，核心资产一律按已失陷标准配置纵深防御、最小权限与恢复能力；概率模型只决定增量资源的投放顺序，永远不能作为豁免底线防护的理由。

## 6.8 与项目代码的对应

本章模型在项目代码中的落点集中在 lateral_sim 模块。cvss_to_prob 函数实现 6.2.1 的分段锚点插值，锚点表为模块级常量，替换锚点即完成组织级校准；边构造器把扫描结果翻译成带 p(e) 的边，可达性修正内嵌在构造阶段；路径求解器是 log 域 Dijkstra，边代价在建边时算好缓存；风险输出 risk = prob × value，权重表中 core 资产取 10。直接对应关系如下表：

| 模型构件 | 代码落点 | 实现说明 |
| --- | --- | --- |
| 分段映射 | lateral_sim 的 cvss_to_prob | 锚点插值，常量可替换 |
| log 域最短路 | lateral_sim 路径求解器 | Dijkstra，非负权堆实现 |
| 风险排序 | risk = prob × value | core 权重取 10 |

表中三处是本章公式的直接实现。贝叶斯更新由扫描回调触发，更新后整图重算；蒙特卡洛精算属于预留扩展点，默认关闭，避免小规模演示场景承担不必要的计算与配置负担。

## 6.9 失效条件与边界

模型的边界清单如下，每条都对应一类真实的翻车方式：

- **独立性崩塌**：凭据复用与共享镜像使连乘低估尾部风险，关键资产必须用采样法复核。
- **映射误差传播**：CVSS 一分的标定偏差经连乘放大，长路径上可达数量级偏差，须定期重校锚点。
- **拓扑陈旧**：攻击图建立在 CMDB 与防火墙导出之上，数据滞后一到两周，可达性判定整体失真，拓扑必须有 TTL 与增量重建。
- **先验循环**：威胁情报与公开 POC 信息重复计入使后验虚高，证据源必须互斥。
- **语义边界**：所有概率是决策辅助量，授权测试中 POC 实测永远优先于模型推断，模型只决定先验证哪条边。

## 6.10 本章速记

- 攻击图 $G = (V, E)$ 中节点是主机加权限状态，边是带前置与后置条件的利用转移。
- 形式化源流一句话：MulVAL 的 Datalog 规则推导与 Nessus 的资产拓扑映射之合流。
- 边概率由 CVSS 分段映射出发，经可利用性与可达性修正，用红队命中率回归校准锚点。
- 独立性假设下路径概率为连乘 $P(\text{path}) = \prod p(e)$，是可解性与真实性的关键折中。
- 连乘下溢用 log 域化解：$w(e) = \ln(1/p(e))$，最大概率路径化为 Dijkstra 最短路。
- 非负边代价满足 Dijkstra 前提，堆实现复杂度 $O((|V| + |E|) \log |V|)$。
- 跳数不是概率的代理，凭据边与高危利用每跳的真实代价可差一个数量级。

- 贝叶斯更新 P(H|E) = P(E|H)P(H)/P(E)，先验来自威胁情报，似然来自扫描器 TPR 与 FPR。
- 一次高置信扫描可把边概率从 0.3 拉到 0.8 量级，POC 实测按确认边处理。
- 相关证据禁止连乘更新，签名引擎同源的扫描结果会双重计数导致过度自信。
- 独立性崩塌时改用蒙特卡洛：公共原因建潜变量，输出分布与 P95 而非均值。
- 稀有事件让朴素采样失效，需重要性采样，预算有限时只精算关键路径与核心资产。
- 单资产风险 R = P(compromise) × Impact，年化口径 ALE = ARO × SLE 用于预算层。
- 修复排序等于风险下降量除以修复成本，价值权重 core 取 10，普通资产取 1。
- 攻击侧效用必须扣检测风险，这是红队 Agent 与理想化攻击者的模型分界。
- 博弈均衡不可解，工程上取假设已失陷下限：概率只排序资源，不豁免底线防护。

# 第 7 章 Token 经济：成本模型与开销控制

前面各章解决 Agent 做得对的问题，本章解决做得起的问题。安全 Agent 单次任务的 LLM 调用从数十次到数百次不等，工具观察动辄数千 token，没有成本模型与熔断机制的 Agent 在生产环境等于一颗成本地雷。本章从计费结构出发推导成本模型，再给出五类控制手段：预算分配、观察摘要化、模型分级路由、缓存复用与预算熔断。

## 7.1 成本模型推导

### 7.1.1 单次任务成本分解

**建模动机**来自 LLM API 的计费结构：按输入与输出 token 计价，按调用次数累计。单次任务成本写成 **C = Σ_阶段 (n × t × price)**，n 是该阶段调用次数，t 是单次调用平均 token 数，price 是所用模型单价。这个分解的价值在于定位杠杆：降成本有四个抓手，减少调用次数、压薄每次 token、把部分调用换到便宜模型、提高缓存命中率。

单次调用的 token 再拆一层：t = t_sys + t_schema + t_hist + t_obs + t_out。system prompt 与工具 schema 每次调用都要随请求重发；历史随轮次线性增长；观察是工具返回内容；输出是模型生成部分。关键推导：若不加裁剪，第 k 轮调用的输入约为 B + k × δ，B 是固定头部，δ 是每轮新增历史，则任务总输入 ≈ K × B + δ × K × (K + 1) / 2，对总轮数 K 是**平方级**。这个平方项是 Agent 成本失控的数学根源，本章大多数手段都在对付它。

该模型的失效条件：缓存命中会大幅降低历史部分的有效单价，按目录价计算会系统性高估成本；重试与失败调用未建模，工程上按一成左右的余量预留；输出 token 单价通常高于输入，t_out 的权重不可忽略，长报告类任务的输出占比会反超观察。

### 7.1.2 观察：安全 Agent 的主要成本源

通用对话 Agent 的成本大头通常是历史，安全 Agent 不是。一次全端口扫描的原始输出数千 token，一份漏洞扫描报告上万，目录枚举与日志摘录同样巨大，单轮

观察超过模型输出一个数量级是常见情形。这意味着安全 Agent 的第一成本杠杆不是精简 system prompt，而是**处理观察**：不摘要、不截断、不复用观察的 Agent，成本结构里最大的一块花在搬运原始输出上。

观察还有乘法效应：观察一旦进入历史，后续每轮调用都要重发一遍，直到上下文裁剪或任务结束。一条 5000 token 的扫描输出若在历史中存活 20 轮，目录价下就是 10 万 token 的搬运量，而其中能用三句话结构化结论存住的信息可能不到一成。观察治理因此优先于一切提示词层面的省钱技巧。

## 7.2 上下文预算分配

### 7.2.1 固定配比留位

上下文窗口要按固定配比给四类内容留位。system 与工具 schema 是固定头部；计划与任务状态必须常驻，否则 Agent 在长任务里会忘记目标；关键发现常驻，是最终报告的原料；观察窗口是唯一的弹性区。配比的意义是**隔离**：观察再大也不能挤掉计划与发现，超限时只压缩观察区本身。示意配比：固定头部 10% 到 15%，计划 10%，发现 20%，观察 50%，输出余量 5% 到 10%，按任务类型微调，取证类任务上调发现区。

配比设计可归纳为下表，数值是工程起点而非定规：

| 上下文区块 | 示意配比 | 超限策略 |
| --- | --- | --- |
| 系统与 schema | 10%~15% | 不可压缩 |
| 计划与状态 | 约 10% | 压缩为当前步 |
| 关键发现 | 约 20% | 结构化去重 |
| 观察窗口 | 约 50% | 摘要加哈希引用 |

四个区块中只有观察窗口允许被主动清空重填，其余三个代表 Agent 的身份、意图与记忆。牺牲它们的直接表现是任务失败率上升而不是成本下降，这是配比隔离的验收判据。

### 7.2.2 观察摘要化的收益推导

**建模动机**：观察治理的本质是一笔租买决策——花一次摘要的钱，省掉后面每一轮的重发。设原始观察大小 T，摘要后大小 T_sum，摘要调用用小模型单价 p_s，主模型单价 p，观察产生后任务还剩 K 轮调用。全量携带成本 $c_f \approx T \times p \times K$，因为观察留在历史里每轮重发；摘要化成本 $c_m \approx T \times p_s + T_{sum} \times p \times K$，第一项是一次性摘要调用。

推导 $c_m < c_f$ 得到摘要化划算条件：**$T_{sum} / T < 1 - (p_s / p) / K$**。代入 $p_s = p/10$、$K = 10$，右端约 0.99，即只要摘要真的变小就赢；K 越小条件越严，唯一会输的情形是任务只剩一两轮且摘要几乎没变小。实践中 K 通常以十计，摘要化几乎恒为正收益，真正的约束不在钱而在信息。

推导只算了钱，没算信息损失：摘要丢掉一个弱口令提示，下游攻击图就少一条边。工程对策有二。其一，把摘要做成**结构化抽取**——主机、端口、凭据、漏洞按固定 schema 输出——而不是自由缩写，损失可审计；其二，对原始观察做**哈希引用**：SHA-256 指针留在上下文，原文落盘，Agent 需要时用一次工具调用取回。取回是摊销的，绝大多数观察一次都不会被回看，为它们保留全量历史是纯浪费。

## 7.3 模型分级路由的经济学

建模动机：规划与工具编排需要强推理，结果抽取、分类、摘要这类窄任务不需要，两类调用的质量要求相差悬殊而单价相差同样悬殊，混用同一种模型必然在某一侧浪费。设贵模型承担的费用比例为 α，则混合成本 **$C_{mix} = \alpha \times C_{big} + (1 - \alpha) \times C_{small}$**。与全贵模型相比，只要 $\alpha < 1$ 成本必然更低，这一步是平凡的，真正的约束在质量侧。

路由判据必须端到端：便宜模型在单次调用上的质量下降会沿流水线传播，端口抽取错一个，规划就少一条边，最终损失远超单次质量差。某类调用路由到小模型的条件是：质量差乘以下游期望损失，小于单价差乘以调用次数，即**省钱必须覆盖出错**。质量不可观测时该判据退化成拍脑袋，必须用任务成功率作回归指标来校准。

两个进阶。级联路由：先小模型，置信度不足再升级大模型，α 成为自适应变量通常优于固定配比；路由稳定性：同一类调用固定同一模型，模型混排会破坏 7.4

节的前缀缓存，省下的模型费可能加倍吐回给缓存未命中。失效条件是小模型在长尾格式上的失败率上升，重试反而更贵，级联阈值要按实测重试成本校准。

## 7.4 缓存与复用

### 7.4.1 prompt prefix caching

主流厂商对完全相同的前缀提供缓存价：命中部分通常按原价的 10% 到 50% 计费，写入可能有小幅加价，具体以厂商计费文档为准。命中条件是**前缀逐字节一致**，由此导出工程纪律：system prompt 与工具 schema 放在最前且内容不可变，任何时间戳、随机 ID 或动态统计都会杀死缓存；对话历史天然只增不改，Agent 的循环结构恰好缓存友好，每轮请求的前缀就是上一轮的全文。

做好缓存，7.1.1 节平方项里历史部分的有效单价能降一个台阶，这是唯一不动算法就能拿到的折扣。反例是上下文重排：把最新观察插到历史中间、或按相关度重排记忆条目，都会使前缀失配，缓存从全程命中退化为全程未命中，成本凭空翻倍。上下文管理策略一旦上线，变更前必须先算缓存账。

### 7.4.2 工具结果缓存与跨任务记忆

工具层缓存与模型层缓存正交。键取工具名、目标与参数，值带 TTL：同目标同参数在 TTL 内不重扫，直接回放。TTL 按数据挥发性设定，端口与漏洞结论可按小时到天计，进程列表只够分钟计。收益是三重的：省 token、省墙钟时间、减少对目标环境的扫描噪声，最后一点在授权测试里直接降低被检测与合规风险。命中要在观察里显式标注来源与时间，避免 Agent 把旧数据当新发现。

跨任务长期记忆解决重复侦察：主机清单、已确认凭据、已验证漏洞这类结论沉淀到持久存储，下一个任务热启动，跳过大段侦察。记忆的风险是陈旧，每条记忆必须带时间戳与置信度，过期即降级为待验证线索而非事实。记忆命中的边际成本接近零，是单位发现成本指标里杠杆最大的单项。

## 7.5 预算熔断机制

预算对象要小而硬。**TokenBudget** 持有总配额，charge(n) 在每次 LLM 调用与重型工具调用前后扣减并记账，exhausted() 是熔断谓词，主循环每一步调用前检查。

两级阈值：达到预警线（如 80%）转入保守模式，观察摘要更激进、优先便宜模型、暂停低价值分支；达到 100% 硬熔断。熔断不是报错退出，而是**降级**：停止探索，强制进入汇总阶段，把已有发现整理成报告，明确标注任务被预算截断、覆盖了什么、没覆盖什么。

降级出报告的理由是期望价值：一份不完整但结构化的报告可以交接、复现、续跑，而超支失控或中途崩溃的任务什么都留不下，**有损但不失控**是预算机制的验收标准。熔断同时是反死循环装置：Agent 陷入同一失败工具的重试循环时，预算耗尽会强制终止，这比收敛检测更兜底。最易漏的工程细节是给报告阶段预留配额，否则熔断时连汇总都产不出，降级路径本身先失败。

## 7.6 迭代控制与自省的成本权衡

自省机制（如 Reflexion 式的重反思与重规划）每一轮的完整成本包括三块：带着更长历史重新规划、按新计划重新执行工具、红队场景下暴露面增加。设一轮成本 c_r，收益是新增信息带来的期望损失下降或成功率提升 g_r。**继续迭代的条件是 E[g_r] > c_r**。工程含义：只有自省能触发新的工具动作、获取新的外部证据时，收益才可能为正；纯提示词层面的自我批评不产生新信息，大多在改写已有失败描述，期望收益近零而成本照付。

停止条件双保险。**改进停滞检测**：连续两轮收益低于阈值，或状态无变化——同一工具、同一报错、同一结论——立即停止；**最大迭代数**是硬上限，兜住分布长尾里最坏的任务。迭代预算要独立记账：一轮自省烧的 token 与普通轮次无异，但有独立的失败模式即原地打转，需要独立的计数器与告警，而不是藏在总预算里被动等熔断。

## 7.7 并行分支的 token 复利

Send 扇出让 N 个子代理并行，token 侧是**乘法**：每个分支持有独立上下文，都要重付 system prompt、工具 schema 与自身历史，父代理还要付聚合成本。N 等于 8、流水线三段时，扇出段的 token 近似乘 8。并行买的是墙钟时间，付出的是成本倍增与上下文重复，两本账要分开算：分支任务彼此独立、单分支上下文小、聚合便宜时并行划算，反之串行更省。

控制总量的关键是**分支内上下文隔离**。Send 子状态要裁剪：传给每个分支的只有它的子目标、目标资产与相关发现，而不是父上下文全文；分支返回时只交结构化结论，几百 token 量级，而不是完整过程转录。扇出前估算分支预算并在子状态里下发各自的 TokenBudget，防止单个分支吃掉全局配额，这是并行架构下熔断粒度的最小单位。

## 7.8 度量体系

成本控制没有度量就是玄学。每个任务结束输出 token 报表，按三个维度切分：按阶段看侦察、利用、汇报各占多少；按工具看哪个工具的观察最贵；按模型看大小模型的调用次数与费用占比。衍生指标包括缓存命中率、熔断触发次数与重试次数。报表的作用是**归因**：某工具原始输出占了近半预算，就给它包一层摘要器；某阶段占比异常，就去查它的上下文管理策略。

运营北极星指标是**单位发现成本**：任务总成本除以有效发现数，有效发现指经过验证、可行动的结论，而不是原始告警条数。该指标把花费与价值直接挂钩，能同时惩罚两类失败：花得多发现得少，以及发现多但全是不可行动的噪声。按周跟踪趋势，回归往往先在单位发现成本上显形，然后才反映到任务成功率上，适合作为预警指标。

## 7.9 与项目代码的对应

本章机制在项目代码中落点明确。config.py 的 **BudgetConfig** 声明总配额、预警比例与阶段配比，是 7.2.1 配比留位与 7.5 两级阈值的配置载体；circuit_breaker.py 的 **TokenBudget** 提供 charge 与 exhausted，charge 在每次调用记账，exhausted 被主循环每步轮询，触发即切换降级路径；llm.py 的 **Usage** 累计每次调用的输入输出 token 并按模型分桶，是 7.8 节报表的数据源。对应关系如下表：

| 机制 | 代码落点 | 职责 |
| --- | --- | --- |
| 预算配置 | config.py 的 BudgetConfig | 配额与阈值声明 |
| 熔断记账 | circuit_breaker.py 的 TokenBudget | charge 与 exhausted |
| 用量统计 | llm.py 的 Usage | 调用级 token 累计 |

三者构成闭环：Usage 产生数据，BudgetConfig 定义约束，TokenBudget 执行约束并在触界时改变控制流。观察摘要化、工具缓存与模型路由散布在工具层与调度层，对本章成本模型透明，只要 Usage 记账完整，报表就能覆盖它们的效果。

一份可直接对齐 BudgetConfig 的预算配置示例如下：

```
budget:
  max_total_tokens: 1500000      # 单任务硬上限，100% 熔断
  warn_ratio: 0.8                # 80% 进入保守模式
  report_reserve: 30000          # 为汇总报告预留配额
stages:
  recon: 0.3                     # 侦察阶段配比
  exploit: 0.5
  report: 0.2
degrade:
  on_exhausted: summarize_and_report
  aggressive_summary: true
```

## 7.10 本章速记

- 单任务成本 C = Σ_阶段 (调用次数 × 每调用 token × 单价)，四个杠杆各管一项。
- token 构成拆为 system、schema、历史、观察、输出五块，观察是安全 Agent 的大头。
- 无裁剪时任务总输入对轮数是平方级，这是 Agent 成本失控的数学根源。
- 观察摘要化划算条件：T_sum/T < 1 − (p_s/p)/K，任务越长条件越松。
- 摘要要结构化抽取加哈希引用原文，自由缩写的信息损失不可控。
- 模型分级成本 C_mix = α × C_big + (1 − α) × C_small，路由判据是省钱覆盖出错。
- 级联路由按置信度先小后大，通常优于固定配比，阈值要用实测校准。
- prefix caching 命中要求前缀逐字节一致，system 区严禁任何动态内容。
- 上下文重排会杀死缓存，改上下文管理策略前必须先算缓存账。
- 工具结果缓存键为工具加目标加参数，TTL 内不重扫，还顺带降低扫描噪声。
- 长期记忆热启动省掉重复侦察，每条记忆必须带时间戳与置信度。

- TokenBudget 用 charge 记账、exhausted 熔断，主循环每步调用前必查。
- 熔断降级是汇总出报告并标注覆盖范围，有损但不失控是验收标准。
- 报告阶段要预留配额，否则熔断时连汇总都产不出。
- 自省迭代持续条件 E[g_r] > c_r，无新证据的自省收益近零成本照付。
- 停止条件双保险：改进停滞检测加最大迭代数硬上限。
- Send 扇出的 token 成本是乘法，子状态裁剪与结构化回传控制总量。
- 报表按阶段、工具、模型三维度切分，单位发现成本是运营北极星指标。

# 第 8 章 MCP 插件集群：协议、热加载与安全治理

安全智能体的能力扩展面极广：侦察、指纹、情报、流量分析、样本分析、路径推演，每一类背后都是不同的工具生态。如果每个能力都用私有接口硬编码进编排层，工具生态越大集成成本越高——M×N 的集成问题（M 个智能体框架 × N 个工具）。MCP（Model Context Protocol）把成本压成 M+N：工具实现一次协议，所有支持该协议的客户端都能消费。本章讲 MCP 的底层协议设计、安全智能体的 server 设计规范、插件热加载与治理。

## 8.1 协议分层：MCP 与 function calling 的关系

### 8.1.1 两层各管什么

function calling 是**模型决策层**的机制：工具的名字、描述、参数 schema 进入模型上下文，模型输出结构化调用意图。MCP 是**工具连接层**的协议：谁来托管工具进程、客户端如何发现与调用它们、消息如何封装与传回。两者是互补而非竞争——典型链路是模型输出 tool_call，编排层经 MCP client 转发给对应的 MCP server，执行结果作为工具消息回填模型。把这两层混为一谈的常见错误：以为接了 MCP 模型就会自动用工具——不会，模型看到的仍是上下文里的工具描述，MCP 只是把描述与执行的通道标准化了。

### 8.1.2 JSON-RPC 2.0 与生命周期

MCP 报文是 JSON-RPC 2.0：请求带 method 与 params、响应带 result 或 error、双向通知（notification）无 id。生命周期三步：**initialize 握手**（客户端声明协议版本与能力，server 回应自己的 capabilities，双方在能力集上达成一致）；**运行期**（tools/list 发现、tools/call 调用、notifications 感知工具列表变更）；**关闭**（stdio 传输下进程退出即会话终止）。

```
{"jsonrpc": "2.0", "id": 1, "method": "tools/call",
 "params": {"name": "nmap_scan",
            "arguments": {"target": "10.60.0.11", "ports": "1-1000"}}}
{"jsonrpc": "2.0", "id": 1,
 "result": {"content": [{"type": "text", "text": "{\"hosts\": {...}}"}],
"isError": false}}
```

**三原语**的语义差异必须分清：**tools** 是模型可主动调用的函数（有副作用，需审批注记）；**resources** 是可读取的上下文数据（URI 寻址、只读语义，如评估报告、ATT&CK 条目）；**prompts** 是预置提示词模板（用户主动选用）。安全 Agent 里 tools 必须全部走授权门，resources 也要做权限过滤（不是所有报告对所有用户可读）。

### 8.1.3 传输层：stdio 与 Streamable HTTP

stdio 传输：client 把 server 作为子进程启动，stdin/stdout 跑 JSON-RPC——本地场景最简单，进程即隔离单元（崩溃只死一个 server），缺点是生命周期绑定 client。Streamable HTTP：server 作为独立服务，支持远程部署与多客户端复用，用 SSE 流式回传，适合集中化的情报与扫描服务。Aegis 的选型逻辑：侦察与指纹类本地工具走 stdio（沙箱进程内运行），情报类集中服务走 HTTP（缓存与限流集中治理）。

## 8.2 工具描述的工程学

### 8.2.1 描述即提示词

模型选择哪个工具、怎么填参数，几乎完全由工具的 name、description、inputSchema 决定——**工具描述就是写给模型的提示词**。安全工具的描述规范三条：第一，说清能力边界与前置条件（"对授权目标执行 TCP 端口扫描；目标必须已过 ROE 校验"——把约束写进描述能显著降低越界调用尝试）；第二，参数语义精确（target 是 IP/CIDR/域名、ports 的格式、超时默认值）；第三，schema 用足约束力——枚举（action 类型）、required（必填目标）、格式声明（CIDR），schema 约束是**幻觉参数的第一道防线**：受限解码下枚举字段不可能出现枚举外的值。

### 8.2.2 输出体积与分页

扫描类工具的原始输出动辄几十 KB（nmap XML、PCAP 统计），全量回填上下文会瞬间吃掉窗口与预算。server 侧的输出治理三条：结构化摘要优先（统计 + top-N + 异常标记）、原始输出落盘并以哈希引用（模型需要时再分页取）、超长字符串

截断。这与第 7 章的 Token 经济直接挂钩：观察摘要化是安全 Agent 最大的单项成本优化。

## 8.3 安全 Agent 的 server 设计规范

Aegis 的 MCP server 五条军规：**每工具声明 ROE 动作类型与风险等级**（recon/fingerprint/.../low/medium/high，schema 里作为元数据携带，客户端据此路由审批）；**高危工具必须带审批注记**（annotations 里的确认提示，客户端渲染给人类）；**目标参数强制校验**（server 侧也做 ROE 校验，不只依赖 client——纵深防御，server 不信任 client）；**超时与取消语义明确**（长任务支持 progress 通知与 cancellation，扫描超时返回部分结果加标记）；**无状态优先**（server 不存任务状态，状态归编排层，server 可随时重启）。

> 面试官：客户端已经做了授权门，server 为什么还要再做一遍校验？

答要点：纵深防御与信任边界——client 可能被提示注入诱导、可能是未经过审计的第三方实现、client 与 server 之间的通道可能被错误配置；server 侧以自己的 ROE 副本再校验一次，把「安全性质」锚定在离工具最近的一跳。两层校验的规则来源（ROE 文件）必须同源，否则会出现规则漂移。

## 8.4 插件热加载与治理

### 8.4.1 热加载机制

热加载三层设计：**进程级隔离**（每 server 独立进程，新版本 server 启动健康后旧版本退出，崩溃不传染编排层）；**配置驱动注册**（plugins.yaml 声明 server 清单与版本，编排层启动时按配置拉起，改配置即生效）；**能力变更通知**（server 工具列表变化经 notification 推送，client 刷新工具注册表并触发熔断器重建）。Aegis 当前实现是进程内注册表（ToolBus.register），演进路径就是把每个插件抽出为独立 stdio server——ToolBus 的门面不变，调用从函数分发变为 MCP client 转发。

### 8.4.2 恶意 server 的威胁模型

MCP 生态的供应链风险必须正视：**工具描述注入**（恶意 server 在 description 里埋提示词，诱导模型执行越界动作）；**rug pull**（可信 server 发布后更新为恶意行为，client 无感知）；**命名仿冒**（仿冒知名 server 名字混入配置）。治理四条：server 来源白名单与签名校验、工具描述变更视为高危事件需人工确认、最小暴露原则（一次评估任务只挂必需的 server，而不是全量常驻）、所有工具调用过审计链（含 server 版本指纹，事后可归因）。这条威胁模型与第 17 章供应链安全的结论同构：信任必须锚定在构建与分发过程上。

## 8.5 本章速记

- MCP 把 M×N 集成压成 M+N：工具实现一次协议处处可用
- function calling 是模型决策层，MCP 是工具连接层，互补不互替
- JSON-RPC 2.0 加 initialize 能力协商；三原语 tools、resources、prompts 语义不同
- tools 有副作用走审批，resources 只读走权限过滤
- stdio 进程即隔离单元，Streamable HTTP 适合集中情报服务
- 工具描述就是提示词，能力边界与前置条件写进 description
- schema 枚举与 required 是幻觉参数的第一道防线（受限解码）
- 扫描输出必须摘要化加哈希引用，全量回填是 Token 灾难
- server 五军规：声明 ROE 动作与风险、高危审批注记、目标自校验、超时取消、无状态
- client 的授权门不是豁免 server 校验的理由：纵深防御锚定最近一跳
- 两层校验的 ROE 必须同源，防规则漂移
- 热加载三层：进程隔离、配置驱动、能力变更通知
- 恶意 server 三风险：描述注入、rug pull、命名仿冒
- 治理四条：来源白名单签名、描述变更人工确认、最小暴露、全量审计链
- ToolBus 的门面不变即可演进为 MCP：门面稳定是重构的前提

# 第 9 章 情报与知识对接：ATT&CK、CVE 与威胁情报库

智能体的判断力上限取决于它连接的知识底座：攻击手法要不要映射到标准框架（可度量、可沟通）、漏洞候选来自哪里（可靠且新鲜）、历史经验如何沉淀与复用。本章讲三大知识源的工程对接：MITRE ATT&CK（对手行为标准语言）、CVE/NVD（漏洞事实库）、威胁情报（对手画像），以及混合检索架构与知识更新管道。

## 9.1 MITRE ATT&CK 的数据模型

### 9.1.1 结构与编号体系

ATT&CK Enterprise 矩阵按两级组织：**战术**（tactic，攻击者要达成什么，TA 编号如 TA0043 Reconnaissance）与**技术**（technique，怎么达成，T 编号如 T1595），技术下再有**子技术**（T1059.001 PowerShell）。每个技术携带：描述、检测建议、数据源（data source 与组件）、缓解措施，以及 Groups（已知团伙）与 Software（恶意软件与工具）的关系对象。这套结构的价值是**统一语言**：红队按 T 编号报告、蓝队按 T 编号建检测、管理层按战术叙事汇报，三张报表说的是同一套编号。

数据以 **STIX 2.1** 格式发布：SDO（域对象：Attack Pattern、Malware、Intrusion Set）、SCO（网络对象：IP、域名、文件哈希）、SRO（关系对象：把 SDO 连成图）STIX bundle 是自包含 JSON 文件——Aegis 的做法是把官方 Enterprise bundle 离线加载进本地索引（JSON 解析后按对象类型分表），评估期不依赖外网。

### 9.1.2 工程消费三模式

**发现到战术的映射**：每个 finding 携带 attack_tactics 字段（recon 类发现映射 Reconnaissance、beacon 检测映射 Command and Control、路径推演映射 Lateral Movement），报告的 ATT&CK 映射节即此字段的去重汇总。**覆盖度评估**：以适用战术为行、检测状态为列输出热力图（有检测、有日志无规则、无日志），指导建设优先级。**导航器输出**：按 ATT&CK Navigator 的 layer JSON 格式导出覆盖视图，可直接可视化汇报。工程实现要点：映射不是文本匹配而是结构化字段——发现生成时

就由节点标注战术，而不是事后用 LLM 从描述里猜（LLM 映射可以作为补录，标注置信度）。

## 9.2 CVE 与 NVD 对接

### 9.2.1 记录结构与查询模式

CVE 记录的关键结构：受影响产品的 **CPE 配置**（cpe:2.3:a:nginx:nginx:1.18.0:*:*:*:*:*:*:* 的标准化命名）、**CVSS 向量**（v3.1/v4 的分指标打分）、**CWE 关联**（弱点类型）、参考链接。NVD API 2.0 的查询参数：keywordSearch（自由词）、cpeName（精确产品版本）、pubStartDate/pubEndDate（时间窗）、resultsPerPage 与 startIndex（分页）；匿名调用有速率限制，生产必须带 API key 加本地缓存。

**CPE 对齐是指纹探测的价值闭环**：http_fingerprint 产出 "nginx 1.24" 这类产品指纹 → CPE 匹配（产品名归一化 + 版本区间判断）→ cpeName 精确查询 → 候选 CVE 列表。指纹的粒度决定匹配质量：只有 "nginx" 没有 "1.24" 时会召回大量不适用版本——所以指纹插件要尽力提取版本号，模糊时宁可降级为 keyword 查询加人工复核，也不要误报候选。

### 9.2.2 验证闭环与降级策略

**外部情报是假设不是结论**——这是幻觉抑制原则在情报面的落地：NVD 召回的 CVE 只是候选（CPE 匹配本身可能因指纹误差而失真），必须经 poc_verify 独立验证才能升级为已确认发现；验证不了的标注"待验证"进入报告，绝不允许 LLM 直接断言"存在该漏洞"。降级策略：NVD 不可达时 cve_enrich 静默失败（发现保持无 CVE 状态继续主流程），绝不因情报源故障阻塞评估——情报是增强不是依赖。


```
{"vulnerabilities": [{"cve": {"id": "CVE-2021-23017",
  "metrics": {"cvssMetricV31": [{"cvssData": {"baseScore": 7.0,
   "vectorString": "CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:H"}}]},
  "configurations": [{"nodes": [{"cpeMatch": [{"criteria":
   "cpe:2.3:a:nginx:nginx:1.18.0:*:*:*:*:*:*:*", "vulnerable": true}]}]}]}}]}
```

## 9.3 威胁情报库的工程消费

### 9.3.1 IOC 与 TTP 的分工

情报两种粒度用法截然不同：**IOC**（IP、域名、哈希）可直接匹配但寿命短（对手换基础设施即失效），用于实时匹配与历史回溯；**TTP**（对手行为方式）寿命长但需要检测规则承载，用于检测建设与对手画像。Aegis 的用法：IOC 接入做出口流量与样本哈希的比对（命中即 finding），TTP 接入做蓝队仿真时检测规则的生成依据。情报质量三指标——新鲜度（时间戳衰减）、准确性（误报率）、相关性（行业匹配）——接入免费源时先跑误报评估再上线，否则污染告警流。

### 9.3.2 私有情报的沉淀

评估产生的自有情报最有价值：某目标确实存在的漏洞模式、某客户的弱口令分布、某次评估的反思教训——回写为私有 STIX 对象与向量记忆条目，下次同类任务的 planner 直接命中（memory_recall 的输入就是目标特征）。这就是长期记忆的复利：情报不是只进不出的管道，而是**评估→沉淀→检索→加速下一轮**的飞轮。

## 9.4 混合检索架构

关键词倒排与向量语义检索各有盲区：倒排抓不住同义改写（"横向移动"vs"lateral movement"），向量对专有名词与编号不敏感（"CVE-2021-23017" 的语义向量毫无区分度）。混合检索管线四步：**双路召回**（倒排 top-k 加向量 top-k）→ **RRF 融合**（倒数排名融合，无需调分数尺度）→ **rerank**（交叉编码器对候选精排，只对 top 几十条做，成本可控）→ **引用溯源**（每条检索结果携带来源对象 ID，供报告引用与人工核查）。嵌入模型选择看中文效果与维度成本，rerank 用轻量交叉编码器即可显著提升前几名质量。

### 9.4.1 知识更新管道

三个源的更新节奏不同：ATT&CK 季度大版本、NVD 持续日增、情报源实时流。更新管道设计：版本化知识库（新版本构建后跑评测集回归再切换）、情报源 TTL 与失效处理（IOC 过期自动降权）、评测集回归（固定一组检索问答对，防更新引入

检索质量回退）。知识库的版本号进入报告元数据——三个月后复查报告时能知道当时依据的是哪个版本的情报。

## 9.5 本章速记

- ATT&CK 两级结构加子技术，TA 与 T 编号是攻防统一语言
- STIX 2.1 三类对象：SDO 域对象、SCO 网络对象、SRO 关系
- 发现到战术的映射在生成时结构化标注，不做事后 LLM 猜测
- 覆盖热力图与导航器输出是检测建设的度量与汇报载体
- CPE 对齐是指纹到 CVE 的价值闭环，版本粒度决定匹配质量
- NVD 查询带 API key 加本地缓存；情报是增强不是依赖，故障静默降级
- 外部情报是假设不是结论：候选 CVE 必须独立验证才能升级
- 验证不了的标注待验证，绝不让 LLM 直接断言漏洞存在
- IOC 短命用于匹配，TTP 长命用于检测建设，用法截然不同
- 情报三指标：新鲜度、准确性、相关性；免费源先评误报再上线
- 私有情报回写形成评估飞轮：评估、沉淀、检索、加速下一轮
- 混合检索四步：双路召回、RRF 融合、rerank 精排、引用溯源
- 编号与专有名词靠倒排，同义改写靠向量，缺一不可
- 知识库版本化加评测集回归，版本号进报告元数据

# 第 10 章 能力子系统 I：侦察、漏洞挖掘与 POC 验证

## 10.1 侦察子系统：从公开信息到攻击面清单

侦察子系统处于 Aegis 能力链的最上游，输入是 ROE 界定的授权范围，输出是唯一面向下游的结构化产物——攻击面清单。本节先讲被动与主动两条数据通路如何汇聚为清单，再讲授权门如何卡住主动动作。整个子系统遵循一个基本原则：不产生任何超出授权范围的流量，所有主动动作先取得令牌后执行，执行细节全程可审计。

### 10.1.1 被动情报收集与信息增益排序

被动收集不与目标直接交互，数据全部来自第三方数据源与自有授权库。**证书透明度日志**披露子域与证书签发历史，公开记录（WHOIS、被动 DNS）补充资产归属与解析变迁，外部资产测绘聚合组织口径的暴露面。由于这类动作不对目标产生请求流量，引擎默认放行，但仍逐条写入审计流，保持证据完整。

三类被动源与其在攻击面构建中的角色如下表所示。

| 情报源 | 关键产出 | 对清单的贡献 | 更新方式 |
| --- | --- | --- | --- |
| 证书透明度 | 子域与签发历史 | 边缘域名候选 | 订阅增量 |
| 公开记录 | 归属与解析变迁 | 资产归属锚点 | 周期拉取 |
| 资产测绘 | 组织暴露面汇总 | IP 段候选清单 | 授权同步 |

表中产出全部携带来源标识与时间戳，进入统一候选池等待信息增益排序。来源与时间戳是后续审计回放和置信度评估的依据，任何字段缺失的候选不会进入主动探测队列，只作为背景情报留存，候选池的每一次进出都有记录可查。

**信息增益排序**决定哪些候选值得消耗主动探测预算。每个候选按三个因子计分：新增可达面的数量、与已确认资产的重合度、情报时效性。高分项推进主动探测队列，低分项沉淀为背景情报。排序在代码中完成，模型只参与对模糊描述（例如证书 CN 与业务命名风格的归并）的判断，保证打分过程可复现。

### 10.1.2 主动探测与频次控制

主动探测开始对目标产生真实流量，因此是授权与频次控制的第一现场。机制上分两步：**端口发现**输出主机、端口、协议三元组，只做连通性判定；**服务识别**对开放端口发送最小化探针并解析响应特征。两步各自独立授权、独立限速，避免识别阶段的探针放大，也为审计提供清晰的动作粒度。

频次控制用**令牌桶**实现。ROE 声明每个目标的探测速率上限与时间窗，引擎按声明发放令牌；同一目标的并发探针数量单独受限，防止分析分支并行扇出时的流量叠加。令牌耗尽时动作排队等待而非丢弃，保证探测节奏可预期、可回放，也让速率异常在审计中一眼可见。

### 10.1.3 攻击面清单与 ROE acquire

**攻击面清单**是侦察子系统的唯一出口，下游所有子系统只消费清单，不直接接触原始探测记录。每条记录包含目标、服务、粗粒度版本、可达性四要素，并附带证据 ID 与采集时间。可达性区分边缘可达、内网可达与需跳板三类，直接决定后续验证动作的执行位置。清单核心字段定义如下。

| 字段 | 含义 | 由哪步产出 |
| --- | --- | --- |
| 目标 | IP 或域名标识 | 被动与主动汇聚 |
| 服务 | 端口上的协议与服务 | 服务识别 |
| 版本 | 粗粒度版本区间 | 指纹初判 |
| 可达性 | 执行位置约束 | 探测路径记录 |

字段粒度刻意保持保守，精确版本留待指纹子系统修正；可达性字段直接决定动作在沙箱还是授权环境中执行，同时受 ROE 时间窗约束。任何字段在进入下游消费前都会再做一次完整性校验，防止脏数据污染整条分析链路。

**ROE acquire** 是主动动作的统一入口。状态机在调度任何主动动作前调用 ROE 引擎，校验范围、时间窗、速率与动作等级；通过后签发令牌，令牌绑定目标、动作类型与参数哈希，随动作进入审计流，执行完成后销毁。校验失败则动作短路，不进入执行队列。令牌机制把授权从事前约定变成每次动作的运行时检查，是安全四件套中的第一道门。

## 10.2 指纹探测子系统

### 10.2.1 HTTP 指纹：三类信号融合

HTTP 指纹从响应中提取三类独立信号。响应头特征包括 Server 与 X-Powered-By 等显式声明，也包括头序与缺失头模式这类隐式特征；内容特征覆盖静态资源路径、错误页文案与生成器标记；行为特征观察对特定方法与路径组合的差异化响应。三类信号分别打分后融合，输出技术栈清单与版本区间，供归一化层消费。

版本输出刻意保持为**区间**而非精确值。指纹信号通常只能证明不早于某版本或不晚于某版本这类边界，精确断言容易引入误报。区间化版本进入漏洞关联阶段后按保守策略匹配，宁可多报候选、由验证阶段收敛，也不在指纹层过早下结论。

安全头检查与指纹采集同轮完成。CSP、HSTS、X-Content-Type-Options 等逐项核对，缺失即生成**加固类 finding** 并附修复建议。这类 finding 不依赖漏洞库、不触发高危验证，是侦察侧最稳定的报告产出，也常作为与目标方沟通的第一批低风险结论。

### 10.2.2 指纹到 CPE 的归一化与误报控制

第 9 章的漏洞关联以 **CPE** 为统一口径，指纹子系统输出的产品名与版本区间必须经过归一化层转换：vendor 与 product 锚定、版本区间到 CPE 版本范围的换算、多候选时的歧义消解。归一化层维护映射缓存，同一指纹只换算一次，保证全链路对同一资产的称呼一致，避免同名异义导致的错配。

误报控制依赖**多信号交叉确认**。单一信号只能形成假设，至少两类独立信号一致才升格为结论，结论附带置信度分档。低置信度条目进入待验证队列，而不是直接消费给漏洞关联，避免一次错误指纹污染整条下游链路。交叉确认的判定逻辑全部在代码中实现，不交给模型，保证规则可审查。

## 10.3 漏洞挖掘子系统分层

漏洞挖掘子系统分四层，风险与成本自下而上递增，上层只消费下层的结构化输出。分层的目的有二：一是把无害检测与有扰动检测分开授权，让 ROE 按层设门；二是让 LLM 只出现在策略与假设环节，不直接执行任何动作。四层的输入、输出与风险控制如下表。

| 层级 | 输入 | 输出 | 风险控制 |
| --- | --- | --- | --- |
| 已知漏洞关联 | CPE 与版本区间 | CVE 候选清单 | 只读检索 |
| 配置类检测 | 清单与指纹结果 | 加固 finding | 只读不写 |
| 模糊测试辅助 | 参数面提取结果 | 异常行为线索 | 沙箱执行 |
| 逻辑假设生成 | API 面与业务描述 | 假设清单 | 人工确认 |

表中四层自下而上构成依赖链，任何一层失效只影响其上层，不会向下传播。层的边界同时是授权边界，ROE 按层设置动作等级，跨层动作必须重新走授权与审批流程。

### 10.3.1 已知漏洞关联

已知漏洞关联是四层中最成熟的一层。输入是归一化后的 CPE 与版本区间，检索流程在第 9 章已闭环：知识层拉取 CVE 记录、按版本区间保守匹配、输出候选列表。本层的增量工作有两项：按可达性与暴露面过滤候选，给每个候选标注验证优先级，然后把高优先级候选交给 10.4 节的验证器收敛，形成从指纹到确认的完整闭环。

### 10.3.2 配置类缺陷检测

配置类检测**只检测、不改配置**。安全头检查沿用指纹阶段结果；TLS 配置检查对目标做标准握手，记录协议版本、套件选择与证书链完整性，握手动作无害且不在目标侧落地任何状态；默认凭据检查只识别默认页、公开文档路径等凭据存在线索，不做任何凭据尝试。所有检测产出均为 finding 与修复建议，配置变更永远留给目标方执行。

### 10.3.3 模糊测试辅助

模糊测试在本架构中的定位是辅助发现而非主要漏洞来源。流程分三段：参数面提取从清单与指纹结果中收敛可测的参数、头部与接口；异常输入的**生成策略**由 LLM 辅助，覆盖类型混淆、边界值与异常编码等类别，产出的是策略描述而非输入本身；实际执行全部收敛在沙箱内，对目标副本或专用测试环境注入。生成与执行严格分离，是本层的安全底线。

### 10.3.4 面向 LLM 的逻辑漏洞假设生成

业务逻辑漏洞没有特征签名可匹配，只能靠假设驱动。LLM 基于 API 面描述、业务流程文档与权限模型提出**假设清单**，例如水平越权、流程跳过与状态竞态等类别，每条假设附带可观测的判据与验证思路。假设必须经人工确认后才进入验证队列，验证动作同样走 ROE 与审批门。模型的价值在于把散落的接口语义组织成有针对性的假设，而不是替代验证本身。

## 10.4 POC 自主编写与验证

POC 验证是把候选收敛为结论的唯一手段。本节按机制顺序讲解验证器的结构、沙箱内执行、三态结果与脚本安全边界。全节只有一个目标：让 CONFIRMED 这个词在报告里具有确定的工程含义，任何绕过状态机的解读都不被允许。

### 10.4.1 验证器结构与 CONFIRMED 语义

**验证器**是可独立执行的最小程序，结构固定为三段。

19. 前置条件检查：确认版本区间、可达性与 ROE 范围全部成立。

20. 无害化验证动作：只读探测，不在目标侧落地持久化改动。

21. 断言判定：对响应的确定性特征逐条断言，输出布尔结果。

**CONFIRMED 语义**定义为三段同时成立：前置条件满足、动作确实执行、断言全部通过。任何一环不满足都禁止输出 CONFIRMED，该语义由判定函数强制保证而非由模型自由发挥，这是验证结论可信度的根基。

### 10.4.2 沙箱内执行

验证器默认在沙箱内执行。隔离网络默认无外网出口，目标以本地 fixture 呈现，fixture 从指纹与配置类检测结果自动生成；根文件系统只读，写入重定向到临时层超时强杀配合 CPU、内存与进程数限额，防止验证器本身失控。对真实目标的最小化验证动作只在沙箱结论为 confirmed 之后、经人工审批后执行，且复用同一验证器与同一断言集，保证两轮结果可比。

### 10.4.3 三态结果与状态机更新

验证结果只有三态。**confirmed** 表示漏洞成立，状态机写入漏洞清单并触发报告流程；**refuted** 表示候选被排除，状态机关闭候选并保留反驳证据；**inconclusive** 表示证据不足，候选保持排队并记录证据缺口，等待补充信息后重验。三态之外的任何运行异常一律按 inconclusive 处理，防止部分失败被误读为结论。

三态判定由纯函数实现，输入是前置条件检查结果、动作执行标志与断言列表，输出唯一确定。纯函数没有副作用，因而可以脱离沙箱做单元测试，也可以在审计回放中用同一输入复现同一判定，这是验证语义可质证的技术前提。

```
def judge(prec_ok: bool, acted_ok: bool, asserts: list) -> str:
    """验证器三态判定：confirmed、refuted 或 inconclusive。"""
    if not prec_ok or not acted_ok:
        return "inconclusive"    # 前置不满足或动作未执行，禁止下结论
    passed = [ok for ok, _ in asserts]
    if all(passed):
        return "confirmed"       # 断言全部通过才允许确认
    if any(passed):
        return "inconclusive"    # 证据冲突时保持未决，不部分确认
    return "refuted"             # 断言全部不成立，排除该候选
```

判定函数无副作用，状态机只依据返回值驱动转移，判定逻辑因此与执行环境解耦。固化的判定语义让任何一次 CONFIRMED 都能追溯到具体断言集合，报告的严谨性由此建立。

### 10.4.4 证据保留与脚本安全边界

验证不通过时证据必须完整保留：请求与响应的脱敏快照、断言差异明细、沙箱环境指纹与时间戳共同构成证据包，写入审计哈希链。inconclusive 的证据包同样保留，它既是补验与人工复核的输入，也是报告中解释覆盖边界的依据。

LLM 生成验证脚本受三重边界约束。**模板化**：模型只填充声明式模板中的动作与断言字段，不生成自由代码；**人工审批**：模板实例经 interrupt 呈现动作语义与参数摘要，审批通过才落盘执行；**白名单 API**：执行器只暴露白名单的网络原语与断言原语，越权调用在运行时直接拒绝。三重边界叠加后，模型输出永远不直接等价于可执行指令。

## 10.5 全链路的授权与审计

授权与审计以两个横切面贯穿本章所有动作。授权面：动作调度前 ROE 校验并签发令牌，执行中令牌随调用链传递，执行后销毁并回写结果，高危动作额外触发 interrupt 审批。审计面：每条记录包含动作摘要与结果摘要，摘要即哈希，记录之间以哈希链接续，任何事后篡改都会破坏链完整性。

interrupt 审批的人机交互遵循最小认知负担原则。审批卡片固定四要素：动作语义的自然语言解释、目标与参数摘要、预期影响与回滚说明、证据链接。审批人可以批准、拒绝或修改参数后批准；超时未响应按拒绝处理，动作退回队列。审批记录本身也进入审计哈希链，与动作记录一一对应，形成完整闭环。

## 10.6 失败降级与熔断

侦察与验证面对的目标异构且状态不可控，失败是常态而非异常。隔离机制保证单目标失败不拖垮整体：每个目标在 Send API 并行扇出的独立分支中执行，分支异常被捕获后固化为该目标的降级状态，主图继续推进其余目标。目标级异常不进入主图的异常路径，只作为数据参与最终汇总。

**熔断**以计数窗口实现。同类失败信号（超时、异常响应率、沙箱崩溃）在滑动窗口内累计，越过阈值即对该目标降级：停止主动探测与验证，仅保留被动情报与既有证据。降级产出的报告依然成立：以已收集证据出具部分覆盖报告，明确标注覆盖边界与降级原因，避免报告因单点失败而整体缺失。

## 10.7 本章速记

- 侦察子系统唯一出口是攻击面清单，下游不接触原始探测记录。
- 被动收集不产生对目标的流量，默认放行但仍全程审计。
- 信息增益排序决定主动探测预算的投向，打分逻辑在代码中可复现。
- 主动探测按令牌桶限速，令牌与 ROE 时间窗绑定。
- ROE acquire 把授权做成每次动作的运行时检查，令牌绑定参数哈希。
- 指纹输出版本区间而非精确值，精确匹配交给验证阶段收敛。
- 安全头缺失直接产出加固 finding，不依赖漏洞库即可稳定出报告。
- 指纹到 CPE 的归一化是第 9 章漏洞关联的口径前提。
- 单一指纹信号只是假设，多信号交叉确认后才成为结论。
- 漏洞挖掘四层按风险递增，上层只消费下层结构化输出。
- 配置类检测只读不写，配置变更永远留给目标方执行。
- 模糊测试生成策略与执行分离，执行只发生在沙箱内。
- 逻辑漏洞假设由 LLM 提出，人工确认后才进入验证队列。
- CONFIRMED 要求前置满足、动作执行、断言全过三段同时成立。
- 验证结果只有三态，任何异常一律按 inconclusive 处理。
- 验证证据包完整保留，inconclusive 证据同样进入审计链。
- 验证脚本走模板化、人工审批、白名单 API 三重边界。
- 审计记录以哈希链接续，篡改任何一条都会破坏完整性。
- interrupt 审批超时按拒绝处理，审批记录同样入链。
- 单目标失败触发熔断降级，降级后仍可出部分覆盖报告。

# 第 11 章 能力子系统 II：流量、样本与逆向分析

## 11.1 PCAP 深度分析管线

PCAP 管线的输入是授权采集的流量证据，输出是带证据 ID 的结构化流量情报。管线按确定性优先的原则组织：解析、统计、流重建、资产提取、异常检测五段全部由代码完成，LLM 只在下游消费结构化结果。这样设计的根本原因是流量分析的核心结论必须可复现、可回放、可质证。

### 11.1.1 协议统计与会话流重建

协议统计做分层聚合：链路层协议分布、传输层会话数与字节数、应用层协议占比。统计结果一方面提供全局视野，另一方面直接驱动采样决策——占比异常的协议段会被优先送入深度解析。**会话与流重建**把报文按五元组聚合成双向流，TCP 流按序列号重组并处理乱序与重传，输出可供应用层解析的字节流。

### 11.1.2 应用层资产提取

资产提取从重建后的流中拉出三类对象：HTTP 主机来自 Host 头与请求路径；DNS 查询记录域名、类型与响应地址；TLS 会话记录握手中的 SNI 与证书元数据。三类对象做去重与互链，形成 IP、域名、证书三者的**资产图谱**。图谱是后续横向移动分析与威胁情报比对的底座，也是报告里网络侧叙述的主线。

### 11.1.3 Beacon 周期检测

Beacon 检测的机制是周期稳定性度量。对每对内部主机与外部目的地的连接时间戳序列提取相邻间隔，计算变异系数 **CV=σ/μ**，即标准差与均值的比值。CV 越小，间隔越稳定，周期性越强；正常人工浏览的间隔方差大，CV 通常显著高于自动化回连，这一差异就是检测的物理基础。

阈值取舍需要平衡漏报与误报。阈值过严会漏掉带抖动的回连，过宽则把 NTP 同步、健康检查这类正常轮询误判为威胁。工程实践是三项叠加：CV 阈值按环境基线标定、最小样本数门槛防止小样本误判、**长基线窗口**覆盖多个工作日以排除工作时间模式。包大小方差作为次要信号参与复核，进一步压低误报。

检测核心由纯函数实现，输入是间隔序列，输出判定结果。

```
import statistics as st


def beacon_cv(intervals_ms: list) -> float:
    """变异系数 CV = 标准差 / 均值，度量间隔稳定性。"""
    mu = st.mean(intervals_ms)
    return float("inf") if mu == 0 else st.pstdev(intervals_ms) / mu


def is_beacon(intervals_ms: list, cv_max: float = 0.15, min_points: int = 8)
-> bool:
    """阈值与样本门槛为经验起点，须按环境基线标定。"""
    if len(intervals_ms) < min_points:
        return False            # 样本不足不判定，抑制误报
    return beacon_cv(intervals_ms) <= cv_max
```

阈值并非固定常量，而是部署时按环境基线标定的参数。检测器输出必须携带本次使用的阈值与样本量，报告层据此复现判定，也让阈值调优有据可查。

### 11.1.4 加密流量的元数据分析

流量加密后内容不可读，但元数据仍然丰富。JA3 与 JA4 的指纹思想是对 ClientHello 中的版本、套件、扩展等字段做规范化串联后哈希，输出客户端栈的稳定指纹，用于识别工具与恶意家族的已知客户端栈。SNI 与证书特征补充服务器侧画像：自签证书、有效期异常、证书 SAN 与域名不匹配都是可用线索。

元数据分析的产出是**加密流画像**，与资产图谱互链后进入情报比对。它不试图解密，而是回答谁在和谁通信、用什么栈通信、通信节奏如何这三个问题。合规上元数据来自授权采集的流量证据，分析动作无侵入，因此沿用第 10 章的被动放行与审计策略。

## 11.2 PCAP 与 LLM 的协作分工

分工原则是确定性归代码、生成性归模型。所有数值计算、阈值判定、指纹哈希与统计聚合都在代码里完成，保证可复现；模式解释与报告叙述由 LLM 完成，它擅长把异常统计组织成有因果结构的叙述。分工边界由数据形态天然划定：代码消费原始报文，模型只消费带证据 ID 的结构化中间产物。

可溯源是协作的关键约束。LLM 的每条结论必须引用**证据 ID**，报告层据此回链到原始流、时间戳与统计表；无证据引用的叙述在报告生成层被剔除。模型不接触

原始 PCAP 字节，上下文规模可控，幻觉即使发生也不会越过中间产物的边界，任何错误都可以被证据核对拦截。

## 11.3 恶意代码静态分析

### 11.3.1 哈希与模糊哈希、YARA 扫描

静态分析的第一层是身份与相似性。精确哈希（SHA-256）用于样本去重与情报库比对；**模糊哈希**采用分块滚动哈希思想，对局部改动不敏感，用于家族相似性聚类。YARA 规则扫描以规则库匹配字符串、十六进制片段与结构特征，命中即产出线索并关联规则来源。全部动作只读样本、不执行样本，因此按第 10 章的只读授权处理。

### 11.3.2 PE 结构解析与熵分析

PE 结构解析覆盖节表与导入表。节表给出名称、虚拟地址与可执行标志，逐节计算**熵值**：高熵节提示加壳或加密资源，熵值与节属性的异常组合是壳的典型形态。可疑 API 聚类从导入表抽取网络、进程操作与加密类调用的组合模式；字符串按语义聚类为 URL、路径与配置项等组别，供后续假设生成引用。

### 11.3.3 熵与混淆的关联解读及静态局限

熵与混淆的关联解读可以交给 LLM 辅助叙述，例如把高熵节、反调试字符串与异常导入组合解释为疑似加壳，但最终判定由规则给出，模型只负责把数值证据翻译成可读解释。静态结果的根本局限在于对抗：**加壳、混淆与运行时解密**可以让静态特征整体失效，因此静态结论永远标注为疑似，必须与动态沙箱行为互相印证后才能进入报告正文。

## 11.4 动态沙箱分析

### 11.4.1 一次性容器架构

动态沙箱以**一次性容器**为基本单元：样本执行完毕即销毁，不复用、不回滚。隔离网络使用 internal 网络段，默认无外网出口，出站目的由允许清单或本地伪造服

务承接；根文件系统只读，写入重定向到临时层；CPU、内存与进程数限额配合超时强杀，保证失控样本被硬终止且环境影响归零。

### 11.4.2 行为采集与结构化报告

行为采集按事件序列组织，覆盖四类事件：进程事件记录创建链与命令行，文件事件记录创建、修改与删除路径，注册表事件记录键值读写，网络事件记录连接目的与 DNS 请求。事件统一带时间戳排序，输出**结构化行为报告**，报告内嵌样本哈希与环境指纹，与静态分析结果共用证据 ID 体系。

### 11.4.3 沙箱选型与反沙箱对抗

API 级沙箱与容器级本地沙箱的取舍按检测深度与吞吐需求分层，两者在关键维度上的差异如下表。

| 维度 | API 级沙箱 | 容器级沙箱 |
| --- | --- | --- |
| 监控粒度 | 内核调用级 | 系统事件级 |
| 抗干扰 | 强 | 中等 |
| 部署成本 | 高 | 低 |
| 并行扩展 | 难 | 易 |

工程上以容器级沙箱做初筛，吞吐优先；API 级沙箱承接初筛命中的深检，深度优先。两层共享同一报告结构与证据 ID 体系，深检结果可以直接回填初筛结论，避免两套口径并存。

反沙箱对抗的识别可以用一句话概括：样本长时间空转、确认无监控痕迹后才释放行为，行为报告表现为**低活跃度后的延迟爆发**。对策是多次随机延时运行并差异化环境指纹，交叉比对多次运行的活跃度曲线，抬高对抗成本。

## 11.5 二进制逆向辅助

逆向辅助中 LLM 的定位是**信息压缩与线索提取**。具体场景包括：反汇编伪代码解读，把冗长伪代码压缩为函数级摘要；字符串与常量语义解释，识别 magic 常量、

错误码与配置项的含义；加密算法识别中的辅助定位，通过轮常数与查表结构提示候选算法，再交由规则确认。模型输出一律标注为假设。

人机分工的边界必须清晰：逆向的核心推理仍是人，控制流重构、语义还原与漏洞判据的建立由分析员完成，采纳与否由人决定。符号执行与去混淆提供**概念级辅助**：模型解释约束求解得出的路径含义、为去混淆脚本提供模式建议，但不直接执行对样本的高危改写，改写脚本同样走审批与白名单约束。

## 11.6 取证衔接与证据固定

本章所有产出都要接入第 10 章建立的审计体系。**哈希固定**：样本与流量证据入库即计算 SHA-256，证据 ID 与哈希写入审计哈希链，后续任何分析引用不可变的证据 ID，保证报告结论可回放、可质证。**时间线归一**：流量时间戳、沙箱事件时间与静态分析时间统一到 UTC，对齐到统一时间线，支撑跨证据的因果叙述与溯源复盘。

## 11.7 本章速记

- PCAP 管线五段全部由确定性代码完成，结论可复现、可质证。
- 协议统计既提供全局视野，也驱动深度解析的采样决策。
- 流重建按五元组聚合，TCP 乱序与重传在重组层处理。
- 资产提取产出 IP、域名、证书互链的资产图谱。
- Beacon 检测以 CV=σ/μ 度量间隔稳定性，值越小周期性越强。
- CV 阈值、最小样本数与长基线窗口三项叠加控制误报。
- 检测输出必须携带阈值与样本量，供报告复现判定。
- JA3/JA4 的指纹思想是对握手字段规范化串联后哈希。
- 加密流分析不解密，回答谁、用什么栈、什么节奏三个问题。
- LLM 只消费带证据 ID 的中间产物，不接触原始报文。
- LLM 结论必须引用证据 ID，无引用叙述在报告层剔除。
- 模糊哈希用于家族相似性聚类，精确哈希用于情报比对。
- 高熵节与节属性的异常组合是加壳的典型静态形态。
- 静态结论永远标注疑似，必须与动态行为互相印证。

- 动态沙箱一次性使用，internal 网络、只读根、超时强杀三件套。
- 行为报告按四类事件序列输出，与静态分析共用证据 ID。
- 容器级沙箱做初筛，API 级沙箱承接深检，两层共享报告结构。
- 反沙箱识别一句话：低活跃度后的延迟爆发即对抗信号。
- 逆向核心推理仍是人，LLM 只做信息压缩与线索提取。
- 证据哈希固定加时间线归一，使流量与样本结论可回放。

# 第 12 章 能力子系统 III：检测、溯源、推演与对抗仿真

本章覆盖智能体的防御侧能力集群：入侵检测规则生成、攻击链路溯源、横向移动路径推演、红蓝对抗仿真、防御策略生成与取证分析。这些能力的共同特征是**输出服务于防御者**——即使建模的是攻击行为，产出也始终是检测规则、加固清单与响应预案。边界声明：路径推演与仿真只做建模计算与沙箱靶场内的受控验证，绝不构成对真实环境的攻击执行。

## 12.1 入侵检测规则生成

### 12.1.1 从发现到规则的自动草稿

评估发现是检测规则的天然输入源：beacon 检测发现（目标 IP、周期、频次）可以直接映射为网络检测规则的字段。映射关系设计为**字段级模板**而非自由生成：发现的 target 填 dst_ip、count 决定聚合阈值、tactic 决定规则标签——LLM 只负责生成人类可读的描述与调优建议，结构字段由代码从发现对象提取。这条纪律的意义：检测规则是会进生产的代码，其结构化部分必须可追溯、可测试，LLM 的生成性输出只出现在注释与描述层。

```
title: Beacon to 198.51.100.9
status: draft                # 草稿态：人工评审 + 实测前不生效
logsource:
    category: firewall
detection:
    selection:
        dst_ip: '198.51.100.9'
    condition: selection | count() > 50
tags:
    - attack.command_and_control   # 映射自 finding.attack_tactics
falsepositives:
    - 合法周期性客户端（需业务确认）
```

### 12.1.2 规则的工程纪律

生成规则必须过四道关：**可执行性**（日志源存在且字段可得——对没有防火墙日志的环境生成防火墙规则是空气拳）、**误报预估**（规则说明 falsepositives 并要求业务确认）、**实测**（在评测集或靶场回放验证能命中）、**灰度上线**（先告警不阻

断）。规则生成后进入第 11 章手册的检测规则生命周期（测试→调优→复审），智能体的产出只是"草稿态"，状态字段防止未验证规则直接生效。

## 12.2 攻击链路溯源

溯源的难点不是缺数据而是**线索归并与假设检验**。Aegis 的溯源回路：**时间线归并**（审计日志、PCAP 证据、发现时间统一时钟对齐，构建按序事件表）→ **假设生成**（LLM 基于事件序列提出攻击假设："外联 C 后出现的加密流与凭据读取在时间上强关联，疑似 C2 后横向"）→ **证据检验**（代码验证假设引用的每个证据是否存在、时间是否自洽、哈希是否匹配——LLM 的假设必须引用证据 ID）→ **叙事输出**（通过检验的假设组装成映射 ATT&CK 战术序列的攻击链）。每条攻击链携带置信度标注：证据充分的标 confirmed、单一证据的标 single-source、推理链条的标 inferred——溯源报告的纪律与评估报告一致：不确定性必须显式表达。

## 12.3 横向移动路径推演

推演是纯计算能力（第 6 章的数学建模落地）：攻击图 G=(V,E) 上节点为主机（携带 CVSS 映射的攻陷概率与资产价值），边为可达利用关系；log 域 Dijkstra 求入口到各高价值目标的最大成功概率路径；风险 = 概率 × 价值权重（core=10）排序输出。**推演产出的是加固建议**：路径上的关键跳分段、最小权限收敛、凭据保护——每条路径 finding 的 recommendation 字段直接指向防御动作。与 BloodHound 类工具的关系一句话：它们枚举真实域内关系（数据来自活动目录），Aegis 的推演在评估产出的攻击图上做概率决策，两者可以互补（把域关系图作为边的来源之一）。

## 12.4 红蓝对抗仿真

### 12.4.1 回合制仿真状态机

仿真在隔离靶场（docker-compose 编排的靶机网络）内以回合制推进：**红队回合**——红队 Agent 从 TTP 剧本库（映射 ATT&CK 编号）选择下一动作，动作执行限定在靶场内；**蓝队回合**——蓝队 Agent 消费仿真产生的日志与告警，输出研判与处置建议（隔离、封禁、升级）；**裁判评分**——代码化裁判按三个维度记账：红队动作是否达成子目标（攻陷判定）、蓝队是否在时限内检出（MTTD 仿真）、处置建议

是否正确。整个仿真是一张 LangGraph 子图：红蓝节点交替、裁判节点条件路由（达成最终目标或回合数耗尽则终止）。

### 12.4.2 仿真的产出与价值

仿真的产出不是"谁赢了"，而是三张清单：**检测缺口**（红队执行了哪些 TTP 蓝队没有检出——直接生成检测规则草稿进入 12.1 的流程）、**响应链断点**（检出后处置建议慢或不准确的环节）、**度量基线**（仿真 MTTD 与处置准确率，作为演练改进的回归基线）。价值排序：一次仿真发现一百个问题，不如建立每月自动仿真发现十个问题的机制——仿真是运营节奏器，不是一次性表演。

## 12.5 防御策略生成

策略生成是全链路的收口：把发现、路径推演与检测缺口聚合成**优先级排序的加固清单**。排序依据第 6 章的风险量化：R = P(compromise) × Impact × 价值权重，再除以修复成本得性价比排序。清单以机器可读 yaml 输出（每条：风险项、依据证据 ID、建议动作、预期风险下降、验证方式），人工采纳后进入修复流程，修复完成触发**重评**（同目标复跑关键节点）验证风险确实下降——策略生成的反馈闭环使安全建设可度量。LLM 在这一步的角色是表述：把技术动作翻译成面向管理层的风险语言（第 18 章手册的向上管理技术）。

## 12.6 取证分析

取证能力的四条纪律：**证据固定**（内存先于磁盘、只读挂载、操作时间线记录——易失性顺序不可颠倒）；**哈希链**（每个证据对象计算哈希并记录，任何后续分析引用哈希而非路径，防篡改与误引）；**时间线还原**（多源日志统一时钟——时区偏差是取证第一大坑——归并成统一事件表）；**报告引用完整性**（结论必须引用证据哈希，LLM 的归纳只能出现在假设与解读层，事实层全部来自代码提取）。取证与第 11 章的样本分析共享沙箱与哈希基础设施，与第 13 章的审计日志共享不可篡改设计。

## 12.7 本章速记

- 防御侧能力集群的共同特征：建模攻击行为、产出防御资产

- 检测规则的结构字段由代码从发现提取，LLM 只写描述层——规则是代码
- 规则四道关：可执行性、误报预估、实测、灰度；草稿态字段防未验证生效
- 溯源回路：时间线归并、LLM 提假设、代码验证据、叙事输出
- 溯源结论必须携带置信度：confirmed、single-source、inferred
- 路径推演是 log 域 Dijkstra 的概率计算，产出是加固建议不是攻击执行
- 仿真是回合制子图：红队动作、蓝队研判、裁判记账，全在隔离靶场
- 仿真产出三清单：检测缺口、响应断点、度量基线；机制比一次表演值钱
- 策略生成 = 风险排序加性价比，机器可读 yaml 加人工采纳加重评闭环
- 取证四纪律：证据固定顺序、哈希链、时钟归一、引用完整性
- 事实层全部来自代码提取，LLM 只进假设与解读层

# 第 13 章 安全架构：权限隔离、沙箱、熔断与审计

自主智能体拿着模型 API、工具权限与目标访问能力，它的安全架构不是"附加特性"而是存在前提。本章讲 Aegis 的安全四件套——授权门、熔断器、审计日志、沙箱——如何组成纵深防御，每一层的失效假设是什么，以及为什么这套设计能让人敢于按下运行键。

## 13.1 多级权限隔离模型

### 13.1.1 三层角色与凭据分离

权限模型的三个层次：**编排层**（LangGraph 进程）持有模型 API 凭据与任务状态不直接接触目标网络；**工具层**（插件进程）持有目标访问能力，但每次调用必须过授权门；**沙箱层**（一次性容器）持有对目标的实际动作执行，网络被限制在隔离网段。**凭据分离**是关键纪律：模型 API key 与目标凭据（扫描账号、SSH key）存放在不同的配置通道，工具层拿不到模型凭据、编排层拿不到目标凭据——任一层被攻破，攻击者拿到的都是半张地图。

**任务级 ROE 注入**：每个评估任务携带独立的授权上下文（targets、allowed_actions、频次上限），授权门实例按任务创建——长期运行的服务不会累积跨任务的授权残留。**最小暴露**：一次任务只注册必需的插件（做 PCAP 分析不需要挂载 POC 验证工具），暴露面随任务收放。

### 13.1.2 绕过即缺陷的强制通道

ToolBus 是所有工具调用的唯一通道：授权门 → 熔断器 → 审计日志三件套在总线内串联执行，任何插件无法绕过。这条纪律的代价是调用链多一跳，收益是**安全性质的全局可证明**：审计日志中必然存在每一次工具调用的记录，因为不存在第二条调用路径。代码评审把"绕过总线直连工具"列为与 SQL 注入同级的缺陷——架构不变量靠评审守护。

## 13.2 隔离沙箱的七道防线

POC 验证与样本分析在一次性容器内执行，七道防线逐条讲透：

```
sandbox:
  networks: [aegis_sandbox]      # 1. internal 网络：无外网出口
  read_only: true                # 2. 只读根文件系统
  tmpfs: ["/tmp:rw,size=64m"]    # 3. 临时写区：容器销毁即焚
  mem_limit: 512m                # 4. 内存限额（cgroup）
  pids_limit: 128                # 5. PID 限额：防 fork 炸弹
  cap_drop: [ALL]                # 6. 移除全部 Linux capabilities
  security_opt: [no-new-privileges]  # 7. 禁止提权
```

**internal 网络**是第一道也是最常被省略的一道：样本与 POC 的出网行为（C2 回连、数据外带）在网络层直接不可达，而不是靠样本"自觉"。**只读根 + tmpfs** 保证文件系统变更即焚——样本落盘的任何持久化尝试随容器销毁消失。**cap_drop ALL** 移除容器内全部特权能力（挂载、改网络、加载内核模块在容器内不可行）。**超时与进程组终止**：执行包装器在超时后 rm -f 强杀容器（不是优雅等待），防失控长任务。

**逃逸的现实风险**要诚实面对：容器共享宿主内核，内核漏洞是逃逸面；资源耗尽攻击可以打满宿主。缓解三件：宿主机及时打补丁、容器运行在独立宿主机上（与业务隔离的爆炸半径控制）、严格资源限额把最坏情况约束在可承受范围。沙箱不是绝对堡垒，是**把最坏情况的爆炸半径压到可承受**的工程手段。

## 13.3 熔断机制：三态与预算

### 13.3.1 工具级三态熔断

熔断器与电子断路器同构：**CLOSED** 正常放行并滚动统计失败；连续失败达阈值（默认 5 次）转 **OPEN**，冷却期内直接拒绝；冷却结束转 **HALF_OPEN** 放行一次探测成功恢复 CLOSED、失败重回 OPEN。按工具名隔离：一个工具故障只熔断自己，侦察失败不拖垮报告生成。熔断的价值在自主循环里尤其大：没有熔断的 Agent 会对着一个挂掉的工具无限重试，把 Token 预算烧在同一个坑里——熔断器本质上是给 LLM 的"不要在同一块石头上绊倒"装上了机械 enforcement。

### 13.3.2 预算熔断与降级路径

第二类熔断是预算：Token 总量与工具调用次数任一超限即全局拒绝新调用。预算耗尽后的行为是设计好的：编排层走降级路径——停止探索、汇总已有发现、生

成部分报告并标注"预算受限"。**有损优于失控**：一个 80% 完成度且诚实标注的报告，远好于一个 100% 完成度但烧穿了预算且无人知晓的报告。半开探测让熔断器具备自愈：不是永动拒绝，而是冷却后给一次机会验证恢复。

## 13.4 审计日志：append-only 与可回放

### 13.4.1 记录设计与不可篡改

审计记录八字段：时间戳、执行者（哪个节点）、动作、目标、参数、结果、细节哈希（SHA-256 截断，控制体积）、耗时。append-only JSONL：只增不改，回放接口按 actor/action 过滤——一次评估的全部行为可以在事后完整重演。细节哈希的设计动机：审计日志里存全量扫描输出会膨胀到不可用，存摘要又怕丢证据——折中是存哈希，原始输出按需可从证据存储核对（哈希一致即未篡改）。**外部不可篡改**的进一步增强：日志实时外发到 WORM 存储（一次写入多次读取）或 SIEM，本地被清了远端还在——这与第 10 章手册的"日志外发对抗痕迹清理"同构。

### 13.4.2 审批记录也是审计

人工审批本身进入审计流：谁在什么时间批准了哪个高危动作、当时看到的参数是什么。这一条在合规审查中的价值极高——出了事故可以完整还原"机器申请了什么、人批了什么、依据是什么"，责任链清晰。拒绝路径同样记录：审批被拒后 Agent 的降级行为（跳过 POC、标注待验证）有迹可循。

## 13.5 纵深防御总图

从提示注入到沙箱，防线逐层递进，每层都有明确的失效假设：

| 防线 | 拦什么 | 失效假设 |
| --- | --- | --- |
| ROE 授权门 | 越界目标与动作 | ROE 文件本身不被篡改（文件完整性监控） |
| 提示词约束 | 引导性注入 | 假设必然失效（软约束），只作第一道滤网 |
| 工具白名单 | 幻觉调用不存在的工具 | server 不被投毒（第 8 章来源审计） |

| 防线 | 拦什么 | 失效假设 |
| --- | --- | --- |
| 人工审批 | 高危动作的最终决定权 | 审批人被误导（展示完整参数与依据） |
| 沙箱 | 动作的实际爆炸半径 | 内核无已知逃逸漏洞（打补丁+独立宿主） |
| 审计日志 | 事后追责与异常发现 | 日志通道可用（实时外发） |

这张表是整个安全架构的设计哲学：**每一层都假设上一层会被突破**。提示词会被注入、审批可能被误导、但要让一次恶意成功穿透全部六层，攻击者需要在每个环节都赢——纵深防御的意义不是单层完美，而是串联概率的指数衰减。

## 13.6 本章速记

- 三层角色编排/工具/沙箱，凭据分离：任一层被攻破只得半张地图
- 任务级 ROE 注入：授权上下文按任务创建，不跨任务残留
- ToolBus 唯一通道：绕过总线直连工具是与注入同级的评审缺陷
- 沙箱七道防线：internal 网络、只读根、tmpfs、内存与 PID 限额、cap-drop、no-new-privileges、超时强杀
- internal 网络是最常被省略的一道：出网不可达优于行为不自觉
- 沙箱不是绝对堡垒：把最坏爆炸半径压到可承受是工程目标
- 熔断三态 CLOSED/OPEN/HALF_OPEN，按工具隔离防单点拖垮全局
- 预算熔断的降级路径：停止探索、诚实标注、出部分报告
- 有损优于失控：80% 且诚实的报告好于 100% 且失控
- 审计八字段加细节哈希：摘要控体积、哈希可核对
- 实时外发 WORM 或 SIEM：本地被清远端还在
- 审批记录本身入审计：机器申请、人批什么、依据什么，责任链完整
- 纵深防御哲学：每层假设上层被突破，串联概率指数衰减

# 第 14 章 记忆系统与幻觉抑制

自主智能体的两个长期软肋：没有记忆则每次任务从零开始，有记忆则可能被错误经验污染；LLM 会自信地编造，而安全场景里一句编造的漏洞结论就是一次虚惊或一次漏报。本章把这两件事合成一个工程问题：**让智能体记住正确的、忘记错误的、只对有证据的事情表态**。

## 14.1 记忆分层模型

四层记忆按生命周期与用途划分：**工作记忆**就是状态机上下文（AgentState），任务结束即消亡，容量受窗口约束；**情景记忆**存任务轨迹与结果（做了什么、发现了什么、哪里失败），按任务粒度索引，供回溯与审计；**语义记忆**存提炼后的模式与教训——Reflexion 的反思产物、验证过的漏洞模式、某类目标的通用结论——这是复利的主体；**程序性记忆**是验证模板与提示词库（怎么验证某类漏洞、怎么写某类规则），以配置文件形式版本化管理。四层的存储介质不同：工作记忆在内存、情景记忆在检查点与向量库、语义记忆在向量库加结构化索引、程序性记忆在代码仓库。

**长期向量记忆的工程实现**（与代码对应）：写入时机三个——任务结束写摘要、反思产生写教训、发现确认写模式；条目结构 MemoryItem（id、kind、text、meta），kind 区分 task_summary、reflection、finding、poc_note；检索时机在 planner 之前的 memory_recall 节点（把相关历史注入规划上下文）；双后端 local（NumPy 余弦，零依赖，测试默认）与 chroma（持久化，生产可选）。

## 14.2 记忆的污染与遗忘

记忆不是越多越好，**错误教训的污染**是最危险的失败模式：Reflexion 幻觉出一教训（"扫描超时是因为目标开了 WAF"——其实是网络抖动），这条教训被存入长期记忆并在后续任务中反复注入 planner，一次错误归因被放大成系统性偏见。三道防线：**置信度标注**（记忆条目带来源与验证状态，未验证教训降权）、**冲突消解**（新证据覆盖旧结论时记录覆盖历史而非静默替换）、**衰减与清理**（长期未命中的记忆降权，人工审计接口支持清理——记忆审计也是审计）。记忆的检索命中率要进度量：从不被检索到的记忆是死重，污染检索的记忆是毒药，都需要定期治理。

## 14.3 幻觉的来源分析

抑制的前提是知道幻觉从哪来，四类来源在安全场景的具体形态：**模型内在**——参数化知识的模糊性与过时（编造不存在的 CVE 编号、张冠李戴的组件版本）；**上下文压力**——长上下文中部遗忘（任务约束埋在 2 万 token 前，执行到第 15 轮时"忘了"ROE）、指令遗忘（系统提示的目标被过程细节淹没）；**观察噪声**——扫描器误报被当真、正则匹配的巧合被解读为漏洞；**推理漂移**——多步累积误差（第 3 步的假设在第 10 步已被当成事实引用）。安全场景的特殊放大器：安全工具输出天然充满"可疑"信号，模型对可疑信号的过度解读倾向极强。

## 14.4 幻觉抑制七方案

### 14.4.1 从约束到文化的七层方案

**一、结构化输出约束**：受限解码下模型只能产出 schema 合法的 JSON——枚举字段不存在幻觉值，这是最便宜且最可靠的一层（severity 只能是五级之一，由 schema 保证而非提示词恳求）。**二、证据链绑定**：报告的每条高危发现必须携带 evidence_id，证据对象哈希固定可核对——"无证据不报告"是数据结构层面的约束（字段缺失即校验失败），不是叙事层面的美德。**三、外部验证**：情报候选（CVE、威胁情报命中）一律视为假设，经沙箱验证器产出三态结果（confirmed、refuted、inconclusive）才允许改变发现的状态——验证器是独立代码，不接受模型断言。**四、多层自省**：步内自检（工具结果与预期比对）、轨迹级 Reflexion、评审团——注意自省本身也会幻觉（编造看似合理的教训），所以自省产物同样要过证据检验。

**五、多模型协同评审**：评审团与执行模型解耦（不同模型避免同源偏见），独立打分后去极值均值（抗单模型偏见），一票否决机制（两个评审 reject 即拒绝）。工程细节：评审输入是结构化发现而非自由文本（可比对），评审输出强制 schema（score、issues、must_verify、verdict），单评审失败不阻塞（记录降级）。**六、检索接地**：断言锚定检索到的文档与情报（RAG），引用溯源到对象 ID——让模型说"文档说 X"而不是"我认为 X"。**七、不确定性显式化**：置信度分级（confirmed、probable、unverified）、待验证状态、报告中的局限声明——诚实的不确定性是可用的，自信的编造是致命的。

```
def aggregate_reviews(reviews: list[dict]) -> dict:
    scores = [r["score"] for r in reviews if "score" in r]
    trimmed = scores[1:-1] if len(scores) > 2 else scores   # 去极值
    verdicts = [r["verdict"] for r in reviews]
    return {
        "score": round(statistics.fmean(trimmed), 1) if trimmed else None,
        "verdict": ("reject" if verdicts.count("reject") >= 2
                    else "accept" if verdicts.count("accept") > len(verdicts)
/ 2
                    else "revise"),
    }
```

### 14.4.2 幻觉率的度量

抑制必须有度量否则是玄学：**无证据断言占比**（报告中无 evidence_id 的高危发现比例，目标为零）、**评审推翻率**（评审团 revise 与 reject 的比例，持续偏高说明执行层质量差或评审校准差）、**golden set 回归**（固定一批已知答案的评估场景，每次模型或提示词变更后重跑，幻觉率与召回率进回归门禁）。校准检查：让评审模型对已知对错样本打分，看分数与事实的相关性——校准差的评审团比没有评审团更糟（虚假的安全感）。

## 14.5 本章速记

- 记忆四层：工作、情景、语义、程序性；复利主体是语义记忆
- 写入三时机：任务结束、反思产生、发现确认；检索在 planner 前
- 记忆污染是最危险失败模式：一次错误归因被放大成系统性偏见
- 三道防线：置信度标注、冲突消解留痕、衰减与人工清理
- 幻觉四来源：模型内在、上下文压力、观察噪声、推理漂移
- 安全场景放大器：工具输出的可疑信号诱发过度解读
- 七层抑制：结构化约束、证据链、外部验证、多层自省、评审团、检索接地、不确定性显式化
- 结构化约束最便宜最可靠：schema 保证的不可能幻觉
- 无证据不报告是数据结构约束，不是叙事美德
- 评审团与执行模型解耦：去极值均值加一票否决
- 自省产物也要过证据检验：反思本身会幻觉

- 幻觉率三度量：无证据断言占比、评审推翻率、golden set 回归
- 校准差的评审团比没有更糟：虚假安全感是毒药

# 第 15 章 工程体系与全套开发流程

本章把前面所有设计收拢为可执行的工程体系：项目结构、并发调度、异常回滚、失败回溯重规划、插件热加载，以及从零到生产的九步开发流程与 CI 门禁。本章与开源实现 aegis-agent 严格对应——每个工程决策都能在代码里找到实体。

## 15.1 项目结构总览

```
aegis-agent/
├── src/secagent/
│   ├── config.py          # 配置：模型分层路由/预算/沙箱/记忆
│   ├── main.py            # CLI：run / approve / replay
│   ├── graph/             # state.py 状态与 reducer、nodes.py 节点工厂、
│   │                      # builder.py 图组装（interrupt + checkpointer）
│   ├── safety/            # ★ authorization / circuit_breaker / audit /
sandbox
│   ├── tools/             # registry 工具总线 + plugins 插件集群
│   ├── llm/               # 分层路由 + FakeLLM + multi_model 评审
│   └── memory/            # 长期向量记忆（local / chroma）
├── tests/                 # 离线测试：单测 + 全图 e2e + 中断审批
├── deploy/                # Dockerfile + 隔离拓扑 compose + 沙箱镜像
└── docs/                  # 设计文档 + 制作指南 + 使用手册
```

分层职责一句话：safety 是宪法、tools 是手脚、graph 是神经、llm 是大脑、memory 是经验。依赖单向：graph 调 tools 必穿 safety，反向依赖即架构缺陷。

## 15.2 并发调度：Send 扇出与资源控制

**动态扇出**：analysis_router 按任务字段返回 Send 列表（PCAP 分析、路径推演、蓝队仿真并行），LangGraph 的 BSP 超步保证各分支在同一步内并行执行、在 critic 汇聚点 barrier 等待全部分支完成。**分支上下文裁剪**：Send 携带的子状态按需裁剪（PCAP 分支不需要完整 findings 历史），既是 Token 经济也是隔离——分支间不共享工作上下文，结果经 reducer 合并回主状态。**并发度控制**：分支数受任务字段约束天然有限，若引入动态目标列表的大规模扇出（一次扫一百台主机），需要在路由

层加信号量与分批 Send——并发是收益也是负载，沙箱宿主机的资源上限决定扇出上限。

## 15.3 异常回滚与断点续跑

异常处理三层：**节点级**——每个工具调用 try/except 降级（单目标失败写错误证据继续主流程，这与全局失败不同）；**流程级**——熔断器拦截连续失败，预算耗尽触发降级出报告；**状态级**——checkpointer 每超步落库，任何时点可 time-travel 回上一稳定点重放（修复了环境问题后从断点续跑而不是从头再来）。断点续跑与人工审批共用同一机制：interrupt 挂起也是状态快照，approve 后恢复执行——**中断、审批、回滚、续跑是同一个状态管理能力的四个视角**。

## 15.4 失败回溯重规划

Reflexion 回环的工程约束四条：教训进入 planner 的方式是注入上下文（reflections 列表追加，planner 提示词携带最近三条）；**双重上限**防发散（max_iterations=25、max_reflections=4，任一触顶即退出回环出报告）；**改进停滞检测**（连续两轮 critique 报告同一缺陷——说明缺陷不可自愈，回环无益，转人工）**教训的证据检验**（反思产出的归因必须引用具体证据，防止幻觉教训污染长期记忆——与第 14 章呼应）。重规划的边界纪律：反思可以改变技术路径（换扫描策略、调整顺序），**不能改变授权边界**——planner 提示词明确声明 ROE 约束不可协商，授权门在代码层兜底。

## 15.5 插件热加载与版本管理

热加载三件事：进程隔离（每插件独立进程，新版本健康检查通过后切换，旧进程退出——崩溃不传染编排层）、配置驱动（plugins.yaml 声明清单与版本，改配置即生效）、能力变更通知（工具列表变化推送触发熔断器重建）。版本管理的纪律：工具 schema 变更视为破坏性变更（模型调用行为依赖描述文本），走评审；工具描述变更视为高危事件需人工确认（防 rug pull，第 8 章威胁模型）。ToolBus 门面保持稳定——从进程内注册演进为 MCP server 进程，上层节点代码零改动。

## 15.6 从 0 到 1 的九步开发流程

| 步骤 | 内容 | 完成标准 |
| --- | --- | --- |
| 1 | safety 三件套（授权门/熔断/审计） | 越界拦截与审计落盘的单测全绿 |
| 2 | graph/state 状态与 reducer | 合并交换律结合律测试通过 |
| 3 | tools 工具总线 + 首批插件 | 总线审计记录完整、fake 实现可测 |
| 4 | graph/nodes 节点工厂 | 每节点纯函数可独立调用测试 |
| 5 | graph/builder 图组装 | 离线 FakeLLM 全图 e2e 出报告 |
| 6 | 中断审批链路 | interrupt 挂起、approve 恢复、拒绝路径测试 |
| 7 | 真实模型接入与分层路由 | 评审团输出校验、用量统计进预算 |
| 8 | memory 长期记忆 | 写入检索 roundtrip、kind 过滤 |
| 9 | deploy 沙箱拓扑 | internal 网络验证、只读与限额验证 |

顺序的设计逻辑：**安全组件最先、离线 e2e 最早**——第 5 步的全图测试是整个项目的安全网，此后每次改动都在安全网内迭代。真实模型最后接入：确保自动化行为不依赖模型也能被完整验证。

## 15.7 测试策略与 CI 门禁

测试四层：**单测**（授权门越界矩阵、熔断三态、reducer 律、插件确定性）；**全图 e2e**（FakeLLM 脚本驱动主链路、扇出收敛、中断审批恢复——全部离线确定性可重放）；**打桩**（网络类工具 monkeypatch 固定行为，测试不因外网抖动而红）；**评测集**（golden set 场景跑幻觉率与检索质量，模型或提示词变更必须过回归）。CI 门

禁五件：lint 与类型检查、pytest 全绿、ROE 文件变更强制人工评审（改授权即改宪法）、镜像构建与签名、评测集回归不劣化。上线检查清单见 docs/BUILD_GUIDE.md 第 6 节。

## 15.8 已知边界缺陷清单

诚实列出当前实现的边界（这是工程文档的义务）：**LLM 判断不可靠**——规划与解读仍需人审，系统定位是人机协同而非无人值守；**CVSS 到概率映射粗糙**——分段函数是工程校准不是理论最优，需红队命中率数据回归修正；**沙箱非绝对**——内核共享的逃逸面永远存在，靠独立宿主与版本更新缓解；**长任务漂移**——超长轨迹的指令遗忘尚未根治，靠上下文裁剪与迭代上限缓解；**CPE 对齐误差**——指纹粒度不足时的候选误报，靠多信号确认与人工复核缓解；**评测集覆盖有限**——幻觉率度量只覆盖已建模场景。每条缺陷都有对应的缓解现状与监控指标——**知道系统在哪里不可靠，与知道它在哪里可靠同样重要**。

## 15.9 本章速记

- 分层依赖单向：graph 调 tools 必穿 safety，反向依赖即架构缺陷
- Send 扇出配上下文裁剪：既是 Token 经济也是分支隔离
- 中断、审批、回滚、续跑是同一状态管理能力的四个视角
- 节点级降级、流程级熔断、状态级 time-travel，异常三层处理
- 回环四约束：教训注入、双重上限、停滞检测、证据检验
- 反思可以改技术路径，不能改授权边界：ROE 不可协商
- 热加载三件事：进程隔离、配置驱动、变更通知；描述变更需人审
- 九步流程：安全最先、e2e 最早、真实模型最后
- 全图离线 e2e 是项目安全网：自动化行为不依赖模型可验证
- 测试四层：单测、e2e、打桩、评测集；网络工具一律打桩
- CI 五门禁：lint、pytest、ROE 评审、镜像签名、评测回归
- 六条已知缺陷全部带缓解现状与监控：诚实标注边界是工程义务

# 第 16 章 横向对比与选型：五类 Agent 体系

把 Aegis 放到 2026 年的 Agent 生态里定位，需要对比五类体系：**LangGraph**（代码优先的图编排框架，Aegis 的直接底座）、**低代码 LLM 应用平台**（以 Dify 为代表，用户提问中的 "DSH" 按此对应解读）、**Coze**（面向对话 Bot 的托管平台）、**OpenClaw**（开源本地优先的个人 Agent 框架）、**终端 CLI Agent Harness**（以 ZCode 为代表的终端智能体套件）。需要先声明一个阅读前提：这个领域的产品命名与形态迭代极快，本章对比的是**架构类**的能力与取舍——产品名会过时，架构取舍不会。

## 16.1 五类体系的架构画像

| 体系 | 编排模型 | 扩展模型 | 部署形态 | 最适合 |
|---|---|---|---|---|
| LangGraph | 显式状态图、BSP 超步 | 代码节点与 reducer | 自托管库 | 生产级复杂控制流 |
| 低代码平台（Dify 类） | 可视化工作流 | 插件与 API 节点 | 自托管或云 | 快速构建标准 LLM 应用 |
| Coze | Bot 编排（提示词+插件+知识库） | 商店插件 | 托管云 | 面向 C 端对话产品 |
| OpenClaw 类本地 Agent | 单主循环加工具调用 | 插件与消息通道 | 本地设备常驻 | 个人助理与隐私场景 |
| 终端 CLI Harness（ZCode 类） | 会话式 ReAct 循环 | 工具/技能/权限模式 | 开发者终端 | 软件工程与人机结对 |

五类体系的底层机制其实同源：都是「模型 + 工具调用 + 上下文管理」的变体，差异集中在**控制流的表达方式**与**信任边界的设计**上。逐个展开关键差异。

## 16.2 LangGraph：控制流的极限表达

LangGraph 的本质是把控制流从模型手里收回来一部分：显式的状态图、确定性的路由函数、可证明的并行合并语义（reducer 交换律）、可中断可回放的检查点。这四样恰好是安全 Agent 的刚需——授权门必须在确定性代码路径上、高危动作必

须可中断、审计必须可回放。代价是开发成本最高：每个节点是纯函数、每条边要显式声明、状态 schema 要预先设计。**当控制流的价值高于迭代速度时选它**——安全金融、医疗这类"错了有后果"的领域，以及需要人机协同审批（interrupt）的场景，LangGraph 几乎是唯一严肃选项。弱项：对话式闲聊类应用用它是杀鸡用牛刀；图定义的调试体验仍不如直接写代码直观。

## 16.3 低代码平台（Dify 类）与 Coze：速度换控制

低代码平台的架构画像：可视化工作流编排（节点=提示词/知识库检索/API/条件分支）、内置 RAG 管线、插件市场、开箱即用的观测。它的价值区间是**标准 LLM 应用**——客服、知识库问答、内容生成——三天上线的速度优势是代码框架给不了的。放到安全 Agent 的语境下短板立刻显现：工作流的控制表达弱于图编排（复杂循环、动态扇出、人工中断的语义支持有限）；审批与审计的定制深度不够（安全四件套要嵌进平台插件体系而非一等公民）；权限模型面向"应用用户"而非"任务级 ROE"。Coze 的定位更偏 C 端 Bot：商店化的插件与知识库、托管部署、面向对话产品的分发——它的强项是触达终端用户，弱项与低代码平台同构，且托管形态让"数据不出域"的合规要求更难满足。**选型逻辑一句话：应用是标准形状就选平台省时间，控制流是核心资产就选代码框架。**

## 16.4 OpenClaw 类本地 Agent：个人主权与信任边界

OpenClaw 代表的本地优先个人 Agent 架构（开源社区从 Clawdbot/Moltbot 一脉演化而来）：Agent 常驻在用户自己的设备上，接入即时通讯渠道（WhatsApp/Telegram 等），以工具调用完成日常事务托管，强调数据主权与个人定制。它的架构启示对安全 Agent 有两点：一是**信任边界的定义单位是"个人设备"**——所有凭据与记忆留在本地，这对安全 Agent 的"数据不出域"诉求是同构的解法；二是**常驻进程 + 多渠道入口**的形态让"审批"自然发生在用户已有的聊天界面里——人机协同的接口设计值得借鉴。风险面也同构：常驻高权限进程 + 天然连接外部的 Agent 是攻击者的高价值目标，会话劫持与提示注入的爆炸半径直接落在个人设备上——这类框架的安全性高度依赖通道加密与工具白名单的纪律。

## 16.5 终端 CLI Harness（ZCode 类）：结对范式的极致

以 ZCode 为代表的终端 Agent Harness 是另一种架构形态：Agent 以 CLI 会话存在，在开发者的终端与文件系统里以 ReAct 循环工作（读代码、跑命令、改文件），靠**权限模式**（只读/编辑确认/全自动）与**工具沙箱**控制风险，靠技能与插件体系扩展。它的核心洞察是**工作环境即上下文**：不搬运世界状态给模型，而是把 Agent 放进世界（终端、仓库、shell），用权限模式控制每一步的信任级别。对安全 Agent 的启示：Aegis 的沙箱审批链路与 CLI Harness 的权限模式同构（只读侦察免审、高危动作确认），而 CLI Harness 生态的"技能/插件"组织方式（把领域知识封装为可加载的技能文件）正是 Aegis 插件热加载的对偶物。弱项：会话式循环天然缺少多分支并发的结构化表达——这正是回到 LangGraph 的理由。

## 16.6 选型决策与演进

### 16.6.1 决策树

四个问题定选型：**控制流复杂吗**（动态扇出、人工中断、回环——是则代码框架）、**数据边界敏感吗**（安全/合规——是则自托管，排除托管云）、**用户是谁**（开发者选 CLI/Harness 形态、业务运营选低代码、个人选本地 Agent）、**错了的代价**（低代价选平台快迭代、高代价选图编排加纵深防御）。Aegis 的选择逻辑：安全评估四问全中高代价分支——LangGraph 做骨架、MCP 做工具协议、CLI Harness 的权限模式思想做审批 UX、本地 Agent 的数据主权原则做部署形态。

### 16.6.2 融合趋势

五类体系在互相吸收对方的优点：低代码平台在补"代码节点"（复杂逻辑下探到代码）；LangGraph 生态在补"托管与可视化"（LangSmith 观测、Studio 调试）；CLI Harness 在补"多 Agent 与后台任务"；本地 Agent 在补"团队协作"。2026 年的判断：**形态会继续分化，但能力底座在收敛**——模型接口标准化（OpenAI 兼容）、工具协议标准化（MCP 成为事实标准）、记忆与评测工程化。选型时押注这三层标准而不是某个产品形态，可以让系统在框架更替中存活——Aegis 把 ToolBus 门面与 LangGraph 隔离的设计正是为这个目的。

## 16.7 本章速记

- 五类体系同源异形：模型加工具加上下文，差异在控制流表达与信任边界
- LangGraph：显式状态图、确定性路由、可中断可回放，高代价场景唯一严肃选项
- 低代码平台三天上线标准应用，复杂循环与任务级权限是短板
- Coze 强在 C 端触达与商店生态，托管形态与数据不出域矛盾
- OpenClaw 类本地 Agent 的启示：信任边界以设备为单位、审批融入聊天界面
- 常驻高权限 Agent 是高价值攻击目标，通道加密与工具白名单是纪律
- CLI Harness 的洞察：把 Agent 放进工作环境，权限模式控制信任级别
- Aegis 选型四问全中高代价分支：LangGraph 骨架加 MCP 协议加权限模式 UX
- 押注三层标准：模型接口、工具协议（MCP）、记忆评测工程化
- 门面隔离（ToolBus 不暴露 LangGraph 类型）让框架更替可存活
- 产品名会过时，架构取舍不会：对比看控制流、信任边界、人机接口